



Universidad de Valladolid

Escuela Técnica Superior de Ingenieros de
Telecomunicación

Configuración y actualización dinámica de modelos de aprendizaje profundo

GRADO EN INGENIERÍA DE TECNOLOGÍAS DE LA TELECOMUNICACIÓN

TRABAJO DE FIN DE GRADO

Autor:
Andrés García Treviño

Tutor:
Dr. D. Pablo Casaseca de la Higuera

Valladolid, junio 2026

Resumen

El crecimiento exponencial del número de dispositivos capaces de generar tráfico de red ha aumentado significativamente la necesidad de organizar los recursos disponibles para asegurar un nivel estable de calidad de servicio. La predicción del tráfico de red se ha convertido en la mayor aliada para evitar congestiones y pérdidas de servicio. Sin embargo, los modelos predictivos tradicionales tienen muchas dificultades para seguir el ritmo de cambio del tráfico de red actual. Como el tráfico de red puede ser modelado como una secuencia temporal, en este trabajo se estudian y comparan varios modelos de redes neuronales de aprendizaje profundo aplicadas a la predicción de series temporales con capacidad de adaptarse a las variaciones y anomalías mediante entrenamiento incremental. Se han estudiado varias arquitecturas que combinan redes convolucionales con redes recurrentes LSTM sobre el dataset Milan, que recoge el tráfico de red de la ciudad de Milán, Italia, entre noviembre y diciembre de 2013.

Los resultados de los modelos, analizados a través de diferentes métricas estadísticas, demuestran que los modelos que integran capas convolucionales y *online learning* son ampliamente superiores a los modelos tradicionales, incluso en situaciones de cambios bruscos en el patrón de los datos. El modelo ideal, presentando una capa CNN y una capa LSTM llega a reducir el error (NRMSE) cometido en las predicciones desde un 0.1778 conseguido con un modelo neuronal LSTM sencillo hasta un 0.142. Por último, los modelos analizados en este trabajo han sido optimizados al máximo para conseguir el mayor rendimiento posible en cuanto a carga computacional y tiempo de ejecución.

Palabras clave

Online learning, predicción, redes neuronales, redes convolucionales, tráfico de red.

Abstract

The exponential growth of telecommunication devices has raised the need of correctly allocating the network resources to ensure a reliable Quality of Service level. Network traffic prediction has become the greatest ally in the battle against network congestion and loss of service. Nonetheless the traditional prediction models are burdened with great difficulties to follow the changes in network traffic. As it can be seen as a temporal sequence, this work puts several *online deep learning* models under the microscope to elucidate whether they are able to adapt dynamically to changes in the temporal series. Various distinct architectures featuring convolutional and LSTM layers have been tested against the Milan dataset, which holds information about the traffic load in the city of Milan, Italy, between November and December 2013.

The models' outputs have been analysed via different statistical methods, demonstrating the vast superiority of the models combining convolutional and recurrent layers over the more traditional models, even when the pattern abruptly changes. The best model so far, characterized by a single CNN layer connected to an LSTM layer has been able to boost precision, measured as NRMSE, from 0.1778 to 0.142. Last but not least, all the models scrutinized by this work have been successfully optimized to achieve the greatest possible computation speeds.

Keywords

Convolutional networks, network traffic, neural networks, online learning, prediction.

Agradecimientos

A mi familia, a mis amigos, a mis profesores y, sobre todo, al Dr. D. Juan Pablo Casaseca de la Higuera. A todos ellos les agradezco su infinita paciencia.

Índice

1. Introducción	7
1.1. Objetivos	8
1.2. Estructura del documento	8
2. Antecedentes y estado del arte	9
2.1. Predicción de series temporales	9
2.1.1. Modelos autorregresivos de medias móviles	9
2.2. Aprendizaje automático	11
2.2.1. Procedimiento de selección de modelos	11
2.3. <i>Deep learning</i>	13
2.4. El <i>Perceptron</i>	13
2.5. Entrenamiento de redes neuronales	14
2.6. Redes neuronales convolucionales	15
2.6.1. Mapas de filtros	15
2.6.2. Capas de agregación	16
2.7. Redes neuronales recurrentes	17
2.7.1. Neuronas LSTM	17
2.8. CNN+LSTM	19
2.8.1. TimeDistributed	19
2.8.2. Flatten	20
2.9. <i>Online learning</i>	20
3. Entorno experimental	22
3.1. Descripción de los datos	22
3.2. Arquitecturas empleadas	23
3.3. Procedimiento de entrenamiento	23

3.4. Hiperparámetros considerados	24
3.5. Métricas de rendimiento	25
3.6. Análisis de Spearman	26
3.7. Entorno de ejecución	27
4. Resultados y discusión	28
4.1. Desempeño general	28
4.2. Análisis de Spearman	29
4.2.1. Número de celdas, capas CNN y tamaño del <i>kernel</i>	30
4.2.2. <code>look_back</code> y <code>batch_size</code>	31
4.2.3. <code>updates</code> y <code>threshold</code>	33
4.2.4. Número de filtros de las capas convolucionales	34
4.3. Mejor modelo	34
5. Conclusiones y líneas futuras	38
Referencias	39
A. Fragmentos de código	43
A.1. Código Python del modelo original	43
A.2. Código Python del modelo LSTM adaptado para el entorno Kaggle .	49
A.3. Código Python de los modelos XCNN+LSTM adaptados para el entorno Kaggle	55
B. Representación de los datos	62
B.1. <i>Dataset</i> Milán	62
B.2. Predicciones del mejor modelo	73

Índice de figuras

1.	Campos de visión de dos neuronas convolucionales situadas en el mapa de filtros $k=2$ de la capa convolucional $l=2$ con <i>stride</i> =1. Elaboración propia.	16
2.	Representación de las puertas de una neurona LSTM [34].	18
3.	Representación de las distintas formas de <i>concept drift</i> . [37]	20
4.	Distribución de las medias de NRMSE de los modelos con <i>online learning</i> según el tamaño de la cuadrícula. Elaboración propia.	28
5.	Distribución de las medias de NRMSE de los modelos con <i>online learning</i> según la arquitectura del modelo. Elaboración propia.	29
6.	Densidad de las medias de NRMSE en entrenamiento estático (línea discontinua) y <i>online learning</i> (línea continua) según el tamaño de la cuadrícula y del kernel. Elaboración propia.	31
7.	Influencia del hiperparámetro <i>look_back</i> en la media del NRMSE y en el tiempo de ejecución medio en modelos con <i>online learning</i> . Elaboración propia.	32
8.	Influencia del hiperparámetro <i>updates</i> en la media del NRMSE y en el tiempo de ejecución medio en modelos con <i>online learning</i> . Elaboración propia.	34
9.	NRMSE por celda del mejor modelo con <i>online learning</i> . Elaboración propia.	36
10.	NRMSE por celda de los mejores modelos con <i>online learning</i> de cada arquitectura (LSTM, 2CNN+LSTM y 3CNN+LSTM). Elaboración propia.	36
11.	Datos contenidos en el <i>dataset</i> Milán para la celda 4861 [38].	62
12.	Datos contenidos en el <i>dataset</i> Milán para la celda 4862 [38].	63
13.	Datos contenidos en el <i>dataset</i> Milán para la celda 4863 [38].	63
14.	Datos contenidos en el <i>dataset</i> Milán para la celda 4959 [38].	64
15.	Datos contenidos en el <i>dataset</i> Milán para la celda 4961 [38].	64
16.	Datos contenidos en el <i>dataset</i> Milán para la celda 4962 [38].	65
17.	Datos contenidos en el <i>dataset</i> Milán para la celda 4963 [38].	65
18.	Datos contenidos en el <i>dataset</i> Milán para la celda 5057 [38].	66

19.	Datos contenidos en el <i>dataset</i> Milán para la celda 5058 [38].	66
20.	Datos contenidos en el <i>dataset</i> Milán para la celda 5060 [38].	67
21.	Datos contenidos en el <i>dataset</i> Milán para la celda 5061 [38].	67
22.	Datos contenidos en el <i>dataset</i> Milán para la celda 5062 [38].	68
23.	Datos contenidos en el <i>dataset</i> Milán para la celda 5063 [38].	68
24.	Datos contenidos en el <i>dataset</i> Milán para la celda 5162 [38].	69
25.	Datos contenidos en el <i>dataset</i> Milán para la celda 5163 [38].	69
26.	Datos contenidos en el <i>dataset</i> Milán para la celda 5257 [38].	70
27.	Datos contenidos en el <i>dataset</i> Milán para la celda 5258 [38].	70
28.	Datos contenidos en el <i>dataset</i> Milán para la celda 5262 [38].	71
29.	Datos contenidos en el <i>dataset</i> Milán para la celda 5263 [38].	71
30.	Datos contenidos en el <i>dataset</i> Milán para la celda 5357 [38].	72
31.	Datos contenidos en el <i>dataset</i> Milán para la celda 5358 [38].	72
32.	Datos contenidos en el <i>dataset</i> Milán para la celda 5362 [38].	73
33.	Datos contenidos en el <i>dataset</i> Milán para la celda 5363 [38].	73
34.	Predicciones del mejor modelo para la celda 4959. Elaboración propia.	74
35.	Predicciones del mejor modelo para la celda 4962. Elaboración propia.	74
36.	Predicciones del mejor modelo para la celda 5057. Elaboración propia.	74
37.	Predicciones del mejor modelo para la celda 5058. Elaboración propia.	75
38.	Predicciones del mejor modelo para la celda 5062. Elaboración propia.	75
39.	Predicciones del mejor modelo para la celda 5063. Elaboración propia.	75
40.	Predicciones del mejor modelo para la celda 5162. Elaboración propia.	76
41.	Predicciones del mejor modelo para la celda 5163. Elaboración propia.	76
42.	Predicciones del mejor modelo para la celda 5262. Elaboración propia.	76
43.	Predicciones del mejor modelo para la celda 5263. Elaboración propia.	77
44.	Predicciones del mejor modelo para la celda 5357. Elaboración propia.	77
45.	Predicciones del mejor modelo para la celda 5362. Elaboración propia.	77

46. Predicciones del mejor modelo para la celda 5363. Elaboración propia. 78

Índice de tablas

1. Coeficientes de Spearman (r_s) entre los hiperparámetros y las métricas de rendimiento como media del NRMSE y media del tiempo de ejecución en *online learning*. Elaboración propia. 30
2. Características del mejor modelo en *online learning* y sus métricas de rendimiento como media del NRMS de sus celdas. Elaboración propia. 35

1. Introducción

Las redes de telecomunicaciones tienen como finalidad el intercambio de información entre un nodo transmisor y un nodo receptor. Es un concepto sencillo afectado por un problema de números. Siguiendo la ley de Bell, el número de dispositivos ha aumentado exponencialmente desde 1 ordenador básico en un departamento de una empresa a múltiples dispositivos por persona (teléfonos móviles, relojes inteligentes...) [1]. Casi con la misma tendencia ha evolucionado la tecnología de las telecomunicaciones, impidiendo que el crecimiento de los dispositivos saturase las capacidades de la red. De esta forma, en la actualidad existe una infinidad de nodos transmisores y receptores.

Los flujos de información intercambiada entre los nodos transmisores y receptores forma el tráfico de red. Dicho tráfico puede ser representado y medido. Las medidas o muestras realizadas en diferentes instantes de tiempo pueden aunarse en una serie temporal. Como todas las series, cabe la posibilidad de que sigan un patrón que pueda utilizarse para predecir medidas futuras a partir del histórico de medidas pasadas [2]. Con la inmensa cantidad de flujos de información existentes a día de hoy, resulta imposible analizarlos mediante la intervención humana. Sin embargo, la situación planteada representa el caso de uso de la inteligencia artificial y *Deep Learning*: averiguar el patrón subyacente de un conjunto de datos [3].

La predicción del tráfico de red tiene numerosísimas aplicaciones como, por ejemplo, la correcta planificación y organización de los recursos de las redes de telecomunicaciones. Esto se traduce en un menor desperdicio de recursos, que a su vez se traduce en una inversión económica más eficiente, sin menoscabar la calidad del servicio proporcionado por la red de telecomunicaciones [4]. También es posible aplicar la predicción de tráfico de red a tareas como la detección de anomalías o intrusos, monitorización o mantenimiento [5].

Sin embargo, en la actualidad existen multitud de aplicaciones que generan flujos de datos a demanda. Algunos ejemplos son el *media streaming* o los modelos de negocio *as a Service* (XaaS) [6]. Los modelos de predicción estáticos convencionales no son capaces de adaptarse al dinamismo y la falta de estacionariedad de dichos flujos de datos.

Para solucionar dicho problema se han desarrollado modelos capaces de adaptarse a los cambios en las tendencias o patrones subyacentes en los flujos de tráfico. Estos modelos se basan en *online active deep learning*. Esta técnica permite reentrenar los modelos a medida que se incorporan nuevas muestras para adaptarse a ellas en un proceso llamado *concept drift* [7].

Este trabajo se basa en la investigación realizada por el Dr. D. Juan Pablo Casaseca de la Higuera [8] acerca de modelos LSTM que implementan actualizaciones incrementales donde las muestras sobre las que los modelos tuvieron un mayor error toman mayor importancia. En este trabajo se plantea la hipótesis de que un modelo que combine arquitecturas de redes neuronales convolucionales y de redes neuronales recurrentes tenga una precisión predictiva en *online learning* superior a aquellos modelos basados únicamente en redes neuronales recurrentes.

1.1. Objetivos

El principal objetivo de este trabajo es analizar la capacidad de adaptación mediante entrenamiento incremental de diferentes modelos de redes neuronales convolucionales recurrentes ante cambios en el patrón subyacente de series temporales.

Dentro del objetivo principal se establecen los siguientes objetivos secundarios:

- La comprensión y dominio del marco teórico utilizado en este trabajo.
- El manejo de herramientas de desarrollo como Python o Kaggle.
- La mejora del modelo base proporcionado.
- Evaluar el impacto producido por las redes neuronales convolucionales respecto al modelo base.
- Evaluar el rendimiento de los modelos en diferentes cuadrículas de 49 y 225 celdas.

1.2. Estructura del documento

Esta memoria se divide en 5 capítulos: introducción, antecedentes y estado del arte, entorno experimental, exposición de resultado y discusión y conclusiones y líneas futuras.

El Capítulo 1 se desarrolla la motivación y los objetivos de este trabajo. Además, se plantean la hipótesis y los objetivos y se añade una breve descripción de la estructura de la memoria.

El Capítulo 2 ofrece una visión general del marco teórico relevante a la predicción de series temporales, el aprendizaje automático, *deep learning*, *online learning* y las diferentes arquitecturas de redes neuronales convolucionales y recurrentes.

El Capítulo 3 presenta la base de datos utilizada, las arquitecturas analizadas y los criterios de selección y evaluación de modelos de aprendizaje automático aplicados en este trabajo.

El Capítulo 4 muestra y analiza el rendimiento de los distintos modelos propuestos. También interpreta los resultados obtenidos y comenta las relaciones presentes entre modelos y resultados.

El Capítulo 5 resume las aportaciones de este trabajo y presenta las conclusiones obtenidas. También propone líneas de actuación futuras.

2. Antecedentes y estado del arte

2.1. Predicción de series temporales

Una serie temporal puede ser representada como un vector unidimensional en cuyas posiciones se encuentran muestras secuenciales y equiespaciadas en el tiempo de una misma variable. Existen multitud de métodos de predicción de series de datos basados en la extrapolación. Sin embargo, no son específicos de series temporales sino que se aplican a conjuntos de datos genéricos.

El estudio clásico de las series temporales considera que su comportamiento es fruto de la tendencia, las variaciones cíclicas y estacionales y un componente aleatorio. La tendencia es la componente que refleja la evolución lineal a largo plazo de la serie temporal de forma lineal, exponencial... La componente cíclica recoge las oscilaciones periódicas de amplitud superior a un año. Por último, la componente estacional recoge las oscilaciones periódicas de amplitud inferiores a un año como las oscilaciones mensuales, semanales, diarias... Estas tendencias pueden generar una combinación aditiva si son independientes entre sí o multiplicativa si están correlacionadas entre sí [9].

2.1.1. Modelos autorregresivos de medias móviles

Como su propio nombre indica, los modelos autorregresivos de medias móviles para la predicción de series temporales están basados en los modelos de medias móviles y autorregresivos. Las medias móviles son una manera simple de obtener una predicción cuando no hay una tendencia marcada ni tampoco estacionalidad. Se utiliza el término medias móviles puesto que conforme se va disponiendo de nuevas observaciones se pueden calcular nuevas medias mediante la inclusión de las nuevas muestras y el desprendimiento de las más antiguas [10]. De esta forma se eliminan las oscilaciones de mayor frecuencia, actuando como un filtro paso bajo.

Las medias móviles unilaterales permiten estimar el valor futuro de la serie temporal $\hat{Y}_t$ como la media de las m muestras inmediatamente anteriores Y_t :

$$\hat{Y}_t = \frac{1}{m} * \sum_{i=1}^m Y_{t-i}$$

Asimismo, la media móvil puede combinarse con una ponderación k_i . La ponderación más habitual es la exponencial asumiendo que las muestras más recientes contribuyen más a la correcta predicción que las muestras más antiguas:

$$\hat{Y}_t = \frac{1}{m} * \sum_{i=1}^m Y_{t-i} * k_{t-i}$$

El modelo ARMA (*AutoRegressive Moving Average*) es el modelo más sencillo que combina las medias móviles y la autorregresión. Utiliza tanto las muestras reales de la serie temporal Y_t como los errores cometidos en predicciones pasadas ϵ_t . Concretamente, la cantidad de muestras y errores cometidos está estipulada por los parámetros p y q respectivamente. Siendo α_t y θ_t la ponderación aplicada a cada muestra y error [11–13]:

$$\hat{Y}_t = \sum_{i=1}^p \alpha_i * Y_{t-i} + \sum_{i=1}^q \theta_i * \epsilon_{t-i}$$

$$\epsilon_t = Y_t - \hat{Y}_t$$

El primer sumatorio es la parte autorregresiva, realiza una regresión basada en datos pasados. El segundo sumatorio es la media móvil de los errores cometidos en predicciones pasadas.

ARMA fue desarrollada inicialmente para la predicción de datos financieros [14] pero ha dado grandes resultados en sus adaptaciones a la predicción de tráfico en red a corto plazo [15, 16]. Sin embargo, el modelo ARMA presupone que la serie temporal es estacionaria, es decir, que sus propiedades estadísticas son constantes en el tiempo, sin tendencias ni comportamientos periódicos [11].

El modelo ARIMA (AutoRegressive Integrated Moving Average) introduce restas como instrumento para eliminar las tendencias presentes en la serie temporal. Al restar el valor de una muestra al valor de la siguiente se reduce un orden de la tendencia. Aplicar una segunda resta se reduce otro orden más, etc. De esta forma, la serie temporal resultante es estacionaria.

Tomando una serie temporal de ejemplo [1, 4, 9, 16, 25, 36] con una tendencia cuadrática, tras una etapa de resta se obtiene la serie [3, 5, 7, 9, 11] con una tendencia lineal. Tras una segunda ronda de resta aplicada sobre la serie lineal se obtiene la serie estacionaria [2, 2, 2, 2]. La cantidad de rondas de restas viene especificada en el parámetro d [11, 16].

$$d = 0 \text{ --- } > [1, 4, 9, 16, 25, 36]$$

$$d = 1 \text{ --- } > [4 - 1, 9 - 4, 16 - 9, 25 - 16, 36 - 25] = [3, 5, 7, 9, 11]$$

$$d = 2 \text{ --- } > [5 - 3, 7 - 5, 9 - 7, 11 - 9] = [2, 2, 2, 2]$$

El modelo ARIMA sólo tiene en cuenta tendencias pero no comportamientos cíclicos o estacionales. El modelo SARIMA (*seasonal ARIMA*) utiliza el mismo mecanismo que ARIMA pero modelando la componente estacional para una determinada frecuencia $1/s$. Además de realizar la resta con respecto a la muestra anterior, resta también la muestra en $t - s$. De esta forma, SARIMA utiliza los parámetros p , d y q

para modelar la tendencia y los parámetros P , D , Q y s para modelar la componente estacional/cíclica [11, 17].

2.2. Aprendizaje automático

El estudio de las series temporales ha evolucionado con la aparición del aprendizaje automático. Mientras los modelos clásicos autorregresivos basados en medias móviles siguen los algoritmos recién comentados, los modelos de aprendizaje automático se adaptan al patrón subyacente de las series temporales por sí mismos.

Existen varias definiciones para el aprendizaje automático. Un programa aprende de la experiencia E con respecto a una tarea T y una métrica de rendimiento R si su rendimiento en la tarea, medido por R , mejora con más experiencia E [18]. El aprendizaje automático tiene como objetivo crear sistemas capaces de aprender por ellos mismos a partir de un conjunto de datos sin ser programados de forma explícita [19].

Para resolver un problema de aprendizaje automático es necesario que haya datos con los que conformar la experiencia E , que exista un patrón en dichos datos y, lo más importante, que no exista un modelo matemático que resuelva el problema directamente. Mediante diferentes algoritmos de aprendizaje como la regresión, los árboles de decisión o *k-means clustering*, el programa puede aprender el patrón subyacente en los datos.

Dentro del aprendizaje automático se encuentran dos grandes grupos: el aprendizaje no supervisado y el aprendizaje supervisado. El aprendizaje no supervisado utiliza datos no etiquetados en los que no hay una relación entre los datos de entrada y la salida esperada para realizar tareas como *clustering* o detección de anomalías. Por el contrario, el aprendizaje supervisado utiliza la relación existente y conocida entre los datos de entrada y la salida esperada para resolver problemas de clasificación y regresión.

Las series temporales son datos etiquetados por su propia naturaleza. Una muestra utilizada como dato de entrada tiene a la siguiente muestra como salida esperada. La predicción de series temporales es en sí un problema de regresión. Por lo tanto, entra dentro del aprendizaje automático supervisado para problemas de regresión [20].

2.2.1. Procedimiento de selección de modelos

El aprendizaje automático funciona encontrando el conjunto de valores de las variables utilizadas por el modelo que producen los resultados más precisos. No siempre se consideran todas las variables en la búsqueda del mejor conjunto de valores. Las variables que sí son consideradas se denominan hiperparámetros.

Existen diferentes estrategias en la búsqueda de los mejores hiperparámetros. Entre las estrategias más comunes se encuentran el *grid search* y el *random search*. La primera itera sobre un conjunto de valores concretos establecidos mientras la segunda

utiliza conjuntos de valores obtenidos de forma aleatoria dentro de un rango. *Grid search* ofrece un gran control sobre los valores utilizados en la búsqueda mientras *random search* ofrece la posibilidad de encontrar buenos conjuntos de valores en un rango de búsqueda más amplio. Ninguna estrategia es infalible, siempre cabe la posibilidad de que el modelo para el que se realiza la búsqueda no sea un buen modelo [21].

Según cómo se diseñe el programa o modelo de aprendizaje automático y el valor de sus hiperparámetros, su rendimiento a la hora de obtener el patrón subyacente de unos datos puede variar mucho. A mayores, no existe *a priori* un modelo mejor que otro [22]. Por ello, se ha diseñado un protocolo de selección de modelos de aprendizaje automático.

El aprendizaje automático se divide en tres fases que contienen 3 sets de datos diferentes: entrenamiento, validación y test. El entrenamiento refiere a la primera fase, donde el programa o modelo aprende en la medida que le es posible el patrón subyacente presente en los datos de entrenamiento. Como se verá más adelante, esta fase involucra algoritmos como *backpropagation* o *gradient descent*. El grado de aprendizaje correcto del patrón de los datos de entrenamiento se denomina error de entrenamiento o E_{in} , siendo 0 el mejor valor posible.

El modelo, una vez pasada la fase de entrenamiento, dejará de aprender para convertirse en un modelo en producción que se dedicará a aplicar el patrón aprendido en el entrenamiento a nuevas muestras. Sin embargo, aunque el modelo tenga un E_{in} bajo, puede no dar las respuestas correctas a las nuevas muestras. El error cometido en las muestras en producción se denomina como E_{out} y el error de generalización se calcula como $E_{out} - E_{in}$. Para evitar que esto ocurra y se lance un modelo a producción que no funcione bien, provocando en la mayoría de casos pérdidas económicas y oportunidades de negocio, se introducen las fases de validación y test.

No se pueden comparar modelos entre sí directamente usando E_{in} puesto que esto sólo conllevaría a la elección del modelo con menor E_{in} y, en consecuencia, con mayor E_{out} . Esta situación es conocida como *overfitting*. La fase de validación se encarga de comparar los diferentes modelos propuestos haciéndoles procesar un set de datos de validación como si estuvieran en producción. El desempeño de los modelos en esta fase o E_{val} sí que puede utilizarse para comparar modelos.

Aunque existen múltiples estrategias de validación como *Grid Search*, *Random Search*, *k-fold*... no todas son aplicables a la predicción de series temporales. Los datos de validación siempre han de ser posteriores a los datos de entrenamiento y los datos de test siempre han de ser posteriores a los datos de validación [23]. Por lo tanto, estrategias como *k-fold validation* quedan descartadas de este trabajo.

La fase de validación tiene el mismo problema que la fase de entrenamiento. Cabe la posibilidad de que el modelo escogido haya realizado *overfitting* en los datos de validación y, aunque tenga un valor pequeño de E_{val} , tenga un gran error E_{out} . Por ello, la fase de test reentrena únicamente el mejor modelo de la fase de validación utilizando los datos de entrenamiento y validación y mide el nuevo rendimiento con un set de datos de test. El error E_{test} es entonces una buena estimación de E_{out} .

Es importante recalcar que los datos de test sólo deben ser utilizados en la fase de test. Utilizarlos en otras fases como la de entrenamiento o en el reentrenamiento del modelo tras la validación provocará una distorsión en la medida E_{test} llamada *data leakage*. Tampoco ha de usarse para comparar modelos como si fuera un test de validación [11, 20, 24].

2.3. *Deep learning*

Deep learning es considerado un subconjunto dentro del aprendizaje automático. Mientras el aprendizaje automático o *machine learning* utiliza algoritmos de aprendizaje diseñados por humanos, *deep learning* utiliza estructuras neuronales para extraer el patrón subyacente en los datos de forma directa sin necesidad de algoritmos concretos como árboles de decisión o *clustering*.

Con el objetivo de crear un programa que imitara la inteligencia humana nacieron las neuronas artificiales que imitan el funcionamiento de las neuronas biológicas del cerebro humano. Las neuronas artificiales imitan la sinapsis biológica de tal forma que evalúan la cantidad de entradas binarias activas en un momento dado para determinar si su salida, también binaria, debe activarse o no. Con un modelo tan simple es posible construir una red neuronal artificial¹ capaz de procesar cualquier problema lógico [25].

2.4. El *Perceptron*

El *perceptron* es un modelo de red neuronal que utiliza neuronas conocidas como TLU (*Threshold Logic Unit*). Las neuronas TLU funcionan de forma muy similar a las neuronas artificiales binarias salvo que procesan entradas no binarias x_n para dar una salida no binaria $y_{W,b}(X) = f(z)$. Dentro de la neurona se calcula una función lineal $z = \sum_1^N x_n * w_n + b$ donde cada entrada x_n es asignada un peso w_n además de utilizar un término de *offset* b . Existen diferentes funciones que transforman la combinación lineal de entradas z en la salida $y_{W,b}$ como por ejemplo: la función binaria *Heaviside*(x), la función sigmoide $\sigma(z)$, la función tangente hiperbólica $\tanh(z)$ o la función lineal rectificada *ReLU*(z) [11, 26, 27].

Por sí mismas, ninguna neurona es capaz de aprender. Necesitan un método de ajuste para poder adaptarse al patrón subyacente de los datos. El aprendizaje de las neuronas viene dado por el error cometido en su salida $y_{W,b}(X) = \hat{y}$ con respecto a la salida esperada y . De esta forma, una red neuronal con J neuronas actualiza sus pesos $w_{i,j}$ mediante un proceso llamado entrenamiento de la forma:

$$w_{i,j}^1 = w_{i,j}^0 + \eta * (y_j^0 - \hat{y}_j^0) * x_{i,j}^0$$

Donde $x_{i,j}^0$ es la muestra actual de la entrada i de la neurona j , $(y_j^0 - \hat{y}_j^0)$ es el error

¹En todo este trabajo se hace referencia a las redes neuronales artificiales como redes neuronales y a las neuronas artificiales como neuronas.

cometido por la neurona j en su salida respecto a la muestra actual x^0 (nótese que las neuronas sólo tienen una salida pero I entradas), η es la tasa de aprendizaje o *learning rate*, $w_{i,j}^0$ es el peso no actualizado asignado a la entrada i de la neurona j en el momento de procesar la muestra $x_{i,j}^0$ y $w_{i,j}^1$ es el peso actualizado que se utilizará en la siguiente muestra de la entrada i de la neurona j [11].

2.5. Entrenamiento de redes neuronales

Un *perceptron* formado por una sola neurona puede realizar tareas simples de clasificación binaria al encontrar, tras varias muestras de entrenamiento, los pesos óptimos que minimicen el error cometido en la clasificación. De esta forma, la neurona imitaría el patrón subyacente en los datos. Un *perceptron* formado por J neuronas puede realizar una clasificación binaria en $J + 1$ clases al tener J salidas. Lo más interesante es la posibilidad de conectar las salidas de un *perceptron* a las entradas de otro *perceptron* para crear un *perceptron* multicapa.

La primera capa es llamada capa de entrada o *input layer* mientras la última capa es la capa de salida o *output layer*. El resto de capas intermedias son las *hidden layers*. Las primeras capas son capaces de reconocer las formas más básicas del patrón subyacente a los datos. Las últimas capas utilizan los patrones básicos reconocidos por las primeras para construir patrones más complejos.

Un *perceptron* multicapa forma parte de las redes neuronales profundas o *Deep Neural Networks*. El método de ajuste de pesos de una red neuronal profunda, llamado retropropagación o *backpropagation*, involucra el uso del algoritmo conocido como descenso de gradiente o *gradient descent*.

El algoritmo *gradient descent* consiste en minimizar una función de coste cambiando iterativamente el valor de los parámetros que influyen en la salida evaluada por dicha función de coste. Cuando la red neuronal ha dado al menos dos predicciones, hay al menos dos errores medidos por la función de coste. De esta forma, se puede calcular la pendiente de la recta que une ambos errores, llamada gradiente. El algoritmo *gradient descent* utiliza el valor y signo de dicha pendiente para actualizar los pesos de las redes neuronales en el sentido descendente. A medida que el error se va reduciendo, se reduce también la fuerza de la actualización de pesos hasta llegar al mínimo de la función de coste donde el gradiente es nulo.

Utilizando W_t para denominar al conjunto de pesos o parámetros configurables de la red neuronal, las actualizaciones se realizan de la forma:

$$W_t = W_{t-1} - \beta * \nabla E_t(W_{t-1}) \quad (1)$$

siendo β la tasa de aprendizaje, $E_t(W_{t-1})$ el error medido por la función de coste en la muestra $t - 1$ y ∇ el operador gradiente.

Backpropagation funciona en dos fases: el pase hacia delante o *forward pass* y el pase hacia atrás o *backward pass*. Durante el *forward pass*, la red neuronal procesa una

entrada y guarda el valor de su salida $\hat{y}$ final junto al valor de salida de todas las neuronas que forman parte de la red. En el *backward pass* se mide el error cometido en la salida final respecto a la salida esperada y se calcula, utilizando las salidas de las neuronas de las capas intermedias, la contribución de cada peso $w_{i,j}$ de las neuronas de las capas anteriores hasta llegar a la primera capa. Finalmente, los pesos son modificados siguiendo los principios del algoritmo *gradient descent*. Las muestras de entrada que se procesan en una misma iteración de *backpropagation* se denominan *batch*.

Es muy importante que los pesos $w_{i,j}$ de cada capa l sean inicializados de forma aleatoria. Si todos los pesos fueran iguales, serían actualizados en la misma cuantía por el algoritmo *gradient descent*. De esta forma la red neuronal actuaría como si cada capa sólo tuviera una neurona [11, 27, 28].

2.6. Redes neuronales convolucionales

Las redes neuronales convolucionales o CNNs (*Convolutional Neural Networks*), al igual que los *perceptrons*, imitan la estructura biológica de las neuronas. Sin embargo, las CNNs imitan la arquitectura del córtex visual. Una neurona j de la capa l no está conectada a todas las neuronas de la capa $l - 1$ sino que se conecta a un set pequeño de neuronas circundantes de esa capa. De esta forma, juntando y solapando los campos visuales de diferentes neuronas, se llega a cubrir el campo visual entero, como si se realizara la convolución de un filtro a lo largo de una imagen. Así, las redes neuronales convolucionales son ideales para el procesamiento de datos en dos dimensiones como las imágenes.

Las capas de una red CNN disponen sus neuronas en un plano bidimensional i, j para imitar la estructura biológica. El campo visual de una neurona está definido por su altura f_h y anchura f_w . Así, la neurona i, j, l , está conectada a las neuronas comprendidas en el campo visual $i + f_h - 1, j + f_w - 1, l - 1$. Esto supone que las capas superiores son cada vez más pequeñas puesto que los campos visuales de las neuronas cercanas a los bordes de la capa l se saldrían de las dimensiones de la capa $l - 1$. Para evitarlo, es posible rodear todos los datos con ceros (*zero padding*) para que el campo visual de las capas superiores no se salga de las capas inferiores. Cabe destacar que las posiciones i y j y sus derivados son indexadas desde el valor cero.

Aun así, para reducir la complejidad de la red neuronal, es posible conectar una capa convolucional con otra más pequeña sin tener que estar relacionadas por los parámetros f_h y f_w al usar un salto mayor o *stride*, tanto en vertical s_h como en horizontal s_w [11, 29, 30].

2.6.1. Mapas de filtros

Durante el estudio del córtex visual biológico se descubrió que algunas neuronas sólo reconocían patrones verticales mientras otras reconocían sólo patrones en otras direcciones. Esto condujo a la creación de filtros o *kernels* en las neuronas artificiales

convolucionales. Un filtro es una combinación de pesos determinada que permite a una neurona reconocer patrones en una dirección concreta. Una capa de neuronas que comparten un mismo *kernel* pertenecen a un mismo mapa de filtros. Cada neurona del mapa k de la capa l procesa todos los mapas de filtros de la capa $l - 1$ pertenecientes a su campo visual [11]. La posición k y sus derivados también se indexan desde cero.

La salida $z_{i,j,k,l}$ proporcionada por una neurona situada en la fila i , columna j del mapa k de la capa convolucional l , con un campo visual de altura f_h y anchura f_w y *strides* vertical s_h y horizontal s_w toma la forma:

$$z_{i,j,k,l} = b_{k,l} + \sum_{u=0}^{f_h-1} \sum_{v=0}^{f_w-1} \sum_{k'=0}^{f_k-1} z_{i',j',k',l-1} * w_{u,v,k',k} \quad \forall \quad i' = i * s_h + u; \quad j' = j * s_w + v \quad (2)$$

donde b_k es el término de offset aplicado a todas las neuronas del mapa de filtros k de la capa l ; $z_{i',j',k',l-1}$ es la salida proporcionada por una neurona situada en la fila $i * s_h + u$, columna $j * s_w + v$ del mapa k' de la capa inmediatamente anterior $l - 1$; y $w_{u,v,k',k}$ es el peso que pondera la conexión entre la neurona $x_{i,j,k,l}$ y las neuronas del mapa k' de la capa $l - 1$ situadas en el campo visual u, v [11], como se ilustra en la Figura 1.

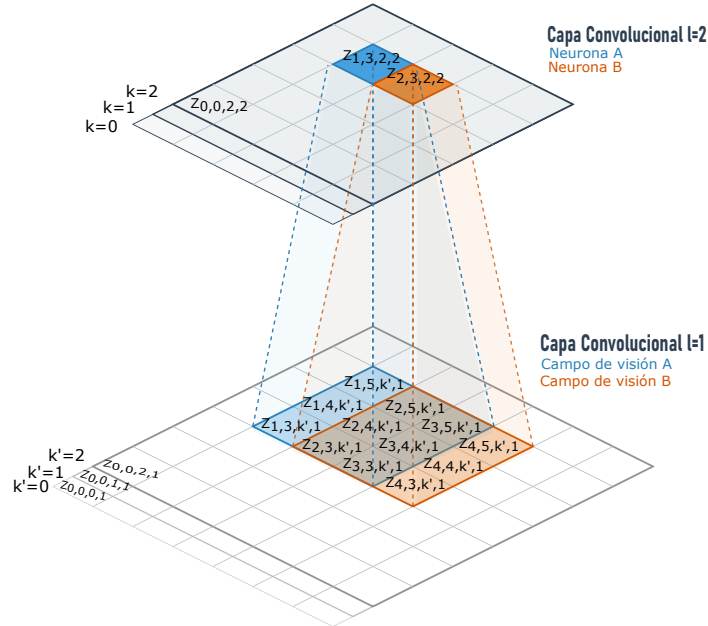


Figura 1: Campos de visión de dos neuronas convolucionales situadas en el mapa de filtros $k=2$ de la capa convolucional $l=2$ con *stride*=1. Elaboración propia.

2.6.2. Capas de agregación

Las redes neuronales convolucionales presentan un gran problema en cuanto a los recursos consumidos, especialmente durante el entrenamiento. Para orquestar el algoritmo de retropropagación se necesita obtener las salidas intermedias de todas las

capas de neuronas, contando con todos sus mapas de filtros. La carga computacional puede llegar a ser muy grande, necesitando grandes inversiones en *hardware*.

Para ello se desarrollaron las capas de agregación. Su cometido es el submuestreo de los datos para reducir la carga computacional perdiendo la menor cantidad de información posible. Son capas cuyas neuronas, también dispuestas en un plano bidimensional y conectadas sólo a las neuronas dentro de su campo visual, sin pesos para sus entradas. Las neuronas de las capas de agregación combinan las entradas con funciones como máximo o promedio. Así, se reduce el tamaño de los datos a procesar eliminando la información menos importante, lo que ayuda también a que las CNNs mejoren su generalización [11]. Sin embargo, el uso de estas capas para la predicción de series temporales es contraproducente puesto que eliminan la información acerca del orden temporal de las muestras [23].

2.7. Redes neuronales recurrentes

Las redes neuronales recurrentes o RNNs (*Recurrent Neural Networks*), a diferencia de las anteriores, no están basadas en ninguna estructura biológica. Son capas de neuronas capaces de retener información de muestras procesadas con anterioridad. Las neuronas recurrentes más simples, además de procesar las entradas x_t , procesan su propia salida de la muestra anterior $\hat{Y}_{t-1}$, con su propio peso $w_{\hat{y}}$ para generar la salida actual $\hat{Y}_t$. Esto hace a las RNN ideales para el procesamiento de secuencias de datos como, por ejemplo, series temporales.

Otras neuronas recurrentes más avanzadas fueron desarrolladas para aumentar la capacidad de memoria de las neuronas más simples. La mayor parte de ellas comienza por memorizar una combinación de las entradas con la salida anterior en vez de sólo la salida anterior. Esta combinación se denomina estado o *hidden state* $h_t = f(x_t, \hat{y}_{t-1})$. La salida de estas neuronas es también una función de las entradas y el estado anterior $\hat{y}_t = g(x_t, h_{t-1})$ [11, 31–33].

2.7.1. Neuronas LSTM

Aun así, con el paso de muchas entradas, la información almacenada en el estado h es desechada en favor de nueva información. Las neuronas LSTM (*Long-Short Term Memory*) fueron diseñadas para escoger si la información obtenida de las muestras procesadas más recientes es merecedora de ser almacenada en el estado de la neurona. En realidad no son en sí una neurona sino una célula compuesta de una capa de neuronas completamente conectadas y operadores de adición y multiplicación.

Las neuronas LSTM dividen el estado en dos vectores: el vector de largo alcance c_t y el vector de corto alcance h_t , siendo éste último equivalente a la salida de la neurona $\hat{y}_t$. Dentro de la neurona LSTM, el vector de largo alcance sufre una pérdida de información antigua, controlada por la puerta de olvido o *forget gate* f_t , y la adición de información nueva controlada por la puerta de entrada o *input gate* i_t y la combinación g_t de la entrada x_t y el vector de corto alcance h_{t-1} .

La puerta de olvido y la puerta de entrada son controladas mediante el procesamiento del vector de corto alcance anterior h_{t-1} y las muestras de entrada actuales x_t por una capa de neuronas completamente conectadas por cada puerta. Dichas capas completamente conectadas tienen la función de activación logística, que arroja valores entre 0 y 1 en su salida. Así, controlan el porcentaje de olvido aplicado al vector c_{t-1} y el porcentaje de la combinación $g_t = g(x_t, h_{t-1})$ que se añadirá al vector c_{t-1} para formar el nuevo vector de largo alcance c_t . La combinación g_t es la salida ofrecida por una neurona recurrente básica que combina la entrada actual x_t con su estado anterior h_{t-1} .

La última parte de las neuronas LSTM es la puerta de salida o *output gate* o_t . Ésta es controlada de la misma forma que las puertas descritas anteriormente e indica qué porción del nuevo vector de largo alcance c_t es considerado como salida $\hat{y}_t$ y nuevo vector de corto alcance h_t .

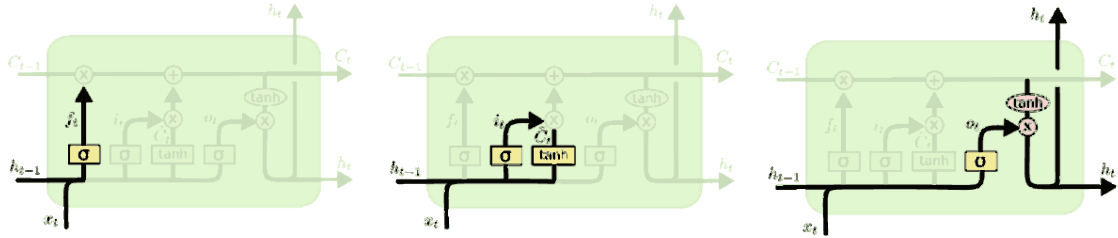


Figura 2: Representación de las puertas de una neurona LSTM [34].

En resumen, las puertas y las salidas de una neurona LSTM son controladas de la forma:

$$\begin{aligned}
i_t &= \sigma(W_{x,i}^T * x_t + W_{h,i}^T * h_{t-1} + b_i) \\
f_t &= \sigma(W_{x,f}^T * x_t + W_{h,f}^T * h_{t-1} + b_f) \\
o_t &= \sigma(W_{x,o}^T * x_t + W_{h,o}^T * h_{t-1} + b_o) \\
g_t &= \tanh(W_{x,g}^T * x_t + W_{h,g}^T * h_{t-1} + b_g) \\
c_t &= f_t \odot c_{t-1} + i_t \odot g_t \\
y_t = h_t &= o_t \odot \tanh(c_t)
\end{aligned}$$

siendo i_t , f_t , o_t , g_t , c_t , y_t y h_t en valor en la muestra t de la puerta de entrada, olvido, salida, la combinación de la entrada x_t y h_{t-1} , el vector de largo alcance, la salida y el vector de corto alcance de la neurona LSTM; $W_{\{x,h\},\{i,f,o,g\}}$ los pesos asignados a la entrada x o al vector de corto alcance h_{t-1} de las capas completamente conectadas de las puertas; $b_{\{i,f,o,g\}}$ los valores de *offset* de las capas completamente conectadas y $\odot$ el producto elemento a elemento de matrices.

La función σ es la función sigmoide:

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

y la función $\tanh$ la función tangente hiperbólica:

$$\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$

De esta forma, las neuronas LSTM pueden reconocer una muestra de entrada relevante para construir el patrón subyacente de los datos y almacenarla a la misma vez que ignora las muestras de entrada irrelevantes. Esto además permite a las neuronas LSTM encontrar patrones de largo plazo en los datos [8, 11, 31–33, 35].

2.8. CNN+LSTM

El presente trabajo analizará varios modelos con el objetivo de encontrar el patrón subyacente en varias series temporales. Dichas series representan el tráfico de red cursado en múltiples nodos situados geográficamente adyacentes entre sí. Aprovechando las capacidades de las redes convolucionales y recurrentes, se analizarán modelos que combinan redes CNN con RNN para encontrar el patrón espacio temporal presente en los datos esperando que ello mejore las capacidades de predicción de tráfico en red.

2.8.1. TimeDistributed

Para poder procesar series temporales formadas por muestras bidimensionales, como es el caso de este trabajo, se debe aplicar los mismos mapas de filtros a todas las muestras de los datos de forma independiente en vez de tratar todas las muestras de un *batch* como una sola secuencia. Si no, se perdería la relación temporal existente entre las muestras.

Esto es posible de forma sencilla gracias a la función `TimeDistributed()` disponible en la librería `TensorFlow`. Ésta permite aplicar una misma capa de neuronas a cada secuencia del *batch* de entrada. La capa de neuronas incluida en `TimeDistributed()` utiliza los mismos pesos para todas las secuencias del *batch* y realiza *backpropagation* para minimizar el error cometido en todo el *batch*, no sólo en cada secuencia [36].

Para aplicar esta idea de forma correcta es posible englobar una capa convolucional bidimensional `Conv2D` dentro de `TimeDistributed()`. Las capas `Conv2D` sólo procesan *batches* de muestras bidimensionales. Al procesar una serie temporal, hay una dimensión nueva: el instante temporal. `TimeDistributed()` se encarga de aislar cada muestra temporal para que `Conv2D` las procese por separado antes de volver a unir las.

2.8.2. Flatten

Las neuronas protagonistas en la predicción de series temporales son las neuronas LSTM pero no son capaces de procesar muestras bidimensionales. Por lo tanto, se necesita realizar una transformación del formato de salida de las capas `TimeDistributed(Conv2D())` para poder procesar las series temporales.

La salida de las capas `TimeDistributed(Conv2D())` es pentadimensional: $(\text{batchsize}, \text{lookback}, \text{altura}, \text{anchura}, \text{canales})^2$ siendo `batchsize` la cantidad de secuencias temporales incluidas en un mismo *batch*, `lookback` la longitud de cada secuencia temporal, altura y anchura las dimensiones de las muestras bidimensionales y canales la cantidad de variables incluidas en las muestras. Por ejemplo, una imagen RGB tendría 3 canales y las series temporales analizadas en este trabajo tienen un único canal puesto que la única variable existente es el volumen de tráfico.

Las capas LSTM procesan muestras tridimensionales: $(\text{batchsize}, \text{lookback}, \text{canales})$. Es posible realizar la transformación de los datos de un formato pentadimensional a un formato tridimensional a través de las capas `Flatten` ofrecidas por `Tensorflow`. Estas capas reorganizan los datos para fusionar las dimensiones altura, anchura y canales en una sola dimensión. No tienen pesos ni realizan ningún cambio en los valores, únicamente modifican el formato. De esta forma, la salida de una capa `TimeDistributed(Conv2D())` es modificada por una capa `TimeDistributed(Flatten())` para obtener el formato $(\text{batchsize}, \text{lookback}, \text{altura} \times \text{anchura} \times \text{canales})$ apto para las capas LSTM.

2.9. Online learning

Uno de los mayores problemas de los modelos de aprendizaje automático o *deep learning* es la evolución de los datos. Los modelos en producción suelen ser estáticos, también llamados *offline*, es decir, no actualizan sus parámetros ni pesos. Esto les aporta una gran solidez pero genera una decadencia en su rendimiento a medida que los datos que procesan modifican su patrón subyacente con el paso del tiempo, alejándose del patrón aprendido por el modelo. Este problema, comúnmente denominado como *concept drift*, no siempre tiene la misma apariencia, dependiendo de los datos procesados el cambio de patrón puede ser más o menos brusco o incluso ser cíclico.

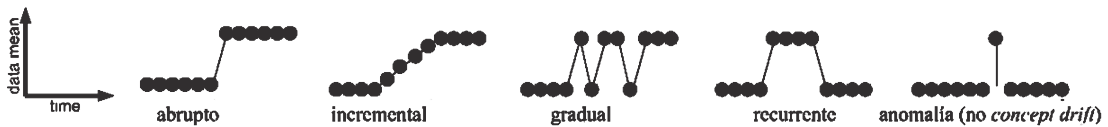


Figura 3: Representación de las distintas formas de *concept drift*. [37]

²Se utiliza `batchsize` y `lookback` por concordancia con la nomenclatura utilizada en este trabajo.

Para solucionar este problema se debe entrenar de nuevo el modelo. Esto conlleva una gran tarea por detrás. Además de tener que generar nuevos sets de entrenamiento, validación y test que representen no sólo el nuevo patrón cambiado sino también el antiguo, la inversión en tiempo y recursos de computación puede ser muy alta. Esto supone una gran inversión financiera que puede no llegar a ser rentable si la frecuencia de cambio del patrón de los datos es demasiado alta.

Como solución alternativa se propuso el entrenamiento incremental u *online learning*: el modelo desplegado en producción procesa de forma secuencial los datos en pequeños grupos o *mini-batches* de los que aprende de forma rápida y barata con el algoritmo *backpropagation* tras dar una predicción sobre los mismos. De esta forma, a medida que procesa nuevos paquetes de datos en producción se ajusta al patrón de los nuevos datos. No todo es ventajas puesto que el modelo debe ser capaz de distinguir las irregularidades en el patrón debidas al ruido presente en los datos de las irregularidades causadas por un cambio real en el patrón [7, 37].

Los modelos basados en *online learning* son especialmente útiles para la predicción de series temporales como el tráfico en red. Debido a la naturaleza de estas series, no necesitan la intervención ni humana ni de otros modelos para generar datos de entrenamiento. Cuando el modelo de *online learning* procesa una secuencia de datos x_t y predice la siguiente muestra $\hat{Y}_{t+1}$, no conoce el error que ha cometido en la predicción puesto que el valor real y_{t+1} es todavía desconocido. Sin embargo, llegado el momento $t + 1$ sólo es necesario medir la variable real para conocer y_{t+1} . Así se puede medir el error cometido $\epsilon_{t+1} = y_{t+1} - \hat{Y}_{t+1}$ de forma sencilla y, a pesar de la necesidad de un elemento externo para medir la variable real, autónoma. Sólo resta ejecutar de nuevo el procedimiento de retropropagación.

3. Entorno experimental

3.1. Descripción de los datos

Los datos utilizados en este trabajo provienen del dataset Milán [38]. Este dataset fue creado con la intención original de descubrir la relación, si existiese, entre los eventos meteorológicos y el tráfico de red en la ciudad de Milán, Italia. El dataset contiene datos tanto climatológicos como de SMS, llamadas telefónicas y tráfico perteneciente a navegación web. Los datos están organizados en mediciones realizadas cada hora que corresponden a una celda cuadrada de $235m$ de lado dentro de un área de $553,66km^2$ de la ciudad de Milán [39]. El dataset contiene mediciones correspondientes a los meses de noviembre y diciembre de 2013. Para este trabajo se han aislado y agregado los datos relativos a SMS, llamadas telefónicas y tráfico web en muestras equiespaciadas una hora en el tiempo, desechando la información relativa a los eventos meteorológicos.

Las celdas utilizadas en este trabajo corresponden a una cuadrícula de 7×7 celdas centrada en la celda 5060. Para averiguar el alcance de las redes neuronales convolucionales, también se han realizado pruebas con una cuadrícula de 15×15 celdas centradas en la celda 5464. Esta última cuadrícula engloba completamente a la cuadrícula 7×7 .

Aproximadamente, todas las series temporales de las celdas son bastante similares entre sí. Todas presentan un patrón subyacente claro y periódico con un periodo de más o menos 24 horas. Además, todas ellas también presentan un patrón periódico con un periodo de una semana, en el que los primeros 5 ciclos de 24 horas tienen bastante más amplitud que los últimos 2. Esto coincide con los 5 días laborables de la semana y los 2 días del fin de semana.

Respecto a las similitudes entre celdas, es difícil afirmar que una celda tenga una serie similar a otra celda contigua a lo largo de toda la cuadrícula. Por ejemplo, la celda 5357 es contigua a las celdas 5257, 5258 y 5358 pero su serie temporal, aunque presenta las dos componentes periódicas comunes a toda la cuadrícula, no es similar. Por otro lado, la celda 4962 es bastante similar a todas sus celdas contiguas (4861, 4862, 4863, 4961, 4963, 5061, 5062, 5063) presentando todas ellas las dos componentes cíclicas comunes de forma muy marcada.

La mayoría de las celdas extienden el patrón semanal hasta el día 22 de diciembre de 2013, a partir del cual el tráfico de red disminuye considerablemente. Esto coincide con las vacaciones de Navidad cuando los nodos que generan mayor tráfico como negocios, oficinas, universidades... están. Algunas celdas, como las celdas 5357 y 5362 presentan dicha disminución del tráfico mucho antes, alrededor del día 1 de diciembre de 2013.

A la vista de estos cambios en los patrones del dataset Milán, se ha decidido entrenar los modelos de redes neuronales con las primeras 400 horas del dataset y utilizar el resto de horas, aproximadamente 1000 horas, como datos de validación. Este reparto asegura que los modelos son entrenados con un patrón invariable mientras el dataset

de validación presenta cambios en el patrón subyacente. Así será posible observar si los modelos con *online learning* son capaces de adaptarse a los cambios tal y como se ha planteado en los objetivos de este trabajo.

3.2. Arquitecturas empleadas

Para este trabajo se han utilizado 5 arquitecturas de redes neuronales. En primer lugar se ha utilizado una arquitectura formada por una única capa de 24 neuronas LSTM en medio de una capa de entrada y una capa completamente conectada de 3 salidas. Con esta arquitectura se ha obtenido el rendimiento base de la propuesta realizada por el Dr. Juan Pablo Casaseca de la Higuera [8], a mejorar en este trabajo. Los datos procesados por esta arquitectura pertenecen a las celdas 4259, 4456 y 5060 del dataset Milán.

Para alcanzar el objetivo de evaluar el rendimiento en dos conjuntos de 49 y 225 celdas contiguas se han utilizado también dos arquitecturas formadas por una capa de entrada, una capa de neuronas LSTM y una capa completamente conectada de 49 y 225 salidas respectivamente. De esta forma se ha obtenido el rendimiento de la arquitectura exclusivamente LSTM³ con las celdas contiguas.

Para intentar extraer la correlación espacial entre las celdas, se han propuesto otras 3 arquitecturas compuestas por una capa de entrada, 1, 2 y 3 capas de neuronas CNN⁴ respectivamente, una capa Flatten, una capa de neuronas LSTM y una capa completamente conectada de 49 y 225 salidas respectivamente.

El hecho de utilizar dos conjuntos, de 49 y 225 celdas, no implica utilizar una arquitectura distinta para cada conjunto. La arquitectura refiere a la disposición de las diferentes capas de neuronas en el modelo, no del tamaño ni la cantidad de neuronas presentes en ellas.

Para aprovechar al máximo las librerías cuDNN se ha utilizado la función de activación **sigmoide** para las puertas de entrada, salida y olvido y la función **tanh** para la combinación de entradas y estado de las neuronas LSTM [40]. Otras funciones de activación no son compatibles con los *kernels* optimizados cuDNN por lo que el tiempo de ejecución aumenta exponencialmente. Aunque cabe la posibilidad de que otras funciones de activación consigan mejores resultados, un tiempo de ejecución excesivamente alto inutiliza la red neuronal.

3.3. Procedimiento de entrenamiento

Para la evaluación y comparación de los modelos, se ha ideado un procedimiento similar a la validación de modelos de aprendizaje automático descrita en el Capí-

³A lo largo de este trabajo se hará referencia a esta arquitectura como LSTM.

⁴A lo largo de este trabajo se hará referencia a estas arquitecturas como CNN+LSTM, 2CNN+LSTM y 3CNN+LSTM respectivamente.

tulo 2. Se ha entrenado los modelos con las 400 primeras muestras del dataset y se han realizado dos validaciones distintas en cada modelo. Una de las validaciones se ha realizado con las otras 1000 muestras del dataset sin utilizar entrenamiento incremental. La otra validación se ha realizado con las mismas 1000 muestras pero implementando entrenamiento incremental con ajuste de gradientes por norma euclídea del error normalizado [8] definida por la ecuación:

$$\left\| \frac{y_t - \hat{y}_t}{y_t} \right\|^2 \quad (3)$$

Y el error cuadrático como función de coste:

$$E_t(W_{t-1}) = (y_t - \hat{y}_t)^2 \quad (4)$$

De esta forma, las ecuaciones 1 y 3 se combinan en:

$$W_t = W_{t-1} - \beta * \left\| \frac{y_{t-1} - \hat{y}_{t-1}}{y_{t-1}} \right\|^2 * \nabla (y_{t-1} - \hat{y}_{t-1})^2 \quad (5)$$

De esta forma se da una mayor importancia a las predicciones que han generado errores mayores. Estos errores dan más información y permiten realizar una actualización de pesos más agresiva para aumentar la velocidad de convergencia hacia el mínimo de la función de coste.

3.4. Hiperparámetros considerados

Hay una selección variada de parámetros configurables a la hora de entrenar las redes neuronales propuestas. No todos los parámetros han sido considerados como configurables como, por ejemplo, el tamaño de los sets de entrenamiento, test estático y test incremental. El criterio de selección se basa en el impacto de los hiperparámetros en el rendimiento de las redes neuronales en el apartado de *online learning*. Por esta razón se ha seleccionado los siguientes parámetros:

- **Número de celdas:** indica si se está procesando una cuadrícula de 7x7 celdas o de 15x15 celdas.
- **Tamaño del *kernel*:** indica el tamaño del campo de visión de las neuronas convolucionales. Está estrechamente relacionado con la cantidad de celdas utilizadas en el dataset. Tiene un gran impacto en todas las fases de entrenamiento y test.
- **Tamaño de ventana deslizante:** también denominado `look_back`, además de ser el número de neuronas presentes en la capa LSTM, establece cuántas

muestras de tráfico de red de cada celda son procesadas al mismo tiempo. Dichas muestras son tomadas como una misma secuencia de longitud `look_back`. Tiene un gran impacto en todas las fases de entrenamiento y test.

- **Tamaño de *batch*:** también denominado `batch_size`, indica cuántas secuencias de longitud `look_back` son procesadas por la red neuronal en un mismo momento. Tiene un gran impacto en todas las fases de entrenamiento y test.
- **`n_epoch_updates`:** a partir de aquí `updates` para abreviar, indica la cantidad de veces que la red neuronal procesa un mismo *batch* en *online learning*. Cada vez que lo procesa, ajusta sus pesos si el error supera el umbral de entrenamiento incremental.
- **Umbral para el entrenamiento incremental:** también llamado `threshold`, indica el error mínimo que la red neuronal ha de cometer durante el entrenamiento incremental para ajustar sus pesos. Un mayor `threshold` elimina el ruido en los datos para evitar ajustes de pesos innecesarios aunque puede provocar que la red neuronal ignore cambios pequeños en el patrón subyacente de los datos.
- **Número de filtros de las capas CNN:** también llamado `num_filter1`, `num_filter2` o `num_filter3`, indica cuántas formas de patrones son utilizadas por las capas convolucionales para intentar componer el patrón subyacente de los datos. Tiene un gran impacto en todas las fases de entrenamiento y test.

Para la búsqueda de los parámetros óptimos se ha preferido la estrategia *grid search* frente a *random search*. Los hiperparámetros considerados tienen un gran impacto en el tiempo de procesamiento de las redes neuronales, que se intenta reducir en la medida de lo posible. Esto establece un límite en el número de filtros, `updates` y el tamaño de la ventana deslizante y los *batches*. Además, un umbral `threshold` demasiado alto inutiliza el entrenamiento incremental. De esta forma, aunque *random search* tenga posibilidades de probar combinaciones con valores fuera de lo normal, es casi seguro que éstos desemboquen en combinaciones que impongan una carga computacional excesiva.

3.5. Métricas de rendimiento

Como métrica de rendimiento se ha utilizado la raíz normalizada del error cuadrático medio o NRMSE (*Root Normalized Mean Squared Error*). Sin embargo, como cada celda tiene su propio patrón temporal subyacente, cada celda tiene un NRMSE distinto. Teniendo 49 o 225 celdas es imposible comparar todos sus NRMSE con los NRMSE de otras celdas con otra combinación de hiperparámetros. Para dicha comparación, como todas las celdas son consideradas igual de importantes, se ha usado la media aritmética del NRMSE de todas las celdas.

El NRMSE eleva al cuadrado las diferencias entre el valor real y_i y el valor predicho por la red neuronal $\hat{y}_i$ y las promedia. Finalmente calcula la raíz cuadrada del promedio y normaliza, en este trabajo, por la desviación típica de los valores reales. El NRMSE queda por tanto definido como:

$$NRMSE = \frac{\sqrt{\frac{1}{n} * \sum_{i=1}^n (y_i - \hat{y}_i)^2}}{\sigma(y)}$$

donde n es el número de muestras del conjunto medido.

El NRMSE es una métrica adimensional independiente de la magnitud de los datos que representa la distancia entre dos puntos. Es ideal para este trabajo puesto que los datos utilizados presentan grandes fluctuaciones que distorsionarían el error cuadrático medio y su raíz. Un valor de 0 de NRMSE sería perfecto, sin errores cometidos en todo el set. Sin embargo, es irracional buscar un NRMSE=0. Teniendo en cuenta los resultados obtenidos por el Dr. Juan Pablo Casaseca de la Higuera [8], el NRMSE buscado es aquel NRMSE<0,1778 en el set de entrenamiento incremental.

3.6. Análisis de Spearman

Para estimar el impacto de cada hiperparámetro en el desempeño de la red neuronal, se ha decidido optar por el análisis de Spearman. Es un método no paramétrico para la estimación de la dependencia estadística entre dos variables. A diferencia del análisis de Pearson, que busca relaciones estrictamente lineales, Spearman busca cualquier tipo de relación.

Al ser un algoritmo no paramétrico, no es necesario conocer la distribución de los datos analizados y es posible comparar variables con rangos de valores muy dispares y con muchas anomalías sin distorsionar el resultado. Esto es posible debido al uso de niveles ordenados que reemplazan los posibles valores de las variables. Por ejemplo, si la variable X toma los valores 3, 25 y 67.42 son sustituidos por los niveles X_1 , X_2 y X_3 respectivamente y los valores de la variable Y: 1580, 40721 y 31.67 son sustituidos por los niveles Y_2 , Y_3 e Y_1 .

El análisis de Spearman calcula la diferencia al cuadrado entre los niveles de ambas variables en cada uno de los puntos conocidos. Finalmente realiza una sencilla operación matemática para obtener el valor final de la correlación entre ambas variables:

$$r_s = 1 - \frac{6 * \sum d^2}{n(n^2 - 1)}$$

siendo d las diferencias entre niveles de las variables y n el número de puntos existentes. El valor del coeficiente de Spearman r_s varía entre -1 para variables completamente correlacionadas de forma inversa y 1 para una correlación total directa, siendo 0 el valor que indica la ausencia de correlación [41].

Debido a la disparidad en la distribución de las variables que se analizan en este trabajo, por ejemplo: el hiperparámetro `look_back` toma valores entre 12 y 32 mientras la métrica NRMSE del modelo con *online learning* toma valores entre 0 y 1; el análisis de Spearman es ideal al ser capaz de evitar que la diferencia entre los rangos de valores de las diferentes variables afecte al cálculo de la correlación entre ellas.

3.7. Entorno de ejecución

Para el desarrollo de las redes neuronales utilizadas en este trabajo se ha utilizado una plataforma virtualizada llamada Kaggle. Ésta permite al usuario de forma gratuita el acceso a GPUs (*Graphics Processing Units*) durante 30h por semana. Las GPUs en cuestión son del modelo NVIDIA TESLA P100, ideales para el entrenamiento de redes neuronales gracias a sus 4 cores y 29 GB de RAM [42]. Además, Kaggle ofrece un entorno de edición gráfico bastante intuitivo.

El lenguaje de programación utilizado en este trabajo es Python. Es un lenguaje orientado a objetos, interpretado e interactivo. Su gran popularidad está basada en su versatilidad. La alta compatibilidad de sus módulos ha hecho que existan numerosas librerías y paquetes especializados y eficientes en multitud de tareas entre las cuales se encuentra el desarrollo de redes neuronales como Tensorflow o SKLearn.

En *frameworks* de computación en paralelo como CUDA (*Compute Unified Device Architecture*), los cálculos realizados por las redes neuronales se reparten entre diferentes hilos. Cada hilo procesa los datos en momentos de tiempo diferentes según el estado del organizador de eventos del sistema operativo. Por lo tanto, en cada entrenamiento se consiguen resultados distintos puesto que el estado del sistema operativo cambia y las pequeñas diferencias en el tiempo de ejecución de los hilos se acumulan a lo largo de todo el entrenamiento y test. Además, el lenguaje de programación Python y las librerías Tensorflow y cuDNN inician sus algoritmos de forma aleatoria.

Para conseguir la reproducibilidad de los resultados se ha de introducir varias restricciones para eliminar la aleatoriedad. Se han fijado todas las semillas de generación de números pseudoaleatorios de Python, Tensorflow y NumPy y se ha forzado a las librerías Tensorflow y cuDNN a acceder a los recursos hardware de forma determinista a través de las variables de entorno del sistema operativo [43, 44].

4. Resultados y discusión

4.1. Desempeño general

El objetivo de este trabajo es la evaluación del rendimiento de los modelos con *online learning* respecto a los mismos modelos con entrenamiento estático u *offline learning*. En todas las combinaciones de hiperparámetros y en todas las arquitecturas los modelos con *online learning* han mostrado un rendimiento superior. Por lo tanto, en este capítulo se analizarán principalmente los resultados obtenidos en los modelos con *online learning*.

El *grid search* realizado en este trabajo no es uniforme. Para ahorrar tiempo se ha comenzado con un *grid search* reducido que se ha ido expandiendo según la correlación entre los hiperparámetros y sus resultados. Por ello, es difícil aislar la influencia de cada hiperparámetro en los resultados aunque analizando todos en conjunto es posible obtener conclusiones coherentes.

Como se ha mencionado anteriormente, la comparación entre los diferentes modelos en el proceso de validación se realiza con la media aritmética del NRMSE obtenido en cada celda. Como en toda media aritmética, la cantidad de valores de la variable a medir influye en gran medida en el resultado final. Como es de esperar, las medias del NRMSE de los modelos con una cuadrícula de 15x15 celdas tienen una desviación menor que las medias del NRMSE de los modelos con una cuadrícula de 7x7 celdas, como se observa en la Figura 4. Esto es debido a que la variación en el NRMSE de una celda es amortiguada por las otras 224 celdas, mientras que 48 celdas tienen un poder amortiguador menor.

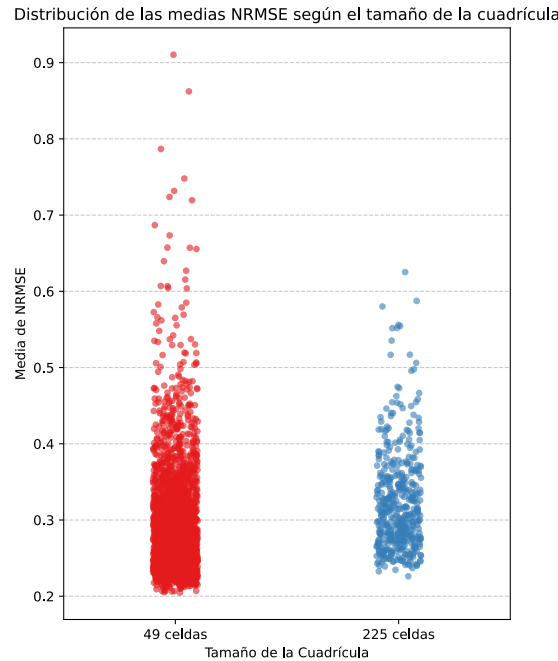


Figura 4: Distribución de las medias de NRMSE de los modelos con *online learning* según el tamaño de la cuadrícula. Elaboración propia.

Según la arquitectura, es decir, el número de capas convolucionales utilizadas, la distribución de las medias de NRMSE es más o menos constante. Como se observa en los diagramas de violín de la Figura 5, al aumentar el número de capas convolucionales, el rendimiento de los modelos se concentra más en torno a 0.27 de media de NRMSE aunque tienen más *outliers*.

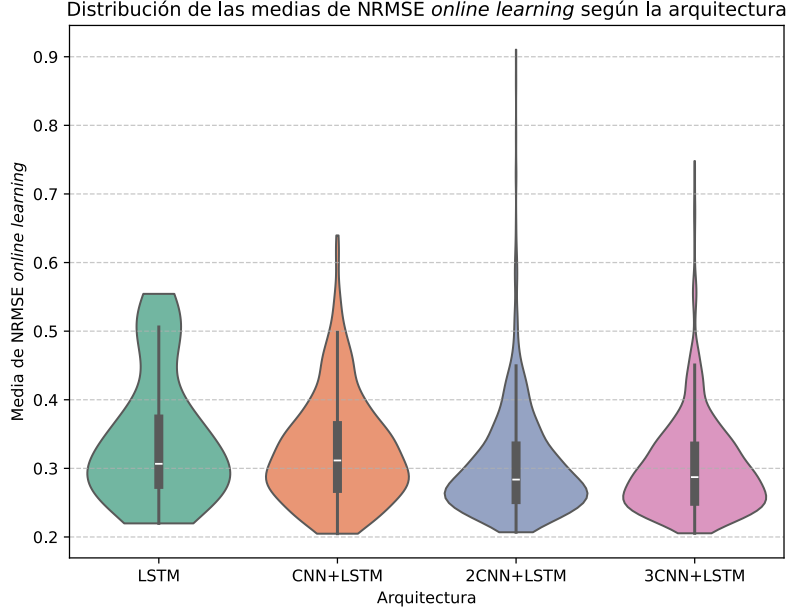


Figura 5: Distribución de las medias de NRMSE de los modelos con *online learning* según la arquitectura del modelo. Elaboración propia.

Aun así, aunque las arquitecturas con más capas y, por ende, más capacidad de procesamiento sean menos susceptibles a cambios en los hiperparámetros, la arquitectura con una única capa convolucional (CNN+LSTM) es la que ha conseguido el mejor modelo.

4.2. Análisis de Spearman

Tras un vistazo rápido a los resultados según el tamaño del dataset y de la arquitectura utilizada, el análisis de Spearman puede arrojar más luz acerca de la contribución de los hiperparámetros a los resultados obtenidos por los modelos con *online learning*. La Tabla 1 muestra el coeficiente de Spearman (r_s) para todos los hiperparámetros y métricas de rendimiento.

	NRMSE train	NRMSE test <i>offline</i>	NRMSE test <i>online</i>	Tiempo (s) <i>online</i>
Tamaño cuadrícula	0,4715	0,6598	0,1613	0,3127
Capas CNN	-0,3044	0,2384	-0,0980	0,5366
Tamaño <i>kernel</i>	-0,2282	-0,3540	-0,0271	-0,1942
<i>look_back</i>	-0,2321	0,3183	-0,2465	0,5053
<i>batch_size</i>	-0,0055	-0,0087	-0,1993	0,1711
<i>updates</i>	-0,1094	0,0101	-0,3473	0,5482
<i>threshold</i>	0,0343	-0,2324	0,0362	-0,2177
Filtros capa 1	0,0068	-0,0815	0,0574	-0,2144
Filtros capa 2	0,0121	0,0238	0,1108	-0,1523
Filtros capa 3	-0,0286	0,0479	0,0659	-0,1166

Tabla 1: Coeficientes de Spearman (r_s) entre los hiperparámetros y las métricas de rendimiento como media del NRMSE y media del tiempo de ejecución en *online learning*. Elaboración propia.

4.2.1. Número de celdas, capas CNN y tamaño del *kernel*

El primer hiperparámetro es el tamaño de la cuadrícula, que está muy relacionado de forma directa con el error cometido en el entrenamiento estático ($r_s = 0,6598$). A medida que se procesan más celdas al mismo tiempo, la red tiene mayores dificultades para generalizar. Sin embargo, el tamaño de la cuadrícula afecta mucho menos al rendimiento de la red cuando se implementa *online learning* ($r_s = 0,1613$). Es un gran indicador de la capacidad de los modelos de adaptarse de forma dinámica a los cambios en los patrones de los datos independientemente de la cantidad de datos procesados.

Algo similar ocurre con el número de capas convolucionales de la red neuronal. A medida que se añaden más capas los modelos *online* mejoran, aunque no demasiado ($r_s = -0,098$). Al contrario de lo que pudiera pensarse, aumentar la capacidad de procesamiento de la red mediante la adición de capas CNN no es beneficioso. Una mayor capacidad de la red provoca *overfitting*, tal y como indica el coeficiente de Spearman entre el número de capas convolucionales y el error cometido por los modelos estáticos ($r_s = 0,2384$).

El siguiente hiperparámetro es el tamaño del *kernel*, es decir, el tamaño del campo de visión de las neuronas convolucionales y está estrechamente relacionado con el tamaño de la cuadrícula. La arquitectura formada únicamente por una capa de neuronas recurrentes LSTM no se ve afectada por este hiperparámetro. Un campo de visión grande en una cuadrícula pequeña suele ser contraproducente puesto que no permite realizar muchas convoluciones. A su vez, un *kernel* pequeño en una cuadrícula grande puede no ser capaz de captar patrones extendidos sobre áreas más extensas.

Para este trabajo se ha incluido en el *grid search* los datasets de 49 y 225 celdas y *kernels* de tamaño 2x2 y 3x3, enfocando la mayor parte del *grid* en el dataset de 49 celdas con un *kernel* 3x3. Un *kernel* mayor de 4x4 sería demasiado grande y no permitiría a las capas convolucionales obtener detalles del patrón de los datos. Obviamente, un *kernel* de 1x1 carece de sentido.

En la Figura 6 se puede observar la influencia tanto del tamaño de la cuadrícula como del campo de visión de las neuronas convolucionales. La influencia del tamaño del *kernel* no es tan grande como la del tamaño de la cuadrícula pero en todos los escenarios un *kernel* menor es ligeramente más eficiente aunque no es una mejora suficientemente grande como para ser tomada en cuenta. Además, su coeficiente de Spearman en entrenamiento estático ($r_s = -0,354$) es menor al del número de celdas y en *online learning* es completamente despreciable ($r_s = -0,0271$).

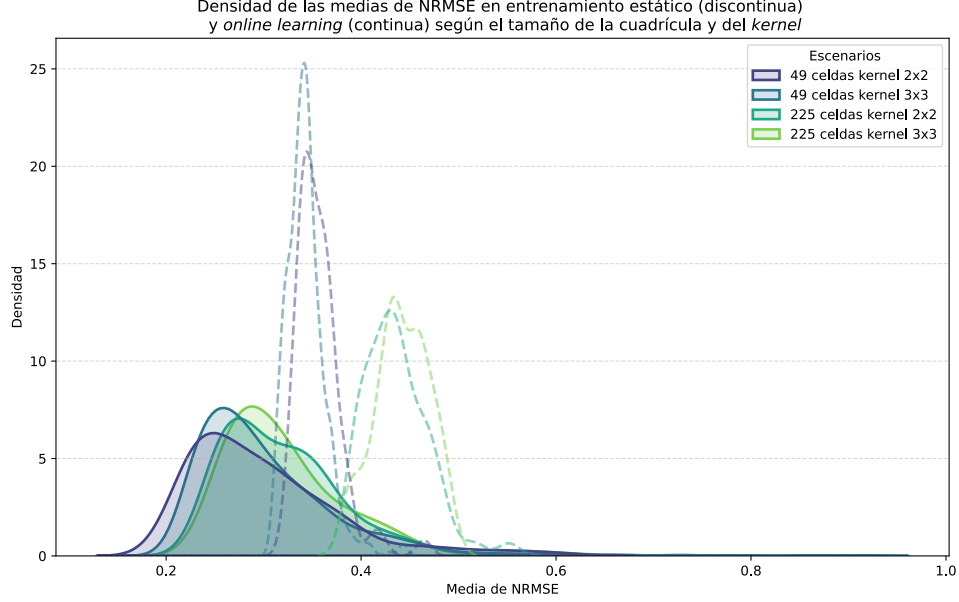


Figura 6: Densidad de las medias de NRMSE en entrenamiento estático (línea discontinua) y *online learning* (línea continua) según el tamaño de la cuadrícula y del kernel. Elaboración propia.

4.2.2. look_back y batch_size

Los hiperparámetros *look_back* y *batch_size* son los que mayor correlación tienen con la media de NRMSE de los modelos con *online learning* junto al hiperparámetro *updates*. Ambos hiperparámetros afecta a los modelos en todas sus fases de entrenamiento y validación. El parámetro *look_back* indica la longitud en horas de las secuencias temporales que la red neuronal procesa y *batch_size* indica la cantidad de esas secuencias que se procesan al mismo tiempo.

Los datos utilizados presentan una componente periódica marcada de forma diaria y semanal. Por lo tanto, es razonable ajustar el parámetro *look_back* a dicha frecuencia. En el presente trabajo se ha probado con valores entre 12 y 32 horas con la intención de permitir que la red aprenda la componente diaria. La componente semanal queda fuera de consideración puesto que un valor grande del hiperparámetro *look_back* impondría una carga computacional excesiva, pudiendo aumentar demasiado el tiempo de procesamiento de los datos.

La tendencia de la media de NRMSE en los modelos con *online learning* respecto al valor de *look_back*, tal y como indica su coeficiente de Spearman ($r_s = -0,2465$),

es inversa. Aun así, su coeficiente de Spearman es muy alto con respecto al tiempo de ejecución ($r_s = 0,5053$). Como se observa en la Figura 7, aunque aumentar la longitud de las series temporales analizadas lleve a una mayor precisión, el tiempo de ejecución aumenta demasiado. Los mejores valores para el hiperparámetro `look_back` están entre 18 y 24 horas de longitud.

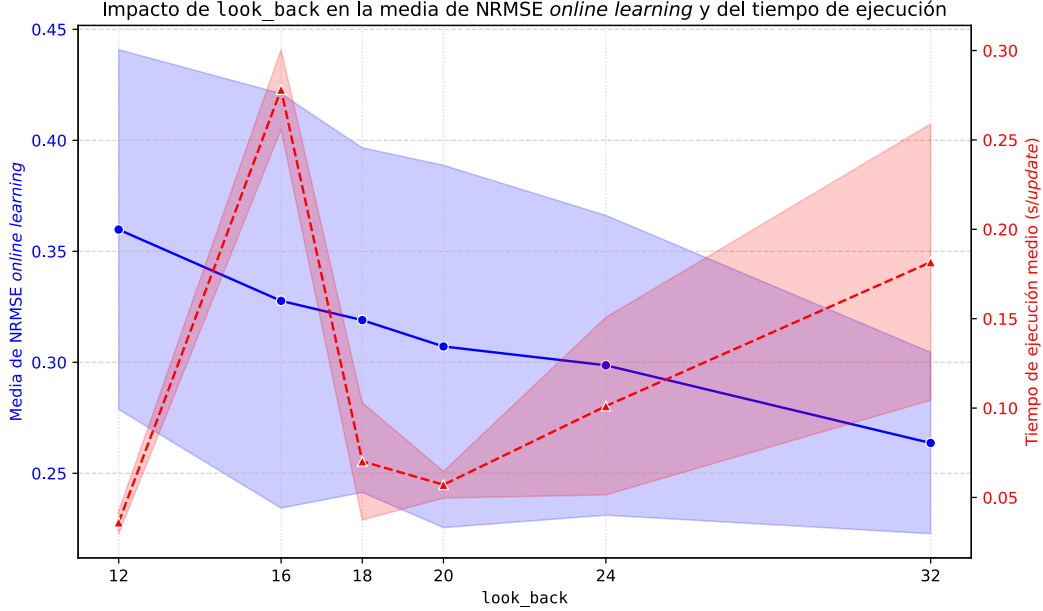


Figura 7: Influencia del hiperparámetro `look_back` en la media del NRMSE y en el tiempo de ejecución medio en modelos con *online learning*. Elaboración propia.

Si nos fijamos en el impacto de `look_back` en la media del NRMSE en modelos con entrenamiento estático es completamente al revés que en modelos con *online learning*. Su coeficiente de Spearman positivo ($r_s = 0,3183$) indica el potencial de *overfitting* latente en el hiperparámetro. Cuando la red neuronal procesa secuencias demasiado largas aprende demasiado bien los datos de entrenamiento, como así lo indica su coeficiente de Spearman ($r_s = -0,2321$) y no generaliza correctamente.

Encontrar valores para incluir en el *grid search* del hiperparámetro `batch_size` no es tarea sencilla. Continuando con el trabajo del Dr. D. Juan Pablo Casaseca de la Higuera [8], en este trabajo se han probado valores en torno a *batches* de 25 secuencias temporales. Al igual que con el hiperparámetro `look_back`, su coeficiente de Spearman respecto al tiempo de ejecución es positivo ($r_s = 0,1711$) por lo que *batches* de muchas secuencias temporales no son viables.

Cada *batch* contiene una cantidad limitada de secuencias temporales dispuestas de una forma peculiar. Cada secuencia está ordenada de tal forma que crean una ventana deslizante en los datos, por ello en el Capítulo 3 se ha denominado al hiperparámetro `look_back` como tal. De esta forma, al escoger valores en torno a 25 para `batch_size`, la red neuronal procesa en cada *batch* varias series temporales parcialmente superpuestas que abarcan aproximadamente 48 horas desde la primera muestra de la primera secuencia hasta la última muestra de la última secuencia. Esto permite a la red neuronal analizar el patrón subyacente de dos días naturales.

Esto es especialmente importante en los modelos con *online learning* puesto que, tras el entrenamiento, **batch_size** dicta la cantidad de información a la que la red neuronal tiene acceso para ajustar sus pesos. Cuanta más información tiene la red, más precisos son sus ajustes ($r_s = -0,1993$). Sin embargo, en el entrenamiento ($r_s = -0,0055$) y test sin *online learning* ($r_s = -0,0087$) el impacto es despreciable puesto que la red neuronal tiene acceso a todo el dataset de entrenamiento.

4.2.3. updates y threshold

Los hiperparámetros **updates** y **threshold** sólo tienen efecto en los modelos con *online learning*. Por ello, aunque algunos de sus coeficientes de Spearman con las métricas de modelos *offline* sean altos, es una coincidencia llamada relación espuria.

Updates indica cuántas veces se procesa el mismo *batch* en *online learning*. Cada vez que se procesa un *batch* se realiza el proceso de *backpropagation* una vez. Por lo tanto, cuantas más veces se realice, mayor precisión tendrá el ajuste de pesos y mayor será la carga computacional y el tiempo de ejecución.

Los coeficientes de Spearman de **updates** respecto a la media de RNMSE con *online learning* ($r_s = -0,3473$) y al tiempo de ejecución ($r_s = 0,5482$) indican que es uno de los hiperparámetros más importantes aunque también uno de los más peligrosos. Al igual que **look_back**, cuantas más veces se ajuste la red neuronal al *batch* actual, mayor será su precisión pero mayor será su coste computacional. Además, el coste computacional también está agravado por la arquitectura de la red neuronal, a mayor cantidad de capas se necesitan más cálculos para realizar *backpropagation*.

Como se observa en la Figura 8, los mejores resultados se encuentran a mayor número de **updates** pero la carga computacional es mayor. Además, se ha de tener en cuenta otros factores que agravan este problema como el tamaño de la cuadrícula, teniendo también un coeficiente de Spearman respecto al tiempo de ejecución bastante alto ($r_s = 0,3127$). Esto no suele ser un problema en modelos desplegados en entornos reales puesto que los procesadores suelen tener más capacidad de procesamiento y no tienen las restricciones impuestas en el Capítulo 3 para la reproducibilidad de los resultados.

En cuanto al hiperparámetro **threshold**, indica si el *batch* actual ha de seguir siendo procesado. No es un hiperparámetro que haya generado un gran impacto puesto que su coeficiente de Spearman es despreciable ($r_s = 0,0362$). Como las actualizaciones de los pesos se han realizado siguiendo el modelo propuesto por el Dr. D. Juan Pablo Casaseca de la Higuera [8], las actualizaciones de pesos provocadas por errores pequeños son menos potentes que las realizadas por los errores mayores. Por lo tanto, aunque el hiperparámetro **threshold** elimine las actualizaciones provocadas por errores pequeños, no son actualizaciones influyentes. Sin embargo, al eliminarlas se libera una gran cantidad de carga computacional, como se puede comprobar a través de su coeficiente de Spearman correspondiente ($r_s = -0,2177$).

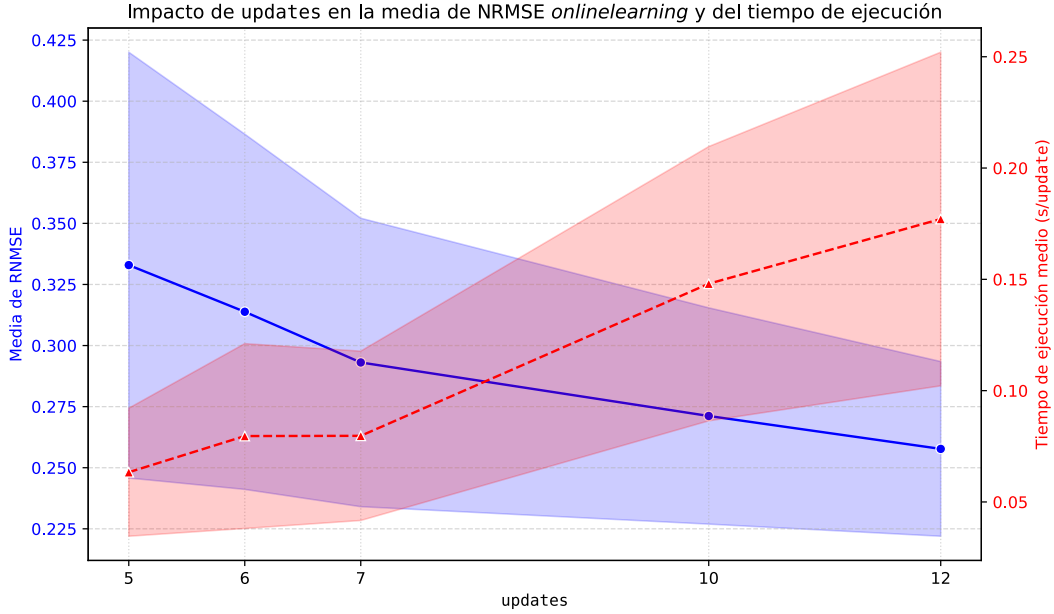


Figura 8: Influencia del hiperparámetro **updates** en la media del NRMSE y en el tiempo de ejecución medio en modelos con *online learning*. Elaboración propia.

4.2.4. Número de filtros de las capas convolucionales

El número de filtros indica la capacidad de las capas convolucionales para analizar patrones geográficos en la cuadrícula analizada. Sin embargo, un gran número de filtros puede provocar *overfitting*. Cuando hay demasiados filtros la red comienza a identificar el ruido presente en los datos como un patrón legítimo.

En cuanto al tiempo de ejecución, el resultado es contraintuitivo ($r_s < 0$). La respuesta está en la librería **Tensorflow** y la forma en que realiza los ajustes en los modelos con *online learning*. En cada modelo, la primera iteración de *online learning* es extremadamente lenta puesto que **Tensorflow** realiza un mapa de la red neuronal y lo convierte en una serie de funciones altamente optimizadas con el algoritmo *Tracing Graph*. En las siguientes iteraciones ya tiene el mapa hecho y lo ejecuta velozmente. Como se ha argumentado antes, las redes con más filtros son más propensas al *overfitting*, lo que aumenta el error NRMSE en las predicciones. Errores mayores tienen más probabilidad de superar el valor del hiperparámetro **threshold**, lo que genera más iteraciones de *online learning*. Todas esas iteraciones adicionales son extremadamente rápidas en comparación con la primera iteración, lo que reduce la media del tiempo de ejecución [23, 45].

4.3. Mejor modelo

El grueso de las medias de NRMSE se encuentra entre 0.20 y 0.50. En sí no son valores satisfactorios puesto que el objetivo se encuentra en un $\text{NRMSE} < 0.1778$. Sin embargo, aunque en media no se alcance el objetivo, es probable que, escogiendo los modelos con mejor media de NRMSE, algunas celdas superen el objetivo.

Los modelos sin capas convolucionales se quedan atrás en su rendimiento medio respecto a los modelos con capas convolucionales como se observa en la Figura 5. La cuadrícula de 49 celdas también es la que resulta más sencilla de procesar para las redes neuronales (*véase Figura 4*).

El modelo con menor media de NRMSE en *online learning* a lo largo de todas sus celdas es un modelo con una capa convolucional y otra capa de neuronas LSTM con la siguiente configuración:

Hiperparámetro	Valor
Tamaño cuadrícula	49
Capas CNN	1
Tamaño kernel	2
look_back	24
batch_size	25
updates	12
threshold	0.1
Filtros capa 1	16
Filtros capa 2	–
Filtros capa 3	–
Métricas de Rendimiento	
NRMSE entrenamiento	0.1221
NRMSE test <i>offline</i>	0.3487
NRMSE test <i>online</i>	0.2048
Tiempo de ejecución medio (s)	0.0768

Tabla 2: Características del mejor modelo en *online learning* y sus métricas de rendimiento como media del NRMS de sus celdas. Elaboración propia.

Podemos observar en la Figura 9 cómo se reparte la media NRMSE=0,2048 entre las 49 celdas. Cada celda tiene un dos números, el superior indicando el identificador de la celda en el dataset Milán y el inferior indicando el NRMSE. No hay nada que indique una distribución concreta ni una relación clara entre la posición de las celdas y su NRMSE. Destacan como celdas con menor NRMSE las celdas 5263, 5163 y 4962 y las celdas 5357, 5058 y 4959 como celdas con peor NRMSE.

NRMSE por celda del mejor modelo *online learning*

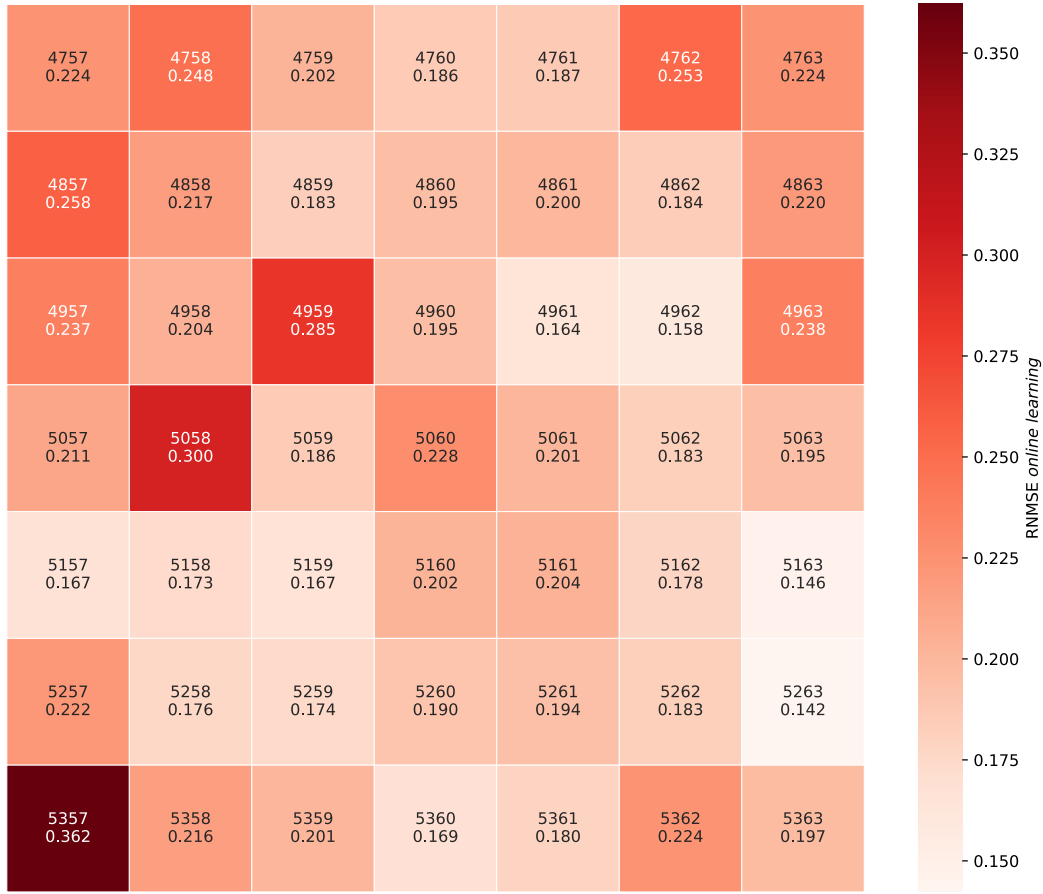


Figura 9: NRMSE por celda del mejor modelo con *online learning*. Elaboración propia.

Al comparar el diagrama de calor de la Figura 9 con los diagramas de calor obtenidos en los mejores modelos de *online learning* de otras arquitecturas, se aprecia que todas las arquitecturas tienen más o menos la misma distribución de errores. Las mejores celdas siempre son las celdas 5263, 5163 y 4962 y las peores celdas siempre son las celdas 5357, 5058 y 4959, como se observa en la Figura 10.

NRMSE por celda de los mejores modelos *online learning*

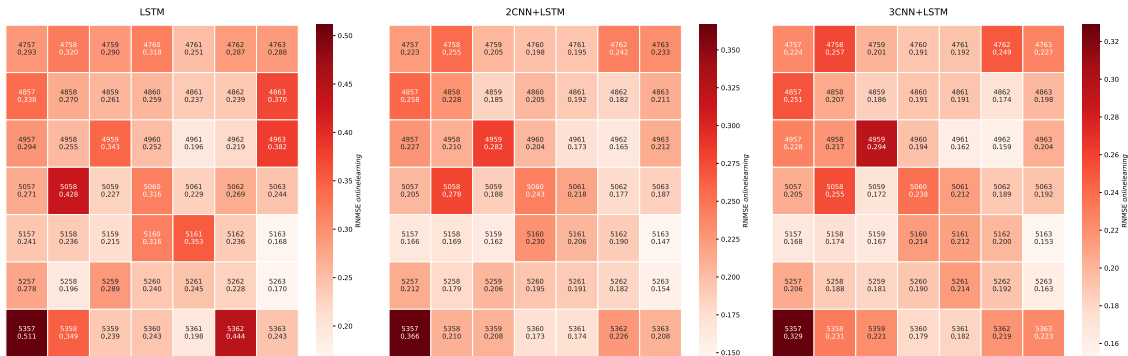


Figura 10: NRMSE por celda de los mejores modelos con *online learning* de cada arquitectura (LSTM, 2CNN+LSTM y 3CNN+LSTM). Elaboración propia.

Para obtener más detalles es necesario observar las predicciones realizadas en cada celda. No es necesario ni eficiente analizar todas las celdas por lo que sólo se escrutarán las 3 mejores y las 3 peores. Como el modelo tiene una arquitectura con una capa convolucional con un kernel 3x3 y una capa recurrente LSTM, es interesante comparar las celdas analizadas con las celdas contiguas para intentar estimar qué datos han tenido en cuenta las neuronas convolucionales.

Respecto a las mejores celdas, las celdas 5263 y 5163 se influyen mutuamente al ser contiguas. Observando los datos de test, ambas tienen un marcado patrón periódico diario y semanal, en el que los primeros cinco días tienen más tráfico que los dos días siguientes. Mirando las celdas contiguas a las mismas, las celdas 5063, 5062 y 5363 presentan el mismo patrón. Las celdas 5162 y 5262 presentan un patrón en el que los siete días de la semana son prácticamente idénticos con mínimos más bajos que las otras celdas. Observando las predicciones sin *online learning*, parece que éstas últimas han influido en las predicciones de las celdas 5263 y 5163 y la red neuronal ha infravalorado el tráfico de red futuro.

Respecto a las peores celdas, las celdas 4959 y 5357 no son fáciles de predecir por ninguna red neuronal. Los datos de test de la celda 4959 contiene mucho ruido y cambios bruscos y rápidos alrededor de los mínimos de tráfico y la celda 5057 presenta un cambio radical en el patrón subyacente de los datos. Los modelos sin *online learning* no son capaces de soportar un cambio tan grande. Aun así, parece que aunque la celda 5357 tiene un patrón más o menos uniforme antes del cambio, la influencia del patrón en forma de 5 y 2 días de las celdas adyacentes se puede ver en las predicciones sin *online learning*.

En general, la mayor contribución al NRMSE es provocado por anomalías en los patrones, como por ejemplo los máximos abruptos de la celda 5058. Sin embargo, es visible en todas las celdas cómo los modelos con *online learning* son muy superiores a aquellos sin *online learning*. Esto se observa muy bien en las celdas con menor precisión, por ejemplo, el modelo con *online learning* ha podido adaptarse rápidamente los cambios radicales de las celdas 5357 y 5362 en cuestión de 2 o 3 días. Por otra parte, el modelo sin *online learning* son completamente incapaces de adaptarse.

5. Conclusiones y líneas futuras

Este trabajo ha plasmado la investigación realizada en torno a las redes neuronales compuestas por capas convolucionales y recurrentes aplicadas a la predicción del tráfico en red con *online learning*. Se han analizado varias arquitecturas de redes neuronales con una capa de neuronas LSTM precedida por una cantidad variable entre ninguna y tres capas CNN. Se ha realizado un extenso proceso de validación basado en la estrategia *grid search* sobre el dataset Milán y se ha obtenido el modelo con mayor precisión utilizando el NRMSE como métrica de rendimiento.

Como regla general, los modelos con *online learning* son ampliamente superiores a aquellos sin *online learning*. Los cambios en el patrón de los datos pasan completamente desapercibidos para los modelos sin *online learning*. En cambio, los modelos con *online learning* presentan una capacidad de adaptación extraordinaria.

La implementación de capas convolucionales al modelo de predicción de series temporales basado únicamente en capas de neuronas LSTM ha dado resultado, reduciendo los errores cometidos en las predicciones. Respecto al modelo original cedido por el Dr. D. Juan Pablo Casaseca de la Higuera, se ha conseguido reducir el error de predicción desde $\text{NRMSE}=0,1778$ hasta $\text{NRMSE}=0,142$.

Por último, se ha alcanzado la optimización del modelo original a través de funciones altamente eficientes proporcionadas por la librería **Tensorflow**. Gracias a ello se ha conseguido reducir considerablemente el tiempo de ejecución requerido por los modelos para los procesos de *backpropagation* y *online learning*.

Para próximos trabajos se plantea la posibilidad de implementar los modelos descritos en este trabajo como redes de grafos orientadas a la predicción de series temporales, más concretamente a la predicción de tráfico de red.

Referencias

- [1] G. Bell, “Bell’s law for the birth and death of computer classes,” *Communications of the ACM*, vol. 51, no. 1, pp. 86–94, Jan 2008. [Online]. Available: <https://gordonbell.azurewebsites.net/cacm%20bell’s%20law%20vol%2051.%202008-january.pdf>
- [2] A. Azzouni and G. Pujolle, “A long short-term memory recurrent neural network framework for network traffic matrix prediction,” *arXiv preprint arXiv:1705.05690*, Jun 2017. [Online]. Available: <http://arxiv.org/abs/1705.05690>
- [3] Y. S. Abu-Mostafa, M. Magdon-Ismail, and H. T. Lin, *Learning from Data: A Short Course*. Amlbook.com, 2012.
- [4] N. Bui, M. Cesana, S. A. Hosseini, Q. Liao, I. Malanchini, and J. Widmer, “A survey of anticipatory mobile networking: context-based classification, prediction methodologies, and optimization techniques,” *IEEE Communications Surveys & Tutorials*, vol. 19, no. 3, pp. 1790–1821, 2017.
- [5] T. Guo and X. Xu, “Robust online time series prediction with recurrent neural networks,” in *2016 IEEE International Conference on Data Mining (ICDM)*, 2016, pp. 1001–1006.
- [6] Y. Duan, G. Fu, and N. Zhou, “Everything as a service(xaas) on the cloud: Origins, current and future trends,” in *2015 IEEE 8th International Conference on Cloud Computing*. IEEE Computer Society, 2015, pp. 621–628.
- [7] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, “Learning under concept drift: A review,” *IEEE Transactions on Knowledge and Data Engineering*, 2018, arXiv:2004.05785.
- [8] P. Casaseca-de-la Higuera, J. M. Aguiar, P. Amado-Caballero, A. J. Morgado, J. d. S. Moreira, C. Luo, and X. Wang, “Online active learning for lstm-based network traffic prediction,” in *IEEE (Unpublished Draft / Preprint)*, 20XX.
- [9] M. González and I. M. del Puerto, *Series Temporales*, ser. Colección manuales uex. Universidad de Extremadura, 2009, vol. 60.
- [10] J. C. García Díaz, *Predicción en el dominio del tiempo: análisis de series temporales para ingenieros*, ser. Manual de referencia. Valencia: Editorial de la Universidad Politécnica de Valencia, 2016.
- [11] A. Géron, *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems*, 3rd ed. Beijing: O’Reilly, 2022.
- [12] P. K. Hoong, I. K. T. Tan, and C. Y. Keong, “Bittorrent network traffic forecasting with arma,” *International journal of Computer Networks & Communications*, vol. 4, pp. 143–156, Jul 2012.

- [13] H. O. A. Wold, *A Study in the Analysis of Stationary Time Series*. Uppsala, Sweden: Almqvist & Wiksell, 1938. [Online]. Available: <https://archive.org/details/in.ernet.dli.2015.262214>
- [14] K. Poo, I. K. T. Tan, and K. Ng, “Computing autoregressive moving average (arma) with cronos: A case study of bursa malaysia,” in *2nd International Conference on Engineering & ICT 2010 (ICEI)*, Melaka, Malaysia, 2010, pp. 516–520.
- [15] M. Papadopoulou, H. Shen, E. Raftopoulos, M. Ploumidis, and F. H. Campos, “Short term traffic forecasting in a campus wide wireless network,” in *16th IEEE Annual International Symposium on Personal Indoor and Mobile Radio Communications*, Berlin, Germany, Sep 2005.
- [16] S. Jung, C. Kim, and Y. Chung, “A prediction method of network traffic using time series models,” in *Computational Science and Its Applications (ICCSA)*, ser. Lecture Notes in Computer Science (LNCS). Springer, 2006, pp. 234–243.
- [17] Y. Yu, J. Wang, M. Song, and J. Song, “Network traffic prediction and result analysis based on seasonal arima and correlation coefficient,” in *2010 International Conference on Intelligent System Design and Engineering Application*, vol. 1, 2010, pp. 980–983.
- [18] T. M. Mitchell, *Machine Learning*. New York: McGraw-Hill, 1997. [Online]. Available: <http://www.cs.cmu.edu/~tom/mlbook.html>
- [19] A. L. Samuel, “Some studies in machine learning using the game of checkers,” *IBM Journal of Research and Development*, vol. 3, no. 3, pp. 210–229, Jul 1959. [Online]. Available: <https://ieeexplore.ieee.org/document/5392560>
- [20] Y. S. Abu-Mostafa, M. Magdon-Ismail, and H. T. Lin, *Learning from Data: A Short Course*. Amlbook.com, 2012.
- [21] J. Bergstra and Y. Bengio, “Random search for hyper-parameter optimization,” *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012. [Online]. Available: <http://jmlr.org/papers/v13/bergstra12a.html>
- [22] D. H. Wolpert, “The lack of a priori distinctions between learning algorithms,” *Neural Computation*, vol. 8, no. 7, pp. 1341–1390, 10 1996. [Online]. Available: <https://doi.org/10.1162/neco.1996.8.7.1341>
- [23] F. Chollet, *Deep learning with Python*, 2nd ed. Madrid: Anaya Multimedia, 2020.
- [24] L. N. Smith, “A disciplined approach to neural network hyper-parameters: Part 1 - learning rate, batch size, momentum, and weight decay,” *CoRR*, vol. abs/1803.09820, 2018. [Online]. Available: <http://arxiv.org/abs/1803.09820>
- [25] W. S. McCulloch and W. Pitts, “A logical calculus of the ideas immanent in nervous activity,” *Bulletin of Mathematical Biophysics*, vol. 5, no. 4, pp. 115–133, Dec 1943. [Online]. Available: <https://doi.org/10.1007/BF02478259>

- [26] F. Rosenblatt, “The perceptron: A perceiving and recognizing automaton (project PARA),” Cornell Aeronautical Laboratory, Buffalo, NY, Report 85-460-1, Jan 1957. [Online]. Available: <https://bpb-us-e2.wpmucdn.com/websites.umass.edu/dist/a/27637/files/2016/03/rosenblatt-1957.pdf>
- [27] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning internal representations by error propagation,” Institute for Cognitive Science, University of California, San Diego, La Jolla, CA, Technical Report ICS-8506, Sep 1985. [Online]. Available: <https://apps.dtic.mil/sti/citations/ADA164453>
- [28] —, “Learning internal representations by error propagation,” Institute for Cognitive Science, University of California, San Diego, CA, Technical Report ICS-8506, Sep 1985. [Online]. Available: <https://apps.dtic.mil/sti/citations/ADA164453>
- [29] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov 1998. [Online]. Available: <https://ieeexplore.ieee.org/document/726791>
- [30] K. Fukushima, “Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position,” *Biological Cybernetics*, vol. 36, no. 4, pp. 193–202, Apr 1980. [Online]. Available: <https://doi.org/10.1007/BF00344251>
- [31] V. Pham, T. Bluche, C. Kermorvant, and J. Louradour, “Dropout improves recurrent neural networks for handwriting recognition,” in *14th International Conference on Frontiers in Handwriting Recognition (ICFHR)*. IEEE, Sep 2014, pp. 285–290. [Online]. Available: <https://ieeexplore.ieee.org/document/6981034/>
- [32] Q. V. Le, N. Jaitly, and G. E. Hinton, “A simple way to initialize recurrent networks of rectified linear units,” 2015. [Online]. Available: <https://arxiv.org/abs/1504.00941>
- [33] N. Ramakrishnan and T. Soni, “Network traffic prediction using recurrent neural networks,” in *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*. Orlando, FL, USA: IEEE, Dec 2018, pp. 187–193. [Online]. Available: <https://ieeexplore.ieee.org/document/8614060/>
- [34] A. Román Antolín, “Predicción de tráfico de red usando arquitecturas neuronales avanzadas,” Trabajo Fin de Grado, Universidad de Valladolid, Sep 2025.
- [35] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov 1997. [Online]. Available: <https://doi.org/10.1162/neco.1997.9.8.1735>
- [36] TensorFlow Developers, “tf.keras.layers.timedistributed,” Jul 2026. [Online]. Available: https://www.tensorflow.org/api_docs/python/tf/keras/layers/TimeDistributed

- [37] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, Mar 2014. [Online]. Available: <https://dl.acm.org/doi/epdf/10.1145/2523813>
- [38] T. Italia, “Telecommunications - SMS, Call, Internet - MI,” 2015. [Online]. Available: <https://doi.org/10.7910/DVN/EGZHFV>
- [39] —, “Milano Grid,” 2015. [Online]. Available: <https://doi.org/10.7910/DVN/QJWLFU>
- [40] Keras Team. (2024) LSTM layer: Keras recurrent layers API. [Accessed: Jun. 20, 2026]. [Online]. Available: https://keras.io/api/layers/recurrent_layers/lstm/
- [41] K. Sijtsma and W. Emons, “Nonparametric statistical methods,” in *International Encyclopedia of Education (Third Edition)*, third edition ed., P. Peterson, E. Baker, and B. McGaw, Eds. Oxford: Elsevier, 2010, pp. 347–353. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/B9780080448947013531>
- [42] Kaggle. (2026) Kaggle notebooks documentation. [Accessed: Jun. 19, 2026]. [Online]. Available: <https://www.kaggle.com/docs/notebooks>
- [43] TensorFlow Developers. (2022, May) What’s new in TensorFlow 2.9. [Accessed: Jun. 19, 2026]. [Online]. Available: <https://blog.tensorflow.org/2022/05/whats-new-in-tensorflow-29.html>
- [44] NVIDIA Corporation. (2026) cudnn backend API developer guide: Miscellaneous. [Accessed: Jun. 19, 2026]. [Online]. Available: <https://docs.nvidia.com/deeplearning/cudnn/backend/latest/developer/misc.html>
- [45] TensorFlow Developers, “Better performance with tf.function,” Jul 2026. [Online]. Available: <https://www.tensorflow.org/guide/function>

A. Fragmentos de código

Se muestran en este apéndice los diferentes *scripts* de Python utilizados para la construcción de los modelos analizados en este trabajo así como para la implementación de *online learning* y la estrategia *grid search*.

A.1. Código Python del modelo original

```
1 import matplotlib
2 matplotlib.use('TkAgg')
3 import matplotlib.pyplot as plt
4 import numpy as np
5 import pandas as pd
6 import tensorflow as tf
7 from tensorflow.keras.models import Sequential
8 from tensorflow.keras.layers import Dense
9 from tensorflow.keras.layers import LSTM
10 from sklearn.preprocessing import MinMaxScaler
11 from sklearn.metrics import mean_squared_error
12 import time as tt
13
14
15 from tensorflow.python.client import device_lib
16 def get_available_devices():
17     local_device_protos = device_lib.list_local_devices()
18     return [x.name for x in local_device_protos]
19 print(get_available_devices())
20
21 # fix random seed for reproducibility
22 tf.random.set_seed(7)
23
24
25 # load the dataset
26 dataframe = pd.read_csv('Traffic3pixels.csv', usecols=[1,2,3],
27     engine='python')
28 dataset = dataframe.values
29 dataset = dataset.astype('float32')
30
31 # normalize the dataset
32 scaler = MinMaxScaler(feature_range=(0, 1))
33 dataset = scaler.fit_transform(dataset)
34 dataset=dataset[0:1400]
35 # split into train and test sets
36 train_size = 400#int(len(dataset) * 0.7)
37 test_size = 1000#len(dataset) - train_size
38 train, test = dataset[0:train_size, :], dataset[train_size:
39     train_size+test_size, :]
```

```

40 # Definir la funcion de perdida RMSE
41 def rmse_loss(y_true, y_pred):
42     return tf.sqrt(tf.reduce_mean(tf.square(y_true - y_pred)))
43 # Configurar el optimizador
44 optimizer = tf.keras.optimizers.Adam()
45
46
47
48 # convert an array of values into a dataset matrix
49 def create_dataset(dataset, look_back=1):
50     dataX, dataY = [], []
51     for i in range(len(dataset) - look_back - 1):
52         a = dataset[i:(i + look_back), :]
53         dataX.append(a)
54         dataY.append(dataset[i + look_back, :])
55     return np.array(dataX), np.array(dataY)
56
57 # reshape into X=t and Y=t+1
58 look_back = 24 #este parametro, permite definir una ventana
    deslizante
59 trainX, trainY = create_dataset(train, look_back)
60 testX, testY = create_dataset(test, look_back)
61
62 n_epochs=400
63 batch_size=25
64 # create and fit the LSTM network
65 model = Sequential()
66 model.add(LSTM(look_back, input_shape=(look_back,trainX.shape
    [2]),batch_input_shape=(batch_size, trainX.shape[1], trainX
    .shape[2]),stateful=True))
67 model.add(Dense(trainX.shape[2]))
68 model.compile(loss='mean_squared_error', optimizer='adam')
69
70 model.summary()
71 n_epochs_update=5
72 # Error threshold
73 error_threshold =0
74 rmse_norm=1
75
76
77
78
79 #El reseteo del estado debe hacerse asi cada epoch. Hacerlo es
    recomendable, si bien para datasets pequenos no es
    relevante
80 print('Fitting model on verbose 0')
81 for i in range(n_epochs):
82     model.fit(trainX, trainY, epochs=1, batch_size=batch_size,
        verbose=0, shuffle=True)
83     model.reset_states()
84
85 # make predictions

```



```

86 trainPredict = model.predict(trainX, batch_size=batch_size)
87 testPredict = model.predict(testX, batch_size=batch_size)
88 # invert predictions
89 trainPredict = scaler.inverse_transform(trainPredict)
90 trainYground = scaler.inverse_transform(trainY)
91 testPredict = scaler.inverse_transform(testPredict)
92 testYground = scaler.inverse_transform(testY)
93
94 trainX_copy=trainX[-batch_size:,:,:,:]
95 trainY_copy=trainY[-batch_size:,:]
96 predictions_score=list()
97 times=list()
98 print('Applying SKIMA backpropagation')
99 for i in range(testY.shape[0]-batch_size):
100     if rmse_norm > error_threshold:
101         #print(f"RMSE NORM for sample {i}: {rmse_norm}")
102         st=tt.time()
103         for j in range(n_epochs_update):
104             with tf.GradientTape() as tape:
105                 predictions = model(trainX_copy, training=True)
106                 loss = rmse_loss(trainY_copy, predictions)
107
108                 gradients = tape.gradient(loss, model.
109                     trainable_variables)
110                 # Modifica los gradientes segun rmse_norm
111                 adjusted_gradients = [g * (rmse_norm**2) for g in
112                     gradients]
113                 #adjusted_gradients = [g * (1) for g in gradients]
114                 optimizer.apply_gradients(zip(adjusted_gradients,
115                     model.trainable_variables))
116                 model.reset_states()
117
118             et=tt.time()
119             elapsed=et-st
120             times.append(elapsed)
121
122         # predict
123         X, y = testX[i:i+batch_size,:,:], testY[i:i+batch_size,:]
124         yhat = model.predict(X, batch_size=batch_size)
125         yhat = scaler.inverse_transform(yhat)
126         actual = scaler.inverse_transform(y)
127         rmse_norm = ((np.sqrt(np.mean(((actual[-1,:] - yhat[-1,:])/
128             yhat[-1,:])**2))))
129
130         predictions_score.append(yhat[-1])
131         if i==0:
132             trainX_copy = X #Meto solo un batch en el nuevo
133                             #entrenamiento (no lo meneo mucho)
134             trainY_copy = y
135         else:
136             trainX_copy = np.vstack((trainX_copy[1:,:,:], X
137                 [-1,:,:].reshape(1,look_back,-1)))

```

```

131         trainY_copy = np.vstack((trainY_copy[1:,:], np.reshape
            (y[-1,:],(1,3))))
132
133 predictions_score=np.array(predictions_score)
134 times=np.array(times)
135
136 print(predictions_score[:,0].shape)
137 print(testYground[look_back:,0].shape)
138 print(testPredict.shape)
139 print(trainYground.shape)
140 print(trainPredict.shape)
141 # calculate root mean squared error
142 trainScore=np.empty(3)
143 testScore=np.empty(3)
144 testScore_inc=np.empty(3)
145 for i in range(testY.shape[1]):
146     trainScore[i] = np.sqrt(mean_squared_error(trainYground[:,
        i], trainPredict[:,i]))/np.std(trainYground[:,i])
147     print('Train Score: %.4f RNMSE' % (trainScore[i]))
148     testScore[i] = np.sqrt(mean_squared_error(testYground[:,i
        ], testPredict[:,i]))/np.std(testYground[:,i])
149     print('Test Score: %.4f RNMSE' % (testScore[i]))
150     testScore_inc[i] = np.sqrt(mean_squared_error(testYground[
        look_back:-1,i], predictions_score[:,i]))/np.std(
        testYground[look_back:-1,i])
151     print('Test Score incremental: %.4f RNMSE' % (
        testScore_inc[i]))
152     # Calculate average time elapsed in each incremental
        training
153     timeScore = np.mean(times)
154 print('Average incremental training time: %.4f s' % (timeScore
        ))
155
156
157
158 # shift train predictions for plotting
159 trainPredictPlot = np.empty_like(dataset)
160 trainPredictPlot[:, :] = np.nan
161 trainPredictPlot[look_back-1:len(trainPredict)+look_back-1, :]
        = trainPredict
162 # shift test predictions for plotting
163 testPredictPlot = np.empty_like(dataset)
164 testPredictPlot[:, :] = np.nan
165 testPredictPlot[len(trainPredict)+(look_back*2):len(dataset)
        -2, :] = testPredict
166 testPredictPlotInc = np.empty_like(dataset)
167 testPredictPlotInc[:, :] = np.nan
168 testPredictPlotInc[len(trainPredict)+(look_back*2)+batch_size:
        len(dataset)-2, :] = predictions_score
169 dataplot=scaler.inverse_transform(dataset)
170 real_data_for_comparison=dataplot[train_size+(2*look_back)
        +1:-1,:]

```

```

171 testPredict_for_comparison=testPredict[look_back+1:,:]
172
173
174 # Crear un DataFrame con los resultados
175 results = {
176     'Train Score': trainScore,
177     'Test Score': testScore,
178     'Test Score incremental': testScore_inc,
179     'Time Score': [timeScore] * len(trainScore) # Repetir el
        valor de timeScore para todas las filas
180 }
181
182 df = pd.DataFrame(results)
183
184
185
186 # Guardar el DataFrame en un archivo Excel
187 filename = f'resultados{train_size}_{n_epochs}_{
        n_epochs_update}_{error_threshold}_GWOL.xlsx'
188 df.to_excel(filename, index=False)
189 print(real_data_for_comparison.shape)
190 print(testPredict_for_comparison)
191 print(predictions_score)
192 # Combine arrays to compare horizontally (axis=1)
193 combined_array = np.hstack((real_data_for_comparison,
        testPredict_for_comparison, predictions_score))
194 # Convert the combined array to a pandas DataFrame
195 df = pd.DataFrame(combined_array, columns=['Real1', 'Real2', '
        Real3','Pred1', 'Pred2', 'Pred3', 'PredInc1', 'PredInc2', '
        PredInc3'])
196 # Save the DataFrame to an Excel file
197
198 filename = f'resultados_for_plot{train_size}_{n_epochs}_{
        n_epochs_update}_{error_threshold}_GWOL.xlsx'
199 df.to_excel(filename, index=False)
200 print(f'Resultados guardados en {filename}')
201
202 labels=["RealData","Predicted training data","Predicted test
        data, no online learning","GWOL (our method)"]
203
204 fig,ax=plt.subplots(3,sharex=True)
205
206 for i in range(3):
207     # plot baseline and predictions
208     ax[i].plot(dataplot[:,i],label=labels[0])
209     ax[i].plot(trainPredictPlot[:,i],label=labels[1])
210     ax[i].plot(testPredictPlot[:,i],label=labels[2])
211     ax[i].plot(testPredictPlotInc[:,i],label=labels[3])
212
213 # Create a global legend below all subplots
214 fig.legend(labels, loc="lower center", ncol=2, bbox_to_anchor
        =(0.5, -0.15))

```

```

215 # Label for the X-axis
216 ax[0].set_ylabel("Region 4259")
217 ax[1].set_ylabel("Region 4456")
218 ax[2].set_ylabel("Region 5060")
219 ax[-1].set_xlabel("Time (h)")
220 # Adjust layout to fit the legend
221 fig.tight_layout()
222 #fig.subplots_adjust(global legend
223 fig.subplots_adjust(bottom=0.1)
224 # Add space for th
225
226 # Guardar la figura con un nombre que dependa de a y b
227 filename2 = f'resultados_{train_size}_{n_epochs}_{
    n_epochs_update}_{error_threshold}_GWOL.png' # Genera un
    nombre dinamico
228 plt.savefig(filename2, dpi=300, bbox_inches='tight') # Guarda
    la figura con alta calidad
229 plt.show()
230 print(f"Figura guardada como {filename2}")

```

A.2. Código Python del modelo LSTM adaptado para el entorno Kaggle

El código Python mostrado a continuación ha sido modificado para preservar la privacidad de las cuentas Kaggle utilizadas. Para que el modelo funcione correctamente se debe modificar el *path* especificado en la línea 72 cambiando `username` por un nombre de usuario real identificado en la plataforma Kaggle.

```
1 import random
2 import numpy as np
3 import pandas as pd
4 import tensorflow as tf
5 from tensorflow.keras.models import Sequential
6 from tensorflow.keras.layers import Dense
7 from tensorflow.keras.layers import LSTM, Input
8 from sklearn.preprocessing import MinMaxScaler
9 from sklearn.metrics import mean_squared_error
10 from tensorflow.python.client import device_lib
11 import time as tt
12 import itertools
13
14 import os
15 for dirname, _, filenames in os.walk('/kaggle/input'):
16     for filename in filenames:
17         print(os.path.join(dirname, filename))
18
19 #NOMENCLATURA
20 # _u --> unscaled (scaler.inverse_transform)
21 # _s --> static
22 # _o --> online learning
23 # nada --> scaled (scaler.transform)
24
25 # GET CPU AND GPU
26 def get_available_devices():
27     local_device_protos = device_lib.list_local_devices()
28     return [x.name for x in local_device_protos]
29
30 # FUNCION RMSE
31 def rmse_loss(y_true, y_pred):
32     return tf.sqrt(tf.reduce_mean(tf.square(y_true - y_pred)))
33
34 # ARRAY TO MATRIX
35 def create_dataset(dataset, look_back=1):
36     dataX, dataY = [], []
37     for i in range(len(dataset) - look_back - 1):
38         a = dataset[i:(i + look_back), :]
39         dataX.append(a)
40         dataY.append(dataset[i + look_back, :])
41     return np.array(dataX), np.array(dataY)
42
43 # Callback para resetear los estados de forma ultrarrapida
```

```

44 class ResetStatesCallback(tf.keras.callbacks.Callback):
45     def on_epoch_end(self, epoch, logs=None):
46         for layer in self.model.layers:
47             if hasattr(layer, 'reset_states'):
48                 layer.reset_states()
49
50
51 # tf.function para invocar Tracing Graphs
52 @tf.function
53 def online_train_step(model, optimizer, x, y, norm_sq):
54     with tf.GradientTape() as tape:
55         predictions = model(x, training=True)
56         loss = rmse_loss(y, predictions)
57
58         gradients = tape.gradient(loss, model.trainable_variables)
59         adjusted_gradients = [g * norm_sq for g in gradients]
60         optimizer.apply_gradients(zip(adjusted_gradients, model.
61                                     trainable_variables))
62
63 #Restricciones de reproducibilidad
64 os.environ['PYTHONHASHSEED'] = '0'
65 os.environ['TF_DETERMINISTIC_OPS'] = '1'
66 os.environ['TF_CUDNN_DETERMINISTIC'] = '1'
67 random.seed(7)
68 np.random.seed(7)
69 tf.random.set_seed(7)
70 tf.config.experimental.enable_op_determinism()
71
72 #Lectura del dataset
73 dataframe = pd.read_csv('/kaggle/input/datasets/username/
74                         traffic-7x7-1/Traffic_7x7_1.csv', usecols=range(1,50),
75                         engine='python')
76 dataset_full = dataframe.values
77 dataset_full = dataset_full.astype('float32')
78 num_features = dataset_full.shape[1] #7x7=49
79 total_rows = dataset_full.shape[0]
80
81 #GRID SEARCH EJEMPLO
82 hyperparameters = {
83     'look_back': [16, 18],
84     'batch_size': [22, 23, 24],
85     'n_epochs_update': [6, 7],
86     'threshold': [0.13, 0.15],
87 }
88
89 combinations = list(itertools.product(*hyperparameters.values
90                                         ()))
91 output_csv_file = '/kaggle/working/GridSearch_LSTM.csv'
92 grid_search_summary = []
93
94 #BUCLE GRID SEARCH

```

```

92 for combi in combinations:
93     #Borrar todo rastro de modelos de otras iteraciones para
        evitar influencias no autorizadas
94     tf.keras.backend.clear_session()
95
96     #Restricciones de reproducibilidad de nuevo, por si acaso
97     os.environ['PYTHONHASHSEED'] = '0'
98     os.environ['TF_DETERMINISTIC_OPS'] = '1'
99     os.environ['TF_CUDNN_DETERMINISTIC'] = '1'
100     random.seed(7)
101     np.random.seed(7)
102     tf.random.set_seed(7)
103     tf.config.experimental.enable_op_determinism()
104
105     #Combinacion de hiperparametros actual
106     params = dict(zip(hyperparameters.keys(), combi))
107     current_look_back = params['look_back']
108     current_batch_size = params['batch_size']
109     current_n_epochs_update = params['n_epochs_update']
110     current_threshold = params['threshold']
111
112     # Separacion entre train y test de forma automatica segun
        batch_size,
113     # el numero total de muestras de los datasets ha de ser
        divisible por batch_size
114     desired_train_size = 400
115     train_available_batches = (desired_train_size -
        current_look_back - 1) // current_batch_size
116     train_size = current_batch_size * train_available_batches
        + current_look_back + 1
117     #create_dataset se come las primeras look_back + 1
        muestras para crear el dataset de test/train
118     available_batches = (total_rows - current_look_back -1 -
        train_size) // current_batch_size
119     test_size = current_batch_size * available_batches +
        current_look_back +1
120
121     dataset = dataset_full[0:train_size+test_size]
122
123     scaler = MinMaxScaler(feature_range=(0, 1))
124     train = dataset[0:train_size, :]
125     test = dataset[train_size : train_size+test_size, :]
126
127     #EL MODELO ORIGINAL HACE FIT_TRANSFORM CON TODO EL DATASET
        --> DATA LEAKAGE
128     train = scaler.fit_transform(train)
129     test = scaler.transform(test)
130
131     # X=[T,T+LOOKBACK]   Y=[T+LOOCKBACK+1]
132     # look_back define cuantas muestras se utiliza para las
        series temporales
133     look_back = current_look_back #este parametro, permite

```

```

134         definir una ventana deslizando
135     trainX, trainY = create_dataset(train, look_back)
136     testX, testY = create_dataset(test, look_back)
137
138     #batch_size indica cuantas ventanas look_back se procesan
139     #antes de actualizar los pesos
140     batch_size = current_batch_size
141     # Cuantas vueltas al dataset entero se dan para entrenar
142     n_epochs = 400
143
144     # MODELO LSTM CON CAPA INPUT
145     myLSTM = Sequential()
146     myLSTM.add(Input(batch_shape=(batch_size, trainX.shape[1],
147     trainX.shape[2])))
148     myLSTM.add(LSTM(look_back, stateful=True))
149     myLSTM.add(Dense(trainX.shape[2]))
150     myLSTM.compile(loss='mean_squared_error', optimizer='adam',
151     )
152     myLSTM.summary()
153
154     #Entrenamiento optimizado ultrarrapido
155     #El reseteo del estado debe hacerse asi cada epoch.
156     #Hacerlo es recomendable, si bien para datasets pequenos
157     #no es relevante
158     print('Fitting myLSTM on verbose 0')
159     myLSTM.fit(
160         trainX, trainY,
161         epochs=n_epochs,
162         batch_size=batch_size,
163         verbose=0,
164         shuffle=True,
165         callbacks=[ResetStatesCallback()]
166     )
167     #El myLSTMos LSTM es stateful, es decir, que guarda sus
168     #estados h(t) y c(t) y los usa como valor inicial en el
169     #siguiente batch
170     #Como shuffle=True, el orden de los batches es aleatorio.
171     #Lo que significa que actua como un stateless.
172     #Shuffle=True mejora la generalizacion.
173
174     #TEST ESTATICO
175     train_Y_u = scaler.inverse_transform(trainY)
176     test_Y_u = scaler.inverse_transform(testY)
177
178     train_Yhat_s = myLSTM.predict(trainX, batch_size=
179     batch_size)
180     train_Yhat_su = scaler.inverse_transform(train_Yhat_s)
181
182     test_Yhat_s = myLSTM.predict(testX, batch_size=batch_size)
183     test_Yhat_su = scaler.inverse_transform(test_Yhat_s)
184
185     # ONLINE LEARNING TEST

```



```

176     n_epochs_update = current_n_epochs_update
177     error_threshold = current_threshold
178     rmse_norm = 1
179     online_yhat_u = list()
180     times = list()
181     optimizer = tf.keras.optimizers.Adam()
182     optimizer.build(myLSTM.trainable_variables)
183
184     # Inicializar el modelo con un primer batch para que los
185     # estados h(t) y c(t) no esten vacios.
186     online_X = trainX[-batch_size:,:,:]
187     online_Y = trainY[-batch_size:,:,:]
188
189     for i in range(testY.shape[0]-batch_size):
190         if rmse_norm > error_threshold:
191             #print(f"RMSE NORM for sample {i}: {rmse_norm}")
192             #Temporizador para online learning
193             st = tt.time()
194             # Conversion a tf tensor para el Tracing Graph
195             norm_sq_tensor = tf.cast(rmse_norm**2, tf.float32)
196
197             for j in range(n_epochs_update):
198                 # Llamada al Tracing Graph ultrarrapido para
199                 # online learning
200                 online_train_step(myLSTM, optimizer, online_X,
201                                   online_Y, norm_sq_tensor)
202
203                 # Resetear estados
204                 for layer in myLSTM.layers:
205                     if hasattr(layer, 'reset_states'):
206                         layer.reset_states()
207
208                 #Parar temporizador
209                 et = tt.time()
210                 elapsed = et-st
211                 times.append(elapsed)
212
213             # Siguiete batch en orden cronologico
214             newX, newY = testX[i:i+batch_size,:,:], testY[i:i+
215                                   batch_size,:]
216
217             # Predicciones con el batch siguiente
218             online_yhat = myLSTM.predict(newX, batch_size=
219                                           batch_size)
220             online_yhat = scaler.inverse_transform(online_yhat)
221             newY_u = scaler.inverse_transform(newY)
222
223             # NRMSE de la prediccion
224             rmse_norm = ((np.sqrt(np.mean(((newY_u[-1,:] -
225                                           online_yhat[-1,:])**2))))

```

```

222         online_yhat_u.append(online_yhat[-1])
223
224     # Mover la ventana del batch
225     online_X = newX #Meto solo un batch en el nuevo
        entrenamiento (no lo meneo mucho)
226     online_Y = newY
227     #Empezar de nuevo con el nuevo batch (procesar
        n_epoch_updates veces, coger el batch siguiente,
        predecir sobre ese batch)
228
229     test_Yhat_ou=np.array(online_yhat_u)
230     times=np.array(times)
231
232     #Calcular NRMSE para todas las celdas
233     trainScore=np.empty(num_features)
234     testScore=np.empty(num_features)
235     testScore_inc=np.empty(num_features)
236
237     for i in range(num_features):
238         trainScore[i] = np.sqrt(mean_squared_error(train_Y_u
           [:,i], train_Yhat_su[:,i]))/np.std(train_Y_u[:,i])
239         testScore[i] = np.sqrt(mean_squared_error(test_Y_u[:,i]
            , test_Yhat_su[:,i]))/np.std(test_Y_u[:,i])
240         testScore_inc[i] = np.sqrt(mean_squared_error(test_Y_u
            [batch_size-1:-1,i], test_Yhat_ou[:,i]))/np.std(
            test_Y_u[batch_size-1:-1,i])
241
242     timeScore = np.mean(times)
243     avg_train_score = np.mean(trainScore)
244     avg_test_score = np.mean(testScore)
245     avg_test_score_inc = np.mean(testScore_inc)
246
247     grid_search_summary = {
248         'lookback': current_look_back,
249         'batchsize': current_batch_size,
250         'updates': current_n_epochs_update,
251         'threshold': current_threshold,
252         'AvgTrain_RNMSE': avg_train_score,
253         'AvgTest_RNMSE': avg_test_score,
254         'AvgTestInc_RNMSE': avg_test_score_inc,
255         'AvgTestIncTime': timeScore
256     }
257
258     combi_df = pd.DataFrame([grid_search_summary])
259
260     #Crear csv con los resultados
261     file_exists = os.path.isfile(output_csv_file)
262     combi_df.to_csv(output_csv_file, mode='a', header=not
        file_exists, index=False)

```

A.3. Código Python de los modelos XCNN+LSTM adaptados para el entorno Kaggle

Por simplicidad y gracias a la gran similitud existente entre los *scripts* de las variantes de los modelos CNN+LSTM, 2CNN+LSTM y 3CNN+LSTM, se presenta únicamente el código para la arquitectura 3CNN+LSTM con kernel 3x3 y cuadrícula 7x7. También se han realizado las modificaciones análogas a las mencionadas en el apéndice A.2.

A partir de este código, para obtener el modelo 2CNN+LSTM se ha de comentar las líneas: 86, 120, 161 y 270. Para obtener el modelo CNN+LSTM se ha de comentar, además de las comentadas para obtener el modelo 2CNN+LSTM, las líneas: 85, 119, 160 y 269. Para obtener las versiones con kernel 2x2 simplemente se ha de cambiar el parámetro `kernel_size=(3,3)` por `kernel_size=(2,2)` en las líneas 159, 160 y 161.

Así mismo, para poder procesar una cuadrícula 15x15 se ha de sustituir todas las ocurrencias del número 7 por el número 15 y cambiar el *path* indicado en la línea 72 al *path* del fichero que contenga el *dataset* para la cuadrícula 15x15.

```
1 import os
2 import random
3 import numpy as np
4 import pandas as pd
5 import tensorflow as tf
6 import itertools
7 import time as tt
8 from tensorflow.keras.models import Sequential
9 from tensorflow.keras.layers import Dense, LSTM, Input,
    TimeDistributed, Conv2D, MaxPooling2D, Flatten
10 from sklearn.preprocessing import MinMaxScaler
11 from sklearn.metrics import mean_squared_error
12 from tensorflow.python.client import device_lib
13
14 import os
15 for dirname, _, filenames in os.walk('/kaggle/input'):
16     for filename in filenames:
17         print(os.path.join(dirname, filename))
18
19 #NOMENCLATURA
20 # _u --> unscaled (scaler.inverse_transform)
21 # _s --> static
22 # _o --> online learning
23 # nada --> scaled (scaler.transform)
24
25 # GET CPU AND GPU
26 def get_available_devices():
27     local_device_protos = device_lib.list_local_devices()
28     return [x.name for x in local_device_protos]
29
30 # FUNCION RMSE
```

```

31 def rmse_loss(y_true, y_pred):
32     return tf.sqrt(tf.reduce_mean(tf.square(y_true - y_pred)))
33
34 # ARRAY TO MATRIX
35 def create_dataset(dataset, look_back=1):
36     dataX, dataY = [], []
37     for i in range(len(dataset) - look_back - 1):
38         a = dataset[i:(i + look_back), :]
39         dataX.append(a)
40         dataY.append(dataset[i + look_back, :])
41     return np.array(dataX), np.array(dataY)
42
43 # Callback para resetear los estados de forma ultrarrapida
44 class ResetStatesCallback(tf.keras.callbacks.Callback):
45     def on_epoch_end(self, epoch, logs=None):
46         for layer in self.model.layers:
47             if hasattr(layer, 'reset_states'):
48                 layer.reset_states()
49
50
51 # tf.function para invocar Tracing Graphs
52 @tf.function
53 def online_train_step(model, optimizer, x, y, norm_sq):
54     with tf.GradientTape() as tape:
55         predictions = model(x, training=True)
56         loss = rmse_loss(y, predictions)
57
58         gradients = tape.gradient(loss, model.trainable_variables)
59         adjusted_gradients = [g * norm_sq for g in gradients]
60         optimizer.apply_gradients(zip(adjusted_gradients, model.trainable_variables))
61
62 #Restricciones de reproducibilidad
63 os.environ['PYTHONHASHSEED'] = '0'
64 os.environ['TF_DETERMINISTIC_OPS'] = '1'
65 os.environ['TF_CUDNN_DETERMINISTIC'] = '1'
66 random.seed(7)
67 np.random.seed(7)
68 tf.random.set_seed(7)
69 tf.config.experimental.enable_op_determinism()
70
71 #Lectura del dataset
72 dataframe = pd.read_csv('/kaggle/input/datasets/username/
    traffic-7x7-1/Traffic_7x7_1.csv', usecols=range(1,50),
    engine='python')
73 dataset_full = dataframe.values
74 dataset_full = dataset_full.astype('float32')
75 num_features = dataset_full.shape[1] #7x7=49
76 total_rows = dataset_full.shape[0]
77
78 # Parameter tweak for Grid Search
79 hyperparameters = {

```

```

80     'look_back': [18, 24],
81     'batch_size': [23, 25],
82     'n_epochs_update': [5, 6],
83     'threshold': [0, 0.1],
84     'num_filters1': [12, 16],
85     'num_filters2': [22, 24],
86     'num_filters3': [28],
87 }
88
89 all_combinations = set(itertools.product(*hyperparameters.
    values()))
90
91 combinations_to_run = list(all_combinations)
92
93 combinations_to_run.sort()
94
95 #combinations = list(itertools.product(*hyperparameters.values
    ()))
96 output_csv_file = '/kaggle/working/GridSearch_XCNNLSTM.csv'
97
98 #BUCLE GRID SEARCH
99 for combi in combinations:
100     #Borrar todo rastro de modelos de otras iteraciones para
        evitar influencias no autorizadas
101     tf.keras.backend.clear_session()
102
103     #Restricciones de reproducibilidad de nuevo, por si acaso
104     os.environ['PYTHONHASHSEED'] = '0'
105     os.environ['TF_DETERMINISTIC_OPS'] = '1'
106     os.environ['TF_CUDNN_DETERMINISTIC'] = '1'
107     random.seed(7)
108     np.random.seed(7)
109     tf.random.set_seed(7)
110     tf.config.experimental.enable_op_determinism()
111
112     #Combinacion de hiperparametros actual
113     params = dict(zip(hyperparameters.keys(), combi))
114     current_look_back = params['look_back']
115     current_batch_size = params['batch_size']
116     current_n_epochs_update = params['n_epochs_update']
117     current_threshold = params['threshold']
118     current_filters1 = params['num_filters1']
119     current_filters2 = params['num_filters2']
120     current_filters3 = params['num_filters3']
121
122     # Separacion entre train y test de forma automatica segun
        batch_size,
123     # el numero total de muestras de los datasets ha de ser
        divisible por batch_size
124     desired_train_size = 400
125     train_available_batches = (desired_train_size -
        current_look_back - 1) // current_batch_size

```

```

126     train_size = current_batch_size * train_available_batches
127         + current_look_back + 1
128     #create_dataset se come las primeras look_back + 1
129     muestras para crear el dataset de test/train
130     available_batches = (total_rows - current_look_back -1 -
131         train_size) // current_batch_size
132     test_size = current_batch_size * available_batches +
133         current_look_back +1
134
135     dataset = dataset_full[0:train_size+test_size]
136
137     scaler = MinMaxScaler(feature_range=(0, 1))
138     train = dataset[0:train_size, :]
139     test = dataset[train_size : train_size+test_size, :]
140
141     #EL MODELO ORIGINAL HACE FIT_TRANSFORM CON TODO EL DATASET
142     --> DATA LEAKAGE
143     train = scaler.fit_transform(train)
144     test = scaler.transform(test)
145
146     # X=[T,T+LOOKBACK]   Y=[T+LOOKBACK+1]
147     # look_back define cuantas muestras se utiliza para las
148     series temporales
149     look_back = current_look_back #este parametro, permite
150     definir una ventana deslizante
151     trainX, trainY = create_dataset(train, look_back)
152     testX, testY = create_dataset(test, look_back)
153
154     # Transformacion a 5D para las capas TimeDistributed(
155     Conv2D())
156     trainX = trainX.reshape((trainX.shape[0], look_back, 7, 7,
157         1))
158     testX = testX.reshape((testX.shape[0], look_back, 7, 7, 1)
159         )
160
161     #batch_size indica cuantas ventanas look_back se procesan
162     antes de actualizar los pesos
163     batch_size = current_batch_size
164     # Cuantas vueltas al dataset entero se dan para entrenar
165     n_epochs = 400
166
167     #MODELO 3CNN+LSTM CON CAPA INPUT Y FLATTEN
168     myCNNLSTM = Sequential()
169     myCNNLSTM.add(Input(batch_shape=(batch_size, look_back, 7,
170         7, 1)))
171     myCNNLSTM.add(TimeDistributed(Conv2D(filters=
172         current_filters1, kernel_size=(3,3), padding='same',
173         activation='relu'))))
174     myCNNLSTM.add(TimeDistributed(Conv2D(filters=
175         current_filters2, kernel_size=(3,3), padding='same',
176         activation='relu'))))
177     myCNNLSTM.add(TimeDistributed(Conv2D(filters=

```

```

        current_filters3, kernel_size=(3,3), padding='same',
        activation='relu'))))
162 myCNNLSTM.add(TimeDistributed(Flatten())) # Flatten PARA
        CAMBIAR DE 5d A 3D
163 myCNNLSTM.add(LSTM(look_back, stateful=True))
164 myCNNLSTM.add(Dense(num_features))
165 myCNNLSTM.compile(loss='mean_squared_error', optimizer='
        adam')

166
167     #Entrenamiento optimizado ultrarrapido
168 #El reseteo del estado debe hacerse asi cada epoch.
        Hacerlo es recomendable, si bien para datasets pequenos
        no es relevante
169 print('Fitting myLSTM on verbose 0')
170 myLSTM.fit(
171     trainX, trainY,
172     epochs=n_epochs,
173     batch_size=batch_size,
174     verbose=0,
175     shuffle=True,
176     callbacks=[ResetStatesCallback()]
177 )
178 #El myLSTMos LSTM es stateful, es decir, que guarda sus
        estados h(t) y c(t) y los usa como valor inicial en el
        siguiente batch
179 #Como shuffle=True, el orden de los batches es aleatorio.
        Lo que significa que actua como un stateless.
180 #Shuffle=True mejora la generalizacion.
181
182 #TEST ESTATICO
183 train_Y_u = scaler.inverse_transform(trainY)
184 test_Y_u = scaler.inverse_transform(testY)
185
186 train_Yhat_s = myLSTM.predict(trainX, batch_size=
        batch_size)
187 train_Yhat_su = scaler.inverse_transform(train_Yhat_s)
188
189 test_Yhat_s = myLSTM.predict(testX, batch_size=batch_size)
190 test_Yhat_su = scaler.inverse_transform(test_Yhat_s)
191
192 # ONLINE LEARNING TEST
193 n_epochs_update = current_n_epochs_update
194 error_threshold = current_threshold
195 rmse_norm = 1
196 online_yhat_u = list()
197 times = list()
198 optimizer = tf.keras.optimizers.Adam()
199 optimizer.build(myLSTM.trainable_variables)
200
201 # Inicializar el modelo con un primer batch para que los
        estados h(t) y c(t) no esten vacios.
202 online_X = trainX[-batch_size:,:,::]

```

```

203     online_Y = trainY[-batch_size:,:]
204     for i in range(testY.shape[0]-batch_size):
205         if rmse_norm > error_threshold:
206             #print(f"RMSE NORM for sample {i}: {rmse_norm}")
207             #Temporizador para online learning
208             st = tt.time()
209             # Conversion a tf tensor para el Tracing Graph
210             norm_sq_tensor = tf.cast(rmse_norm**2, tf.float32)
211
212             for j in range(n_epochs_update):
213                 # Llamada al Tracing Graph ultrarrapido para
214                 # online learning
215                 online_train_step(myLSTM, optimizer, online_X,
216                                   online_Y, norm_sq_tensor)
217
218                 # Resetear estados
219                 for layer in myLSTM.layers:
220                     if hasattr(layer, 'reset_states'):
221                         layer.reset_states()
222
223                 #Parar temporizador
224                 et = tt.time()
225                 elapsed = et-st
226                 times.append(elapsed)
227
228             # Siguiente batch en orden cronologico
229             newX, newY = testX[i:i+batch_size,:,:), testY[i:i+
230                 batch_size,:])
231
232             # Predicciones con el batch siguiente
233             online_yhat = myLSTM.predict(newX, batch_size=
234                 batch_size)
235             online_yhat = scaler.inverse_transform(online_yhat)
236             newY_u = scaler.inverse_transform(newY)
237
238             # NRMSE de la prediccion
239             rmse_norm = ((np.sqrt(np.mean(((newY_u[-1,:] -
240                 online_yhat[-1,:])/online_yhat[-1,:])**2))))
241
242             online_yhat_u.append(online_yhat[-1])
243
244             # Mover la ventana del batch
245             online_X = newX #Meto solo un batch en el nuevo
246             #entrenamiento (no lo meneo mucho)
247             online_Y = newY
248             #Empezar de nuevo con el nuevo batch (procesar
249             #n_epoch_updates veces, coger el batch siguiente,
250             #predecir sobre ese batch)
251
252     test_Yhat_ou=np.array(online_yhat_u)
253     times=np.array(times)

```



```

247
248 #Calcular NRMSE para todas las celdas
249 trainScore=np.empty(num_features)
250 testScore=np.empty(num_features)
251 testScore_inc=np.empty(num_features)
252
253 for i in range(num_features):
254     trainScore[i] = np.sqrt(mean_squared_error(train_Y_u
255        [:,i], train_Yhat_su[:,i]))/np.std(train_Y_u[:,i])
256     testScore[i] = np.sqrt(mean_squared_error(test_Y_u[:,i],
257         test_Yhat_su[:,i]))/np.std(test_Y_u[:,i])
258     testScore_inc[i] = np.sqrt(mean_squared_error(test_Y_u
259         [batch_size-1:-1,i], test_Yhat_ou[:,i]))/np.std(
260         test_Y_u[batch_size-1:-1,i])
261
262 timeScore = np.mean(times)
263 avg_train_score = np.mean(trainScore)
264 avg_test_score = np.mean(testScore)
265 avg_test_score_inc = np.mean(testScore_inc)
266
267 grid_search_summary = {
268     'lookback': current_look_back,
269     'batchsize': current_batch_size,
270     'updates': current_n_epochs_update,
271     'threshold': current_threshold,
272     'numfilters1': current_filters1,
273     'numfilters2': current_filters2,
274     'numfilters3': current_filters3,
275     'AvgTrainRNMSE': avg_train_score,
276     'AvgTestRNMSE': avg_test_score,
277     'AvgTestIncRNMSE': avg_test_score_inc,
278     'AvgTestIncTime': timeScore
279 }
280
281 combi_df = pd.DataFrame([grid_search_summary])
282 file_exists = os.path.isfile(output_csv_file)
283 combi_df.to_csv(output_csv_file, mode='a', header=not
284     file_exists, index=False)

```

B. Representación de los datos

En este apéndice se muestran los datos utilizados y obtenidos en este trabajo. Por simplicidad, sólo se muestran aquellos datos relevantes que hayan sido dignos de mención en los Capítulos 3 y 4.

B.1. *Dataset* Milán

En este apartado se muestran los datos contenidos en el dataset Milán para todas aquellas celdas mencionadas en este trabajo. El orden originales en el que se muestran es en orden ascendente según su identificador dentro del *dataset* Milán.

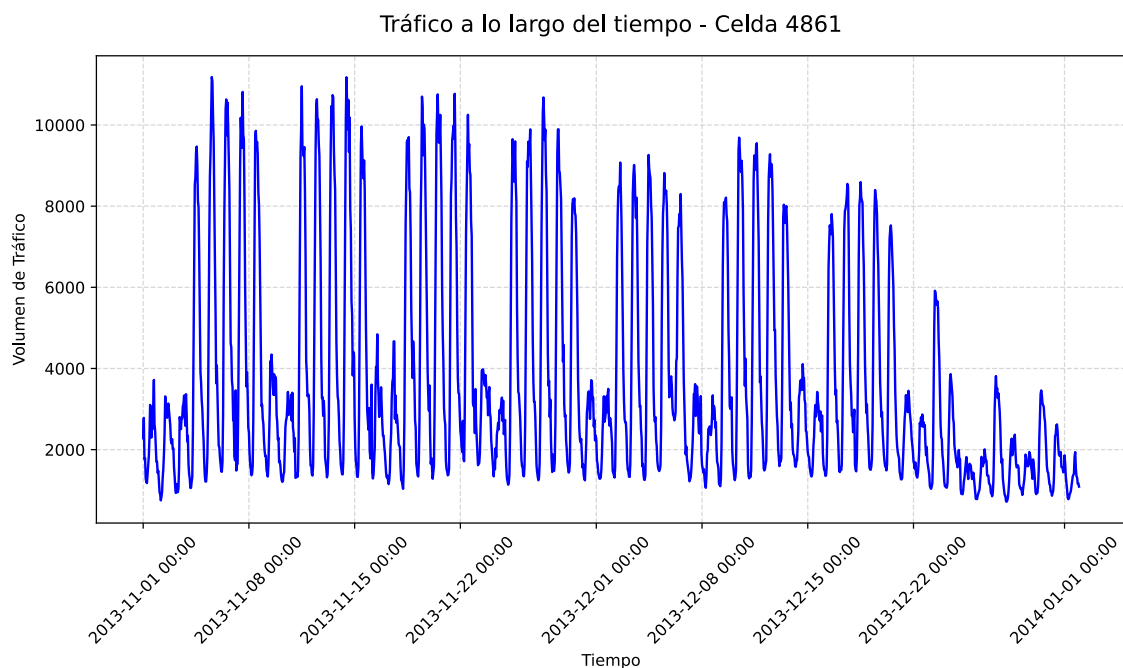


Figura 11: Datos contenidos en el *dataset* Milán para la celda 4861 [38].

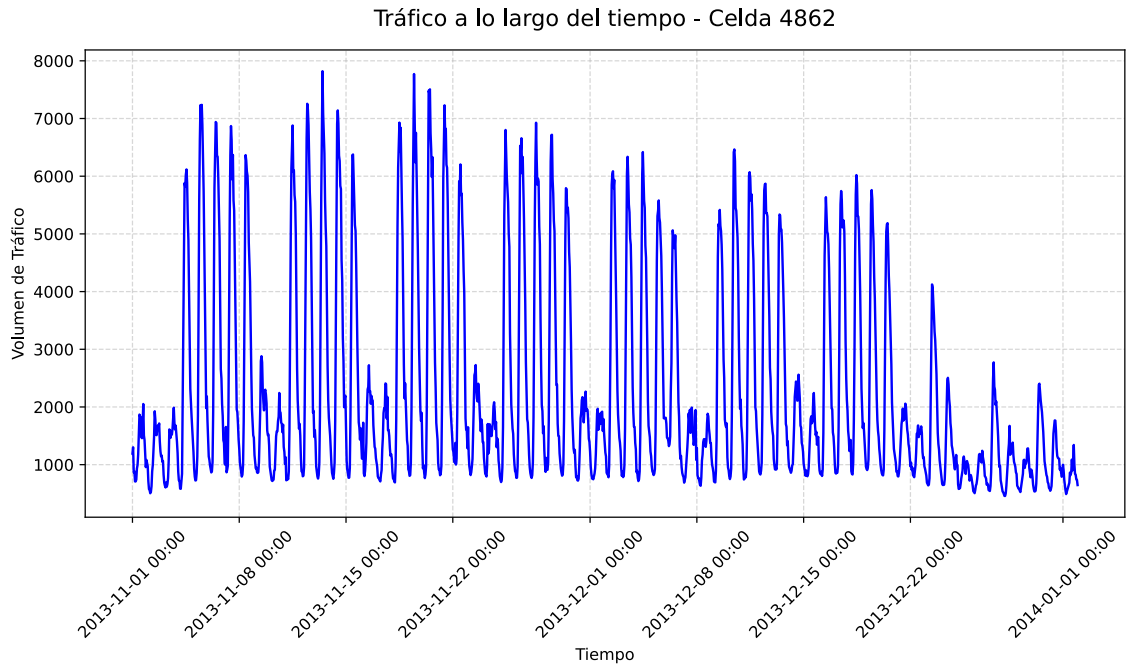


Figura 12: Datos contenidos en el *dataset* Milán para la celda 4862 [38].

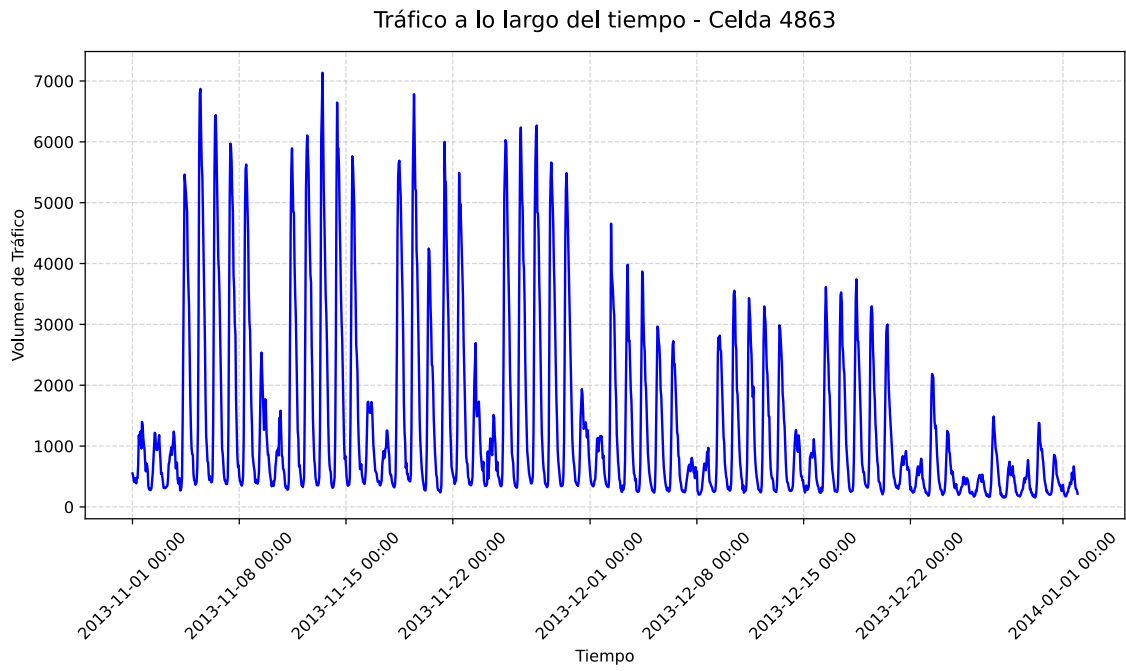


Figura 13: Datos contenidos en el *dataset* Milán para la celda 4863 [38].

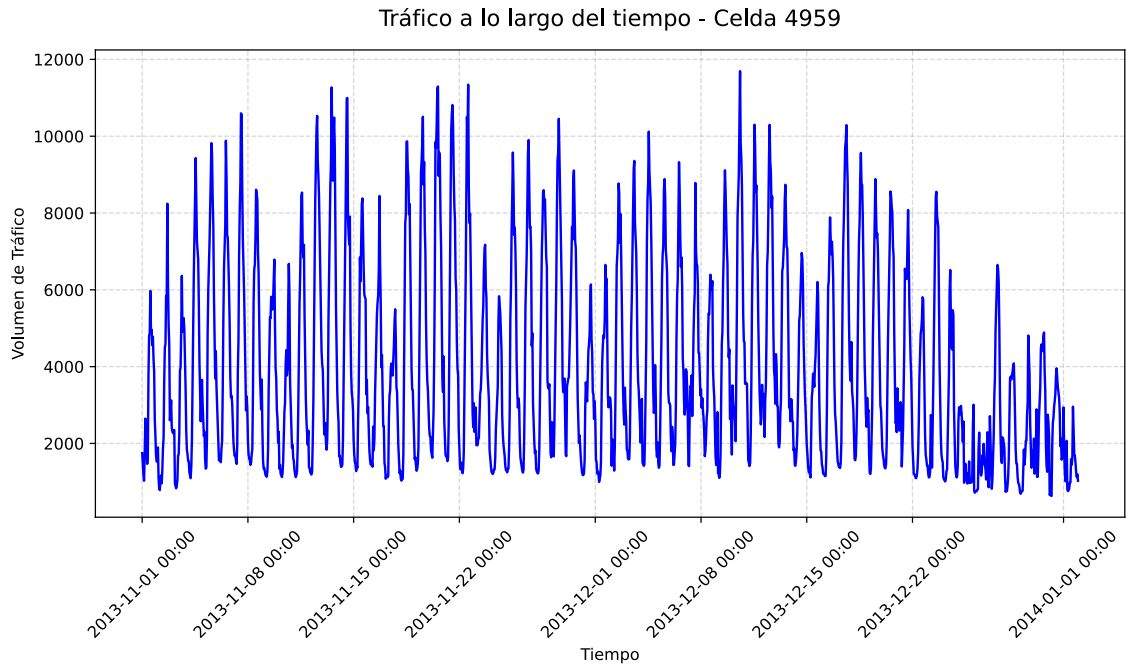


Figura 14: Datos contenidos en el *dataset* Milán para la celda 4959 [38].

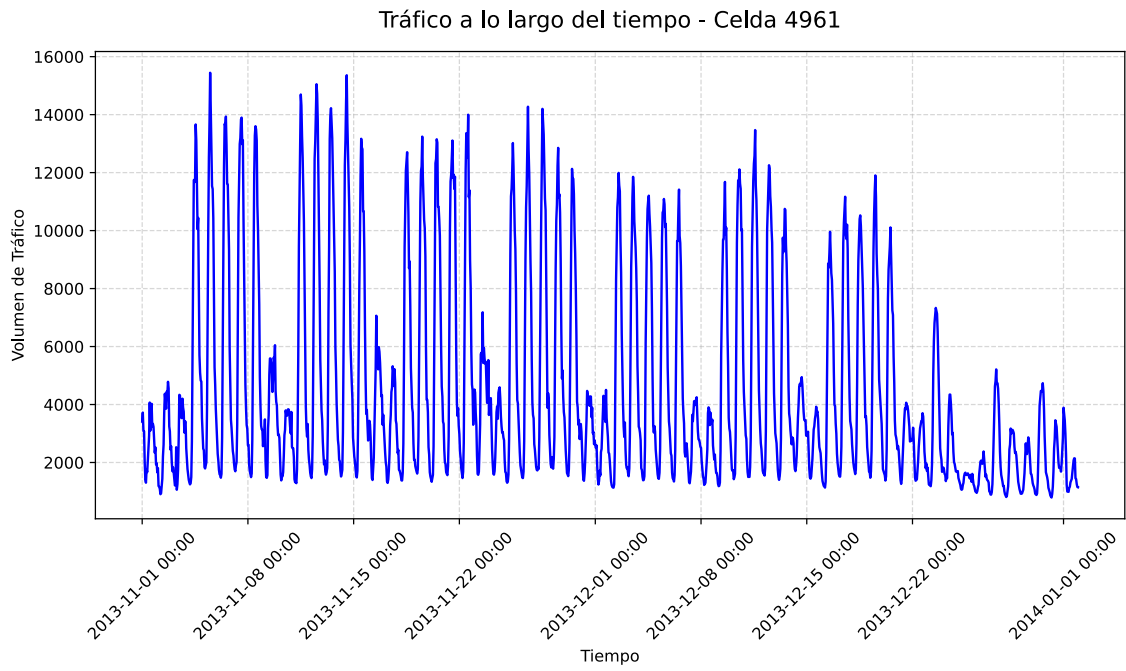


Figura 15: Datos contenidos en el *dataset* Milán para la celda 4961 [38].

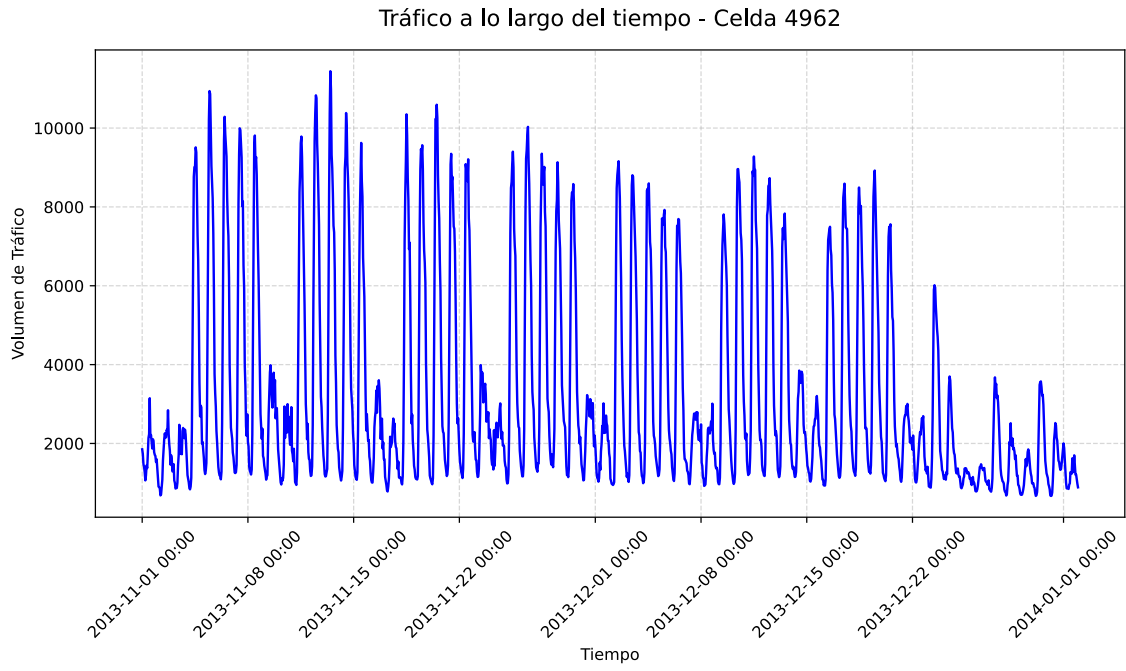


Figura 16: Datos contenidos en el *dataset* Milán para la celda 4962 [38].

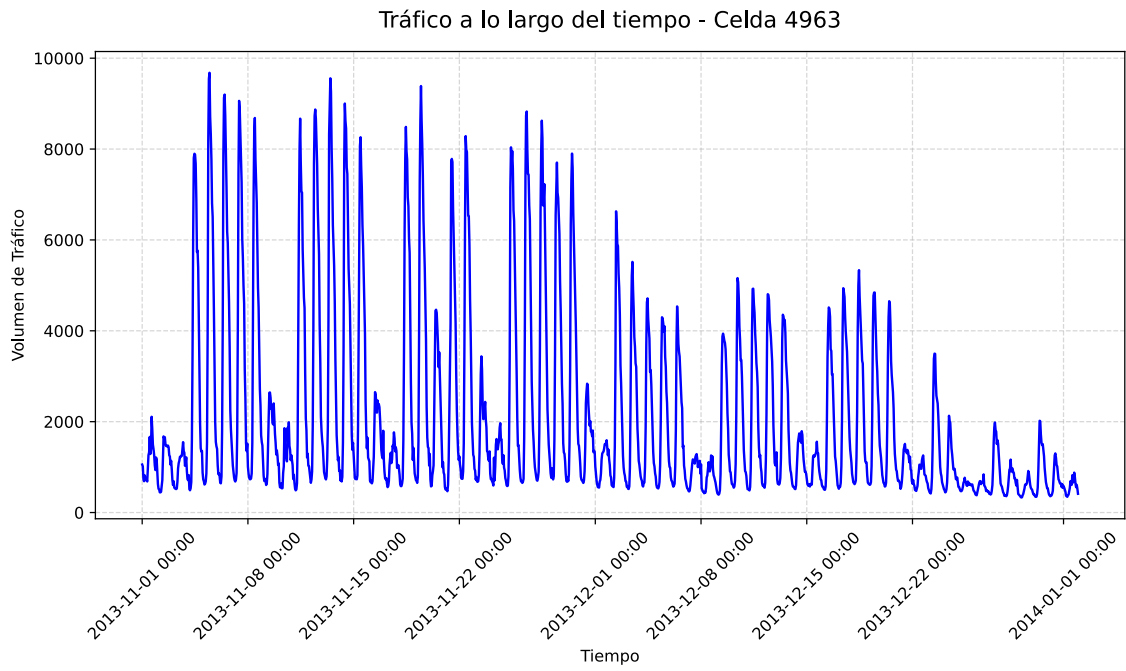


Figura 17: Datos contenidos en el *dataset* Milán para la celda 4963 [38].

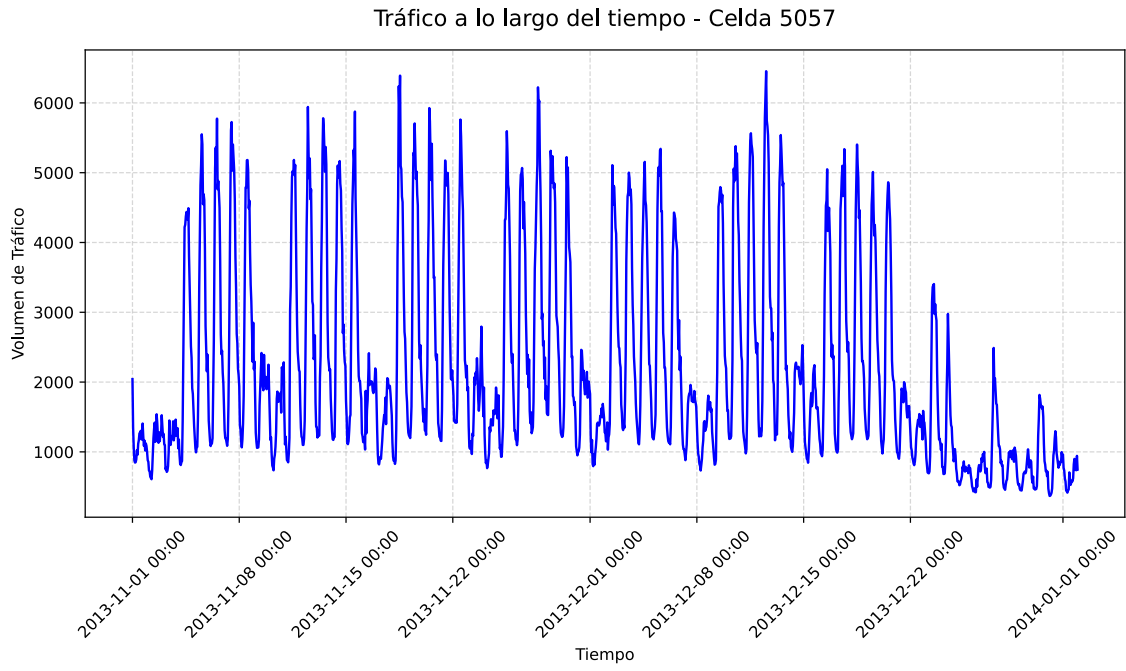


Figura 18: Datos contenidos en el *dataset* Milán para la celda 5057 [38].

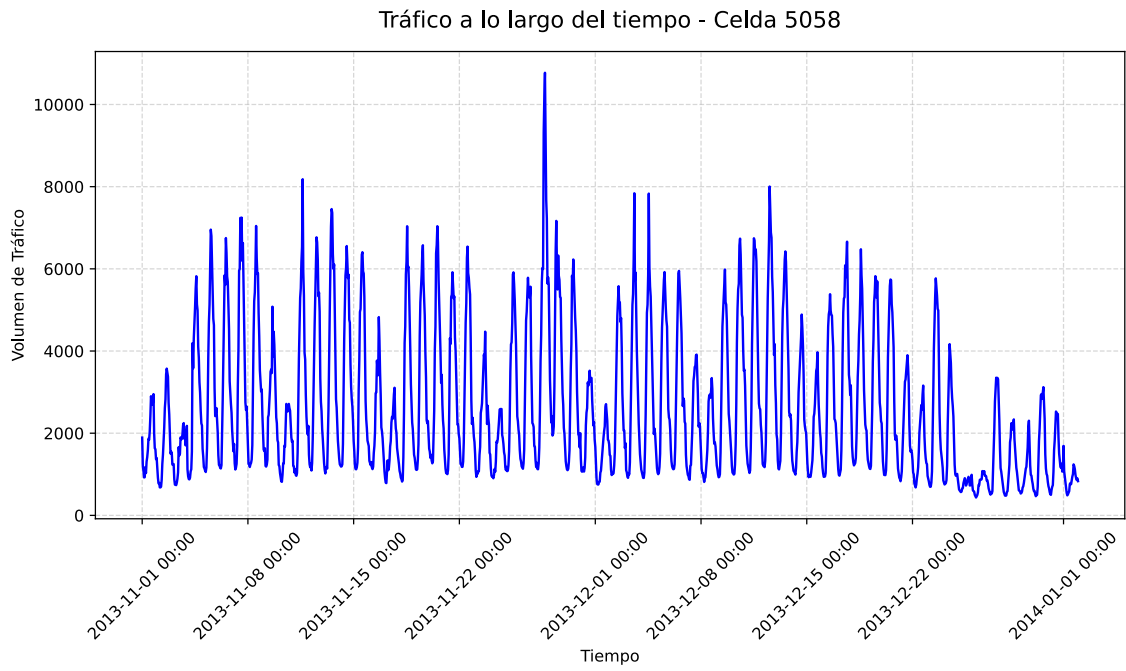


Figura 19: Datos contenidos en el *dataset* Milán para la celda 5058 [38].

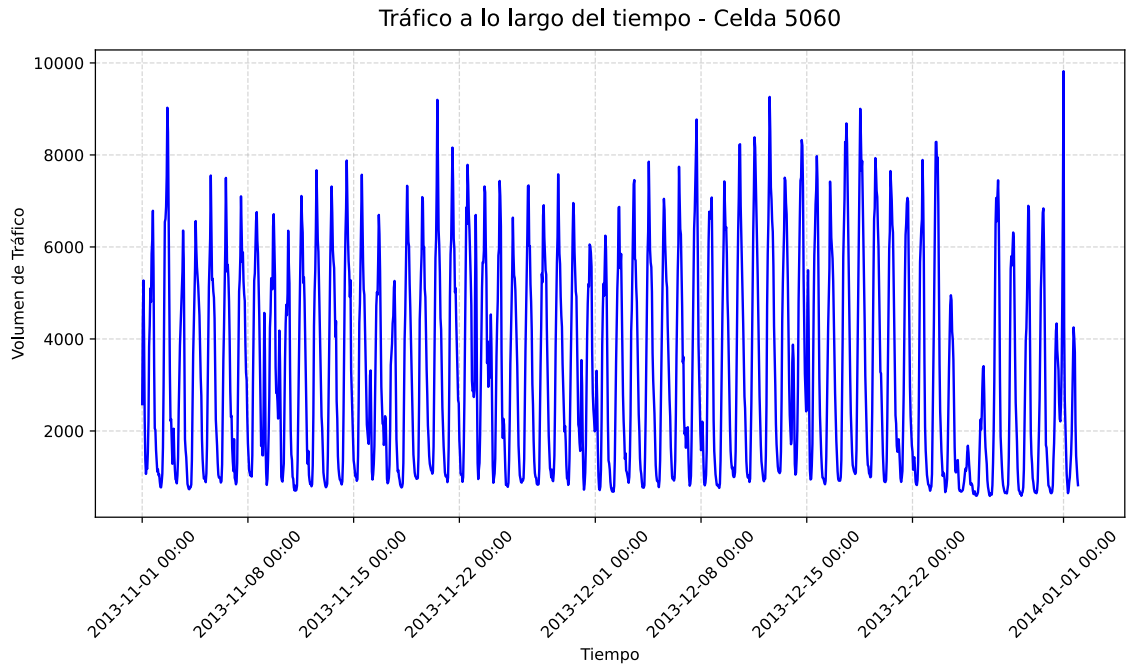


Figura 20: Datos contenidos en el *dataset* Milán para la celda 5060 [38].

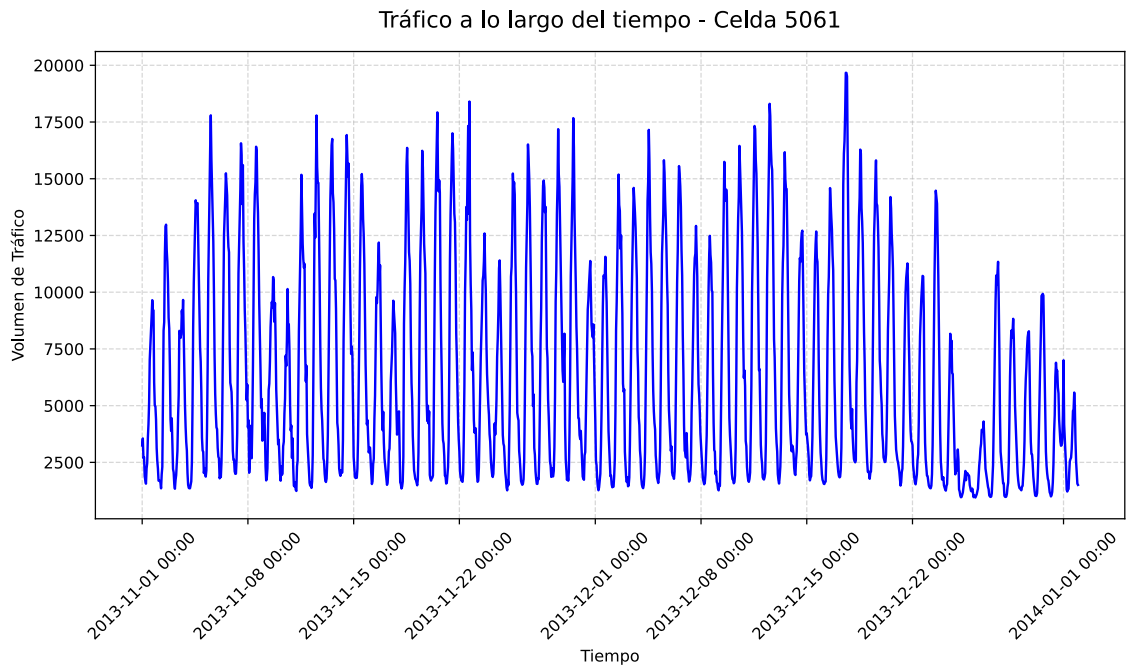


Figura 21: Datos contenidos en el *dataset* Milán para la celda 5061 [38].

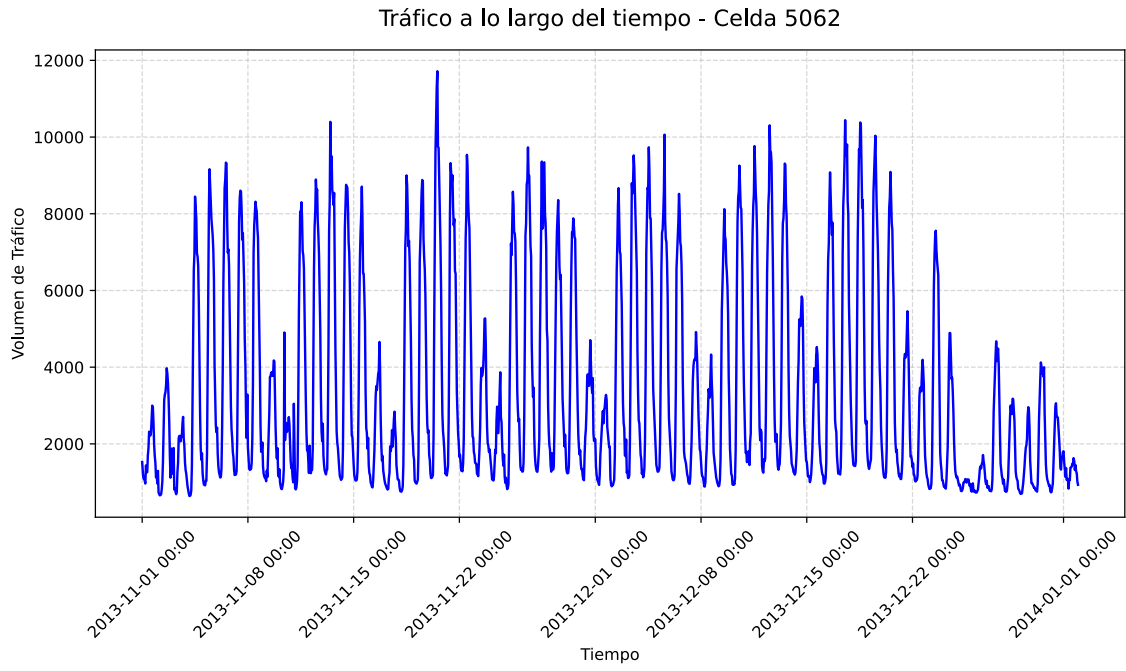


Figura 22: Datos contenidos en el *dataset* Milán para la celda 5062 [38].

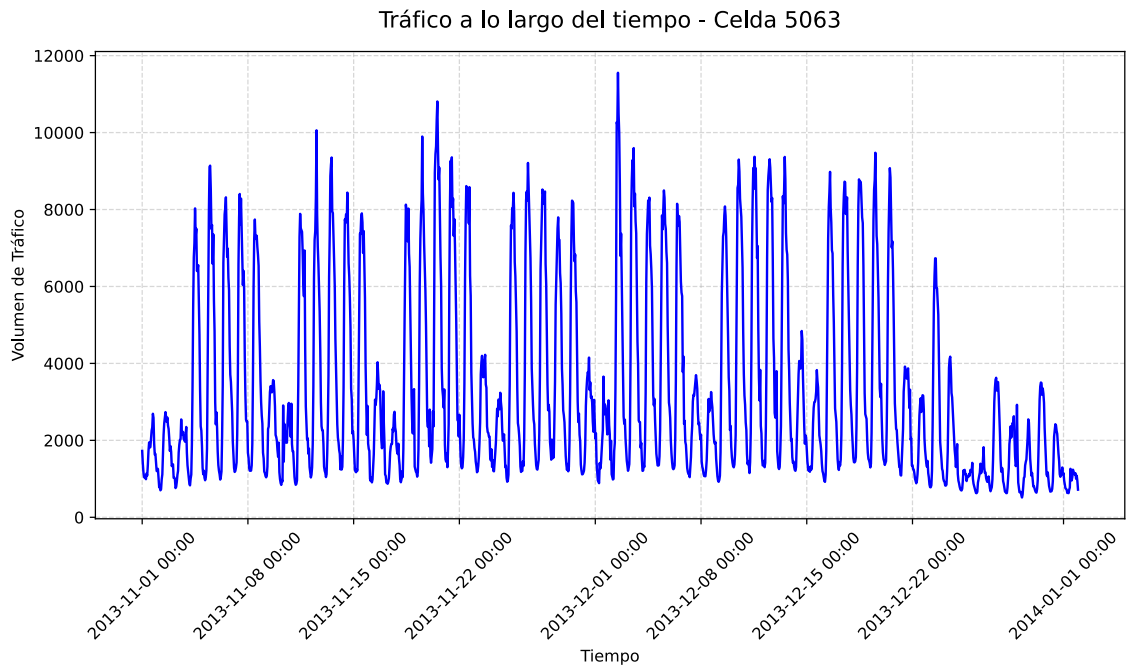


Figura 23: Datos contenidos en el *dataset* Milán para la celda 5063 [38].

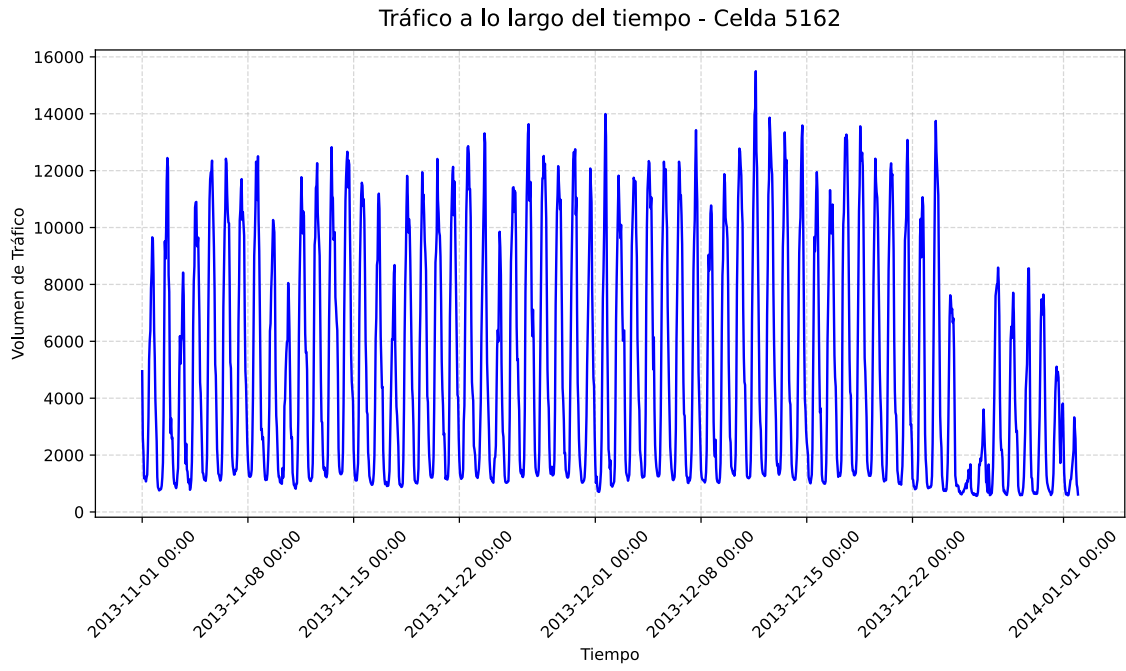


Figura 24: Datos contenidos en el *dataset* Milán para la celda 5162 [38].

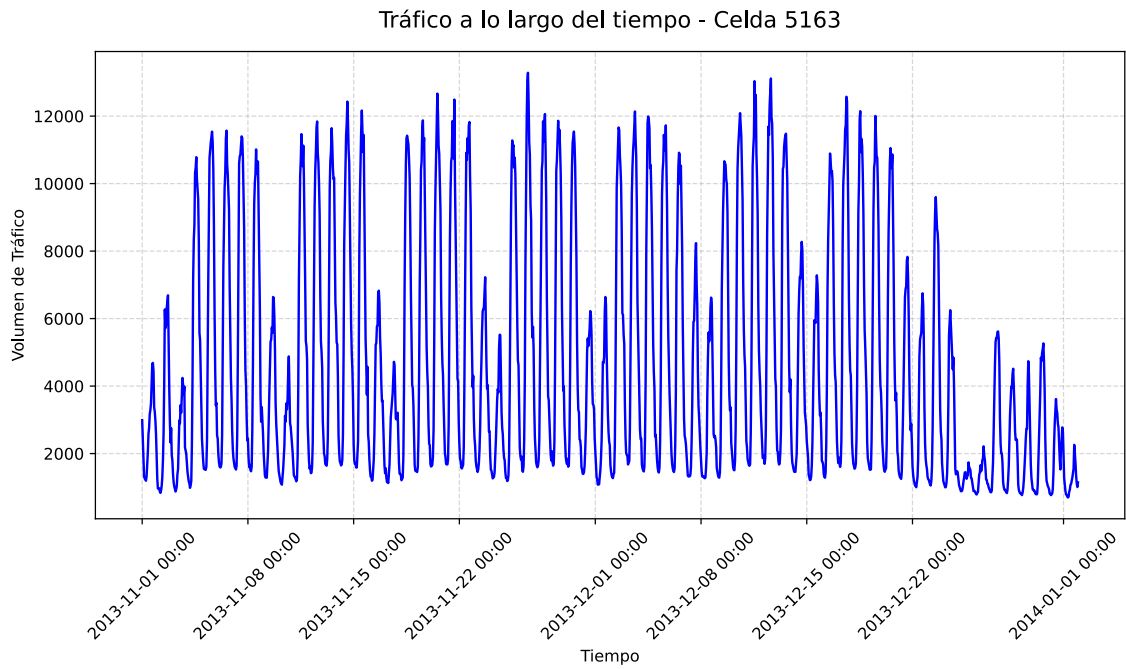


Figura 25: Datos contenidos en el *dataset* Milán para la celda 5163 [38].

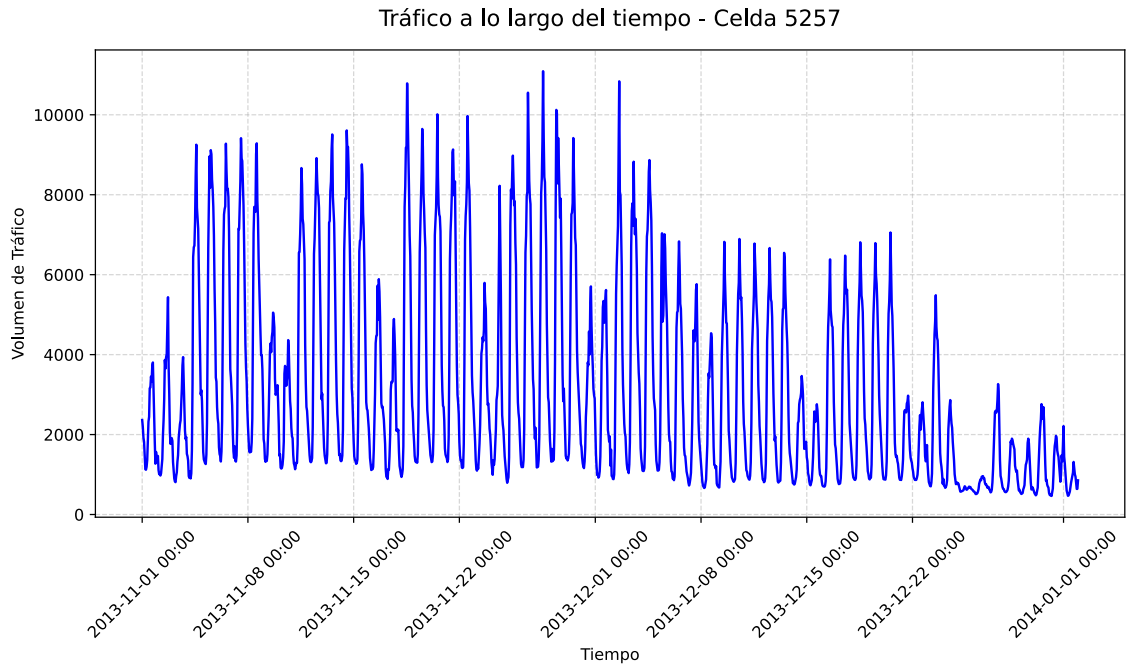


Figura 26: Datos contenidos en el *dataset* Milán para la celda 5257 [38].

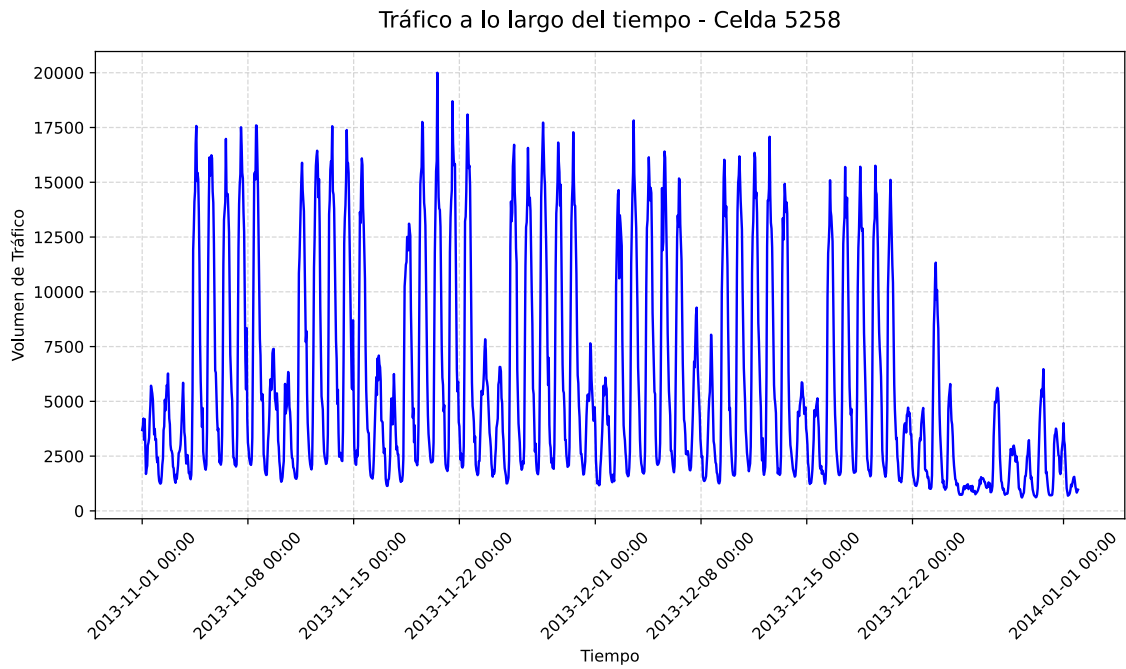


Figura 27: Datos contenidos en el *dataset* Milán para la celda 5258 [38].

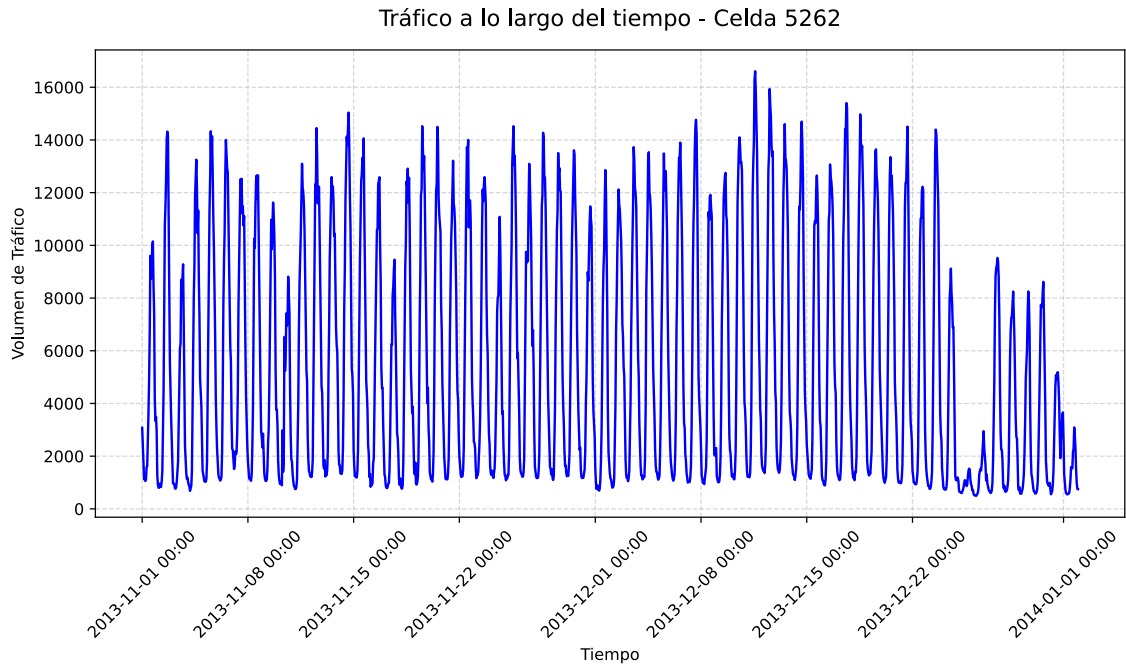


Figura 28: Datos contenidos en el *dataset* Milán para la celda 5262 [38].

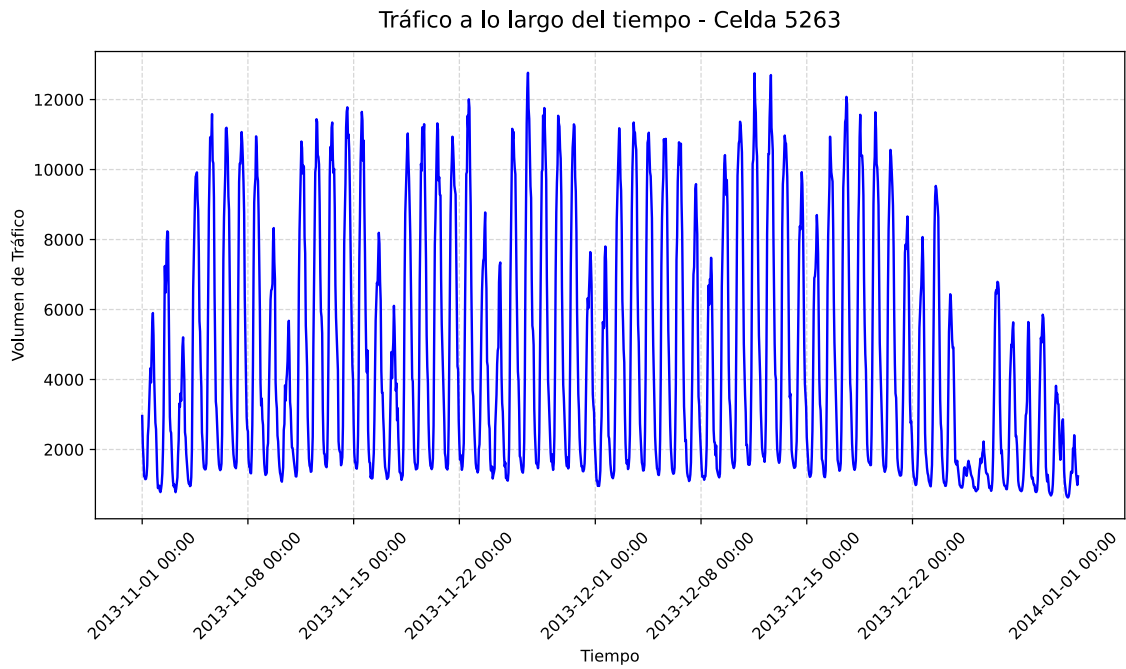


Figura 29: Datos contenidos en el *dataset* Milán para la celda 5263 [38].

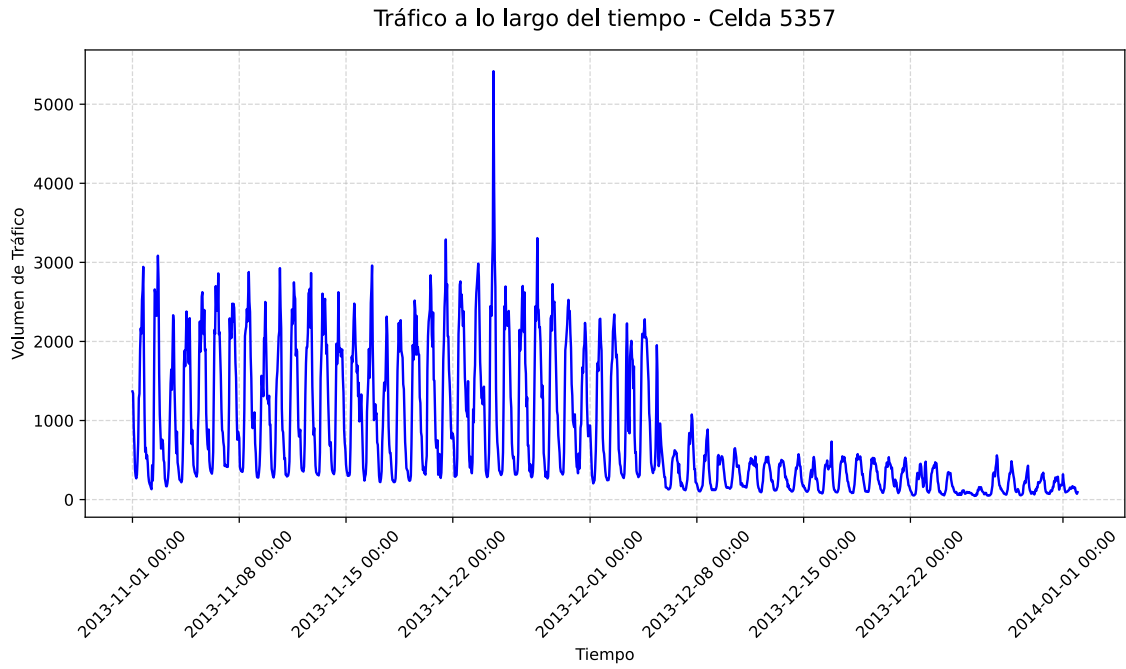


Figura 30: Datos contenidos en el *dataset* Milán para la celda 5357 [38].

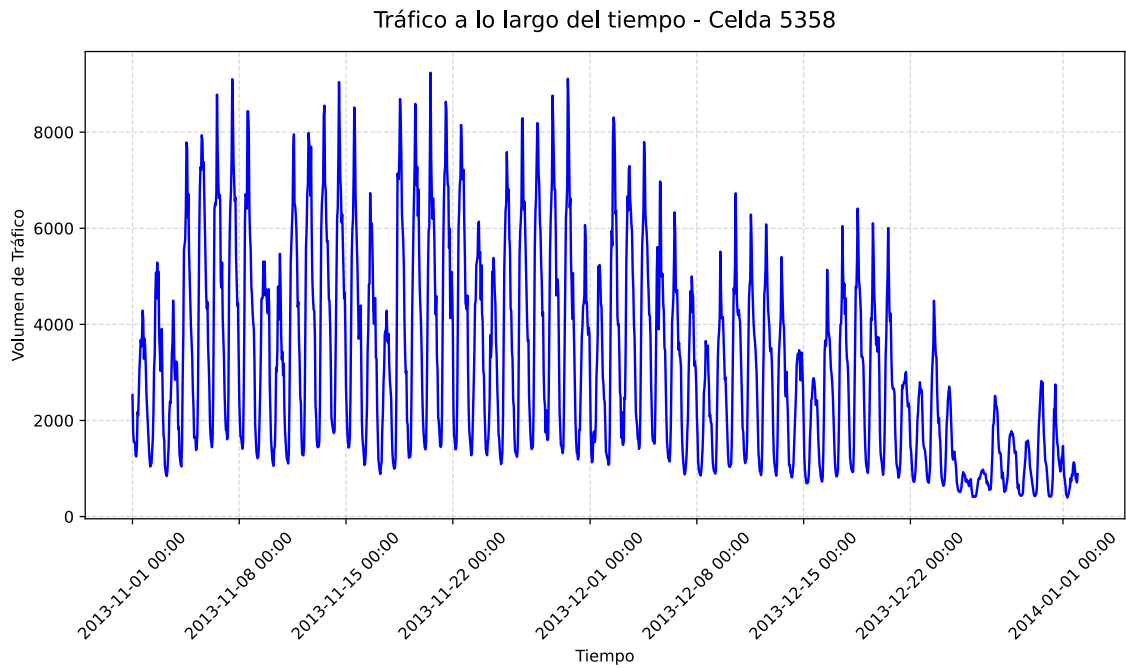


Figura 31: Datos contenidos en el *dataset* Milán para la celda 5358 [38].

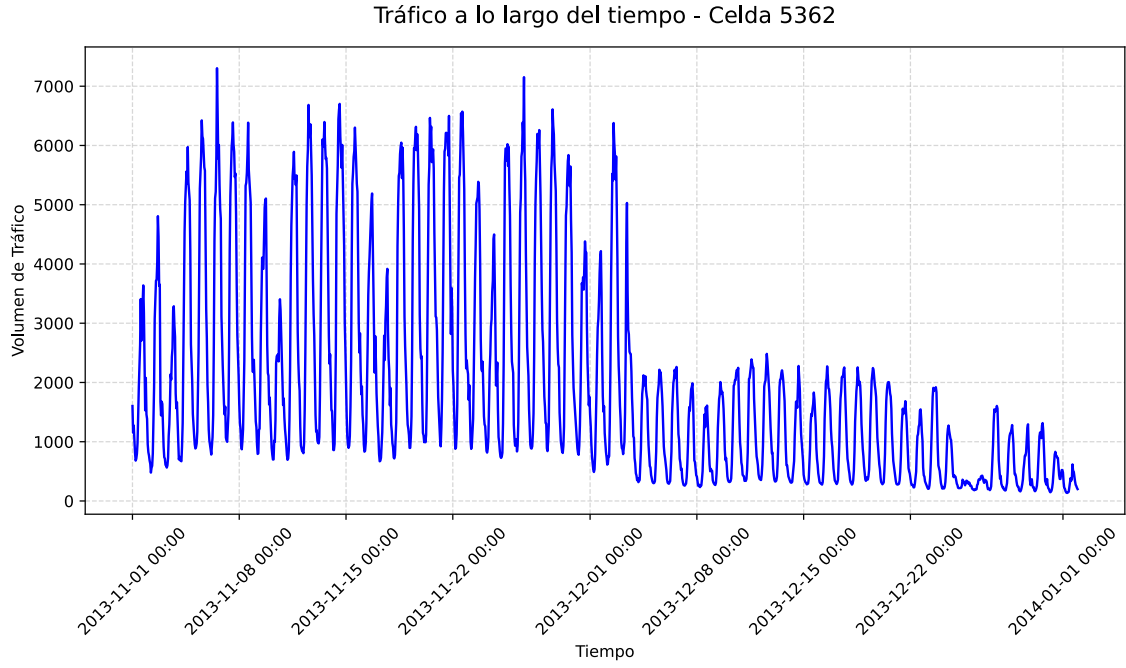


Figura 32: Datos contenidos en el *dataset* Milán para la celda 5362 [38].

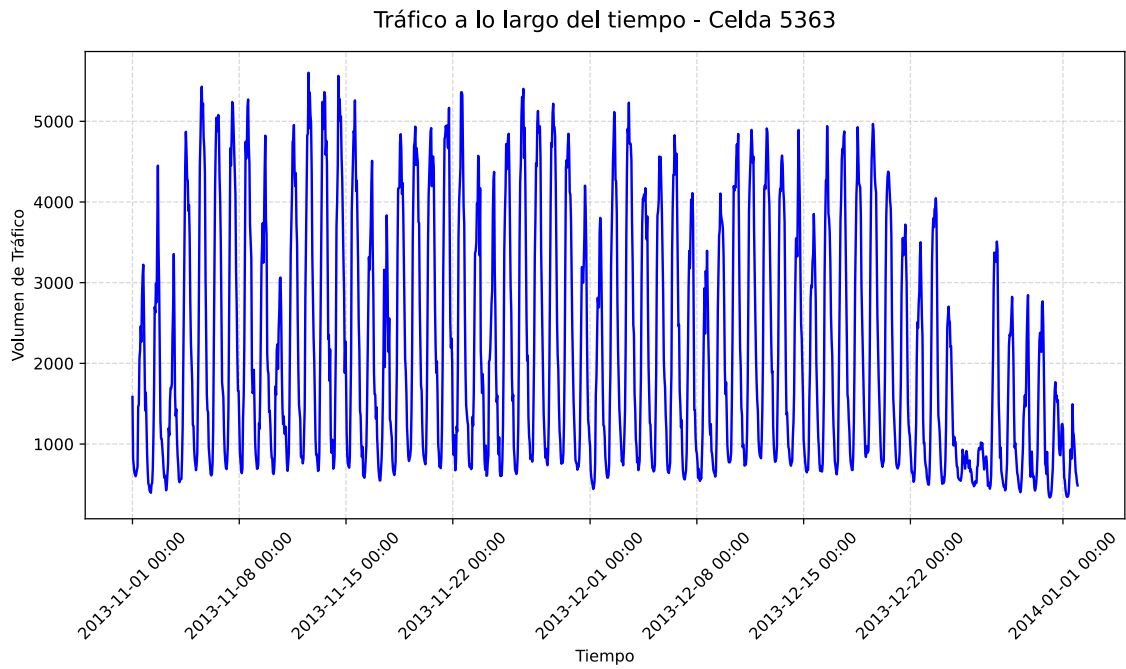


Figura 33: Datos contenidos en el *dataset* Milán para la celda 5363 [38].

B.2. Predicciones del mejor modelo

En este apartado se muestran las predicciones realizadas sobre el set de validación por el mejor modelo sobre las celdas mencionadas en el apartado 4.3 en orden ascendente según su identificador en el *dataset* Milán.

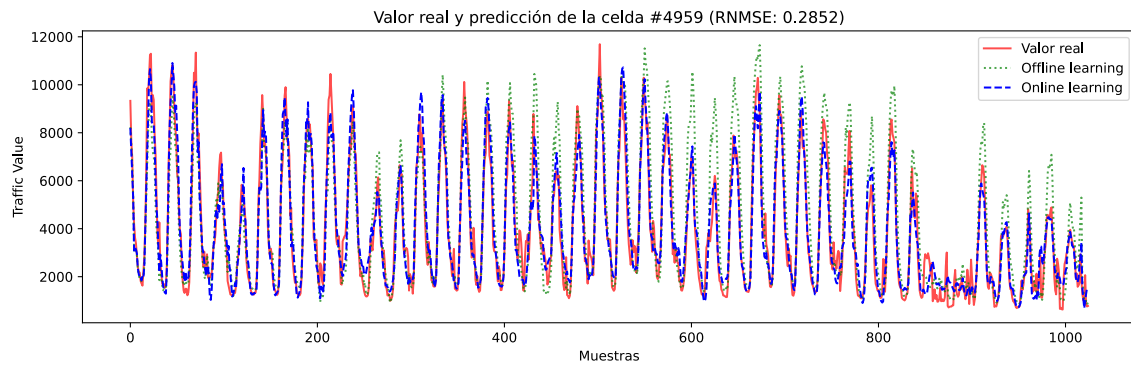


Figura 34: Predicciones del mejor modelo para la celda 4959. Elaboración propia.

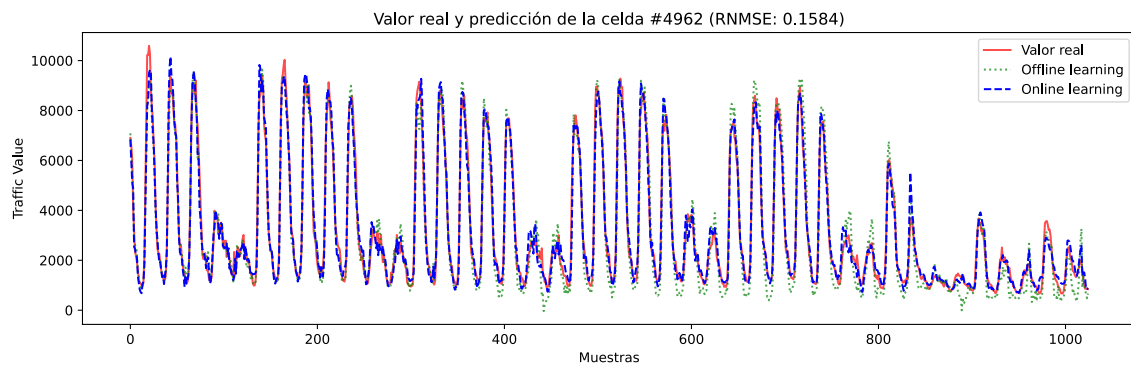


Figura 35: Predicciones del mejor modelo para la celda 4962. Elaboración propia.

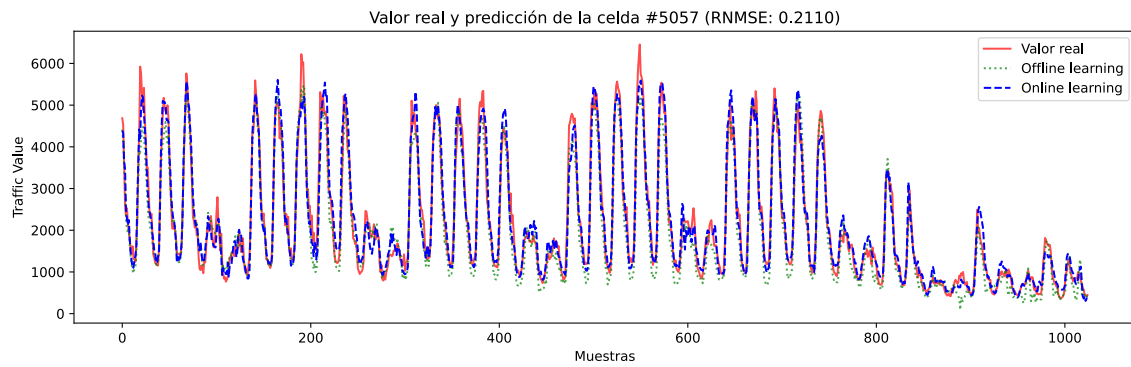


Figura 36: Predicciones del mejor modelo para la celda 5057. Elaboración propia.

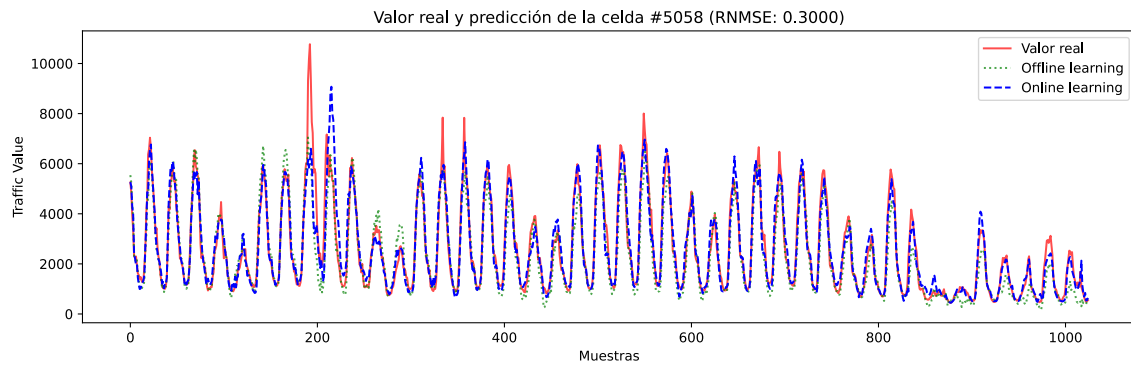


Figura 37: Predicciones del mejor modelo para la celda 5058. Elaboración propia.

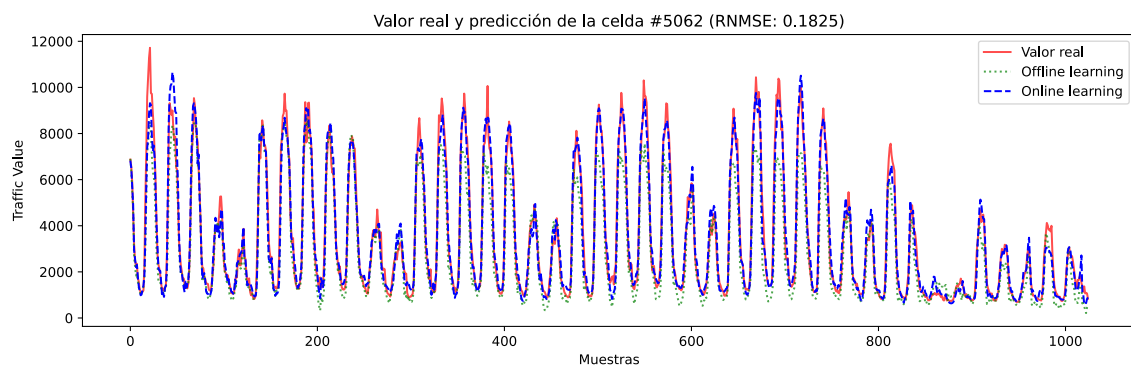


Figura 38: Predicciones del mejor modelo para la celda 5062. Elaboración propia.

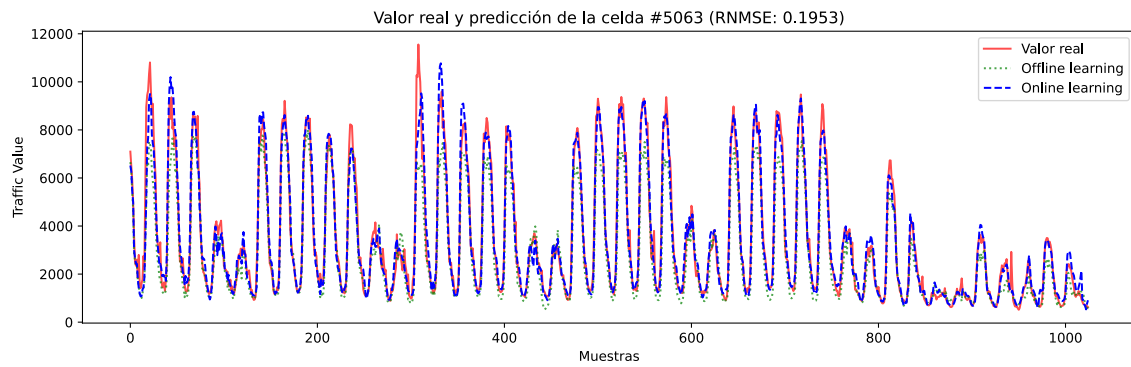


Figura 39: Predicciones del mejor modelo para la celda 5063. Elaboración propia.

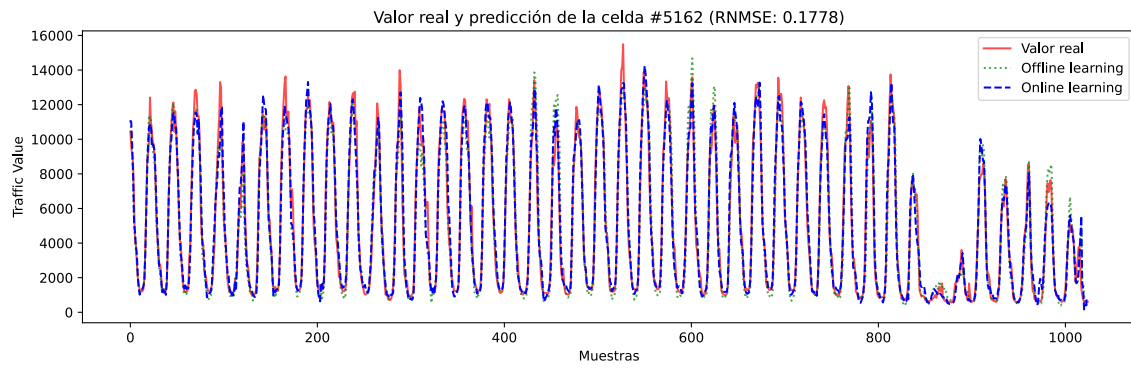


Figura 40: Predicciones del mejor modelo para la celda 5162. Elaboración propia.

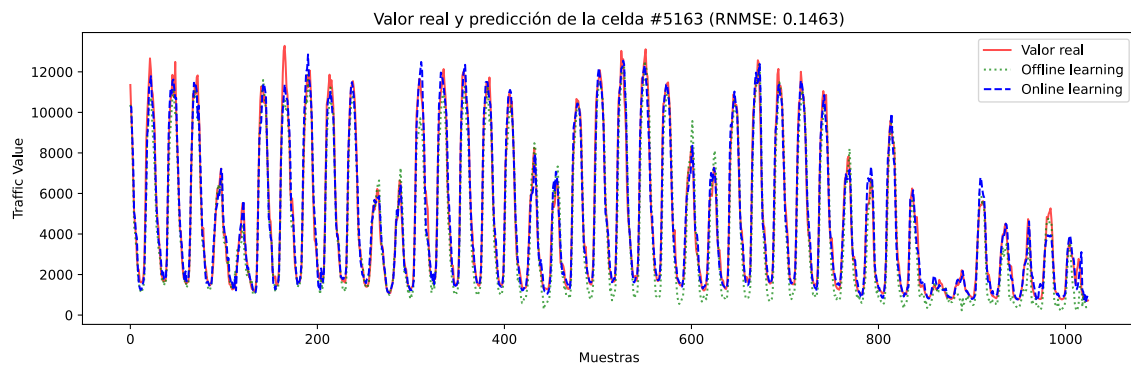


Figura 41: Predicciones del mejor modelo para la celda 5163. Elaboración propia.

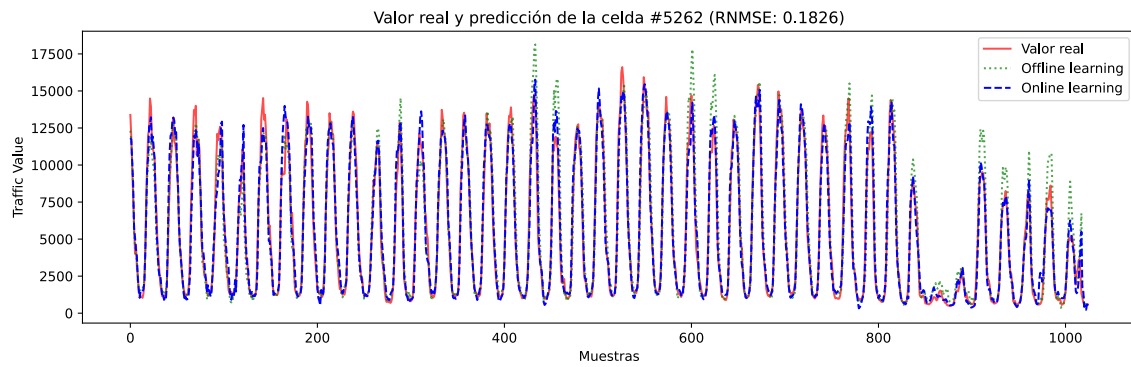


Figura 42: Predicciones del mejor modelo para la celda 5262. Elaboración propia.

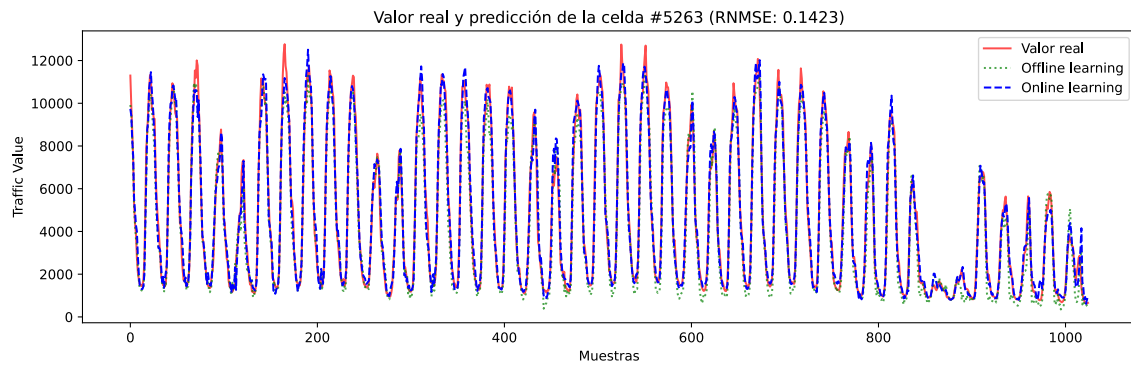


Figura 43: Predicciones del mejor modelo para la celda 5263. Elaboración propia.

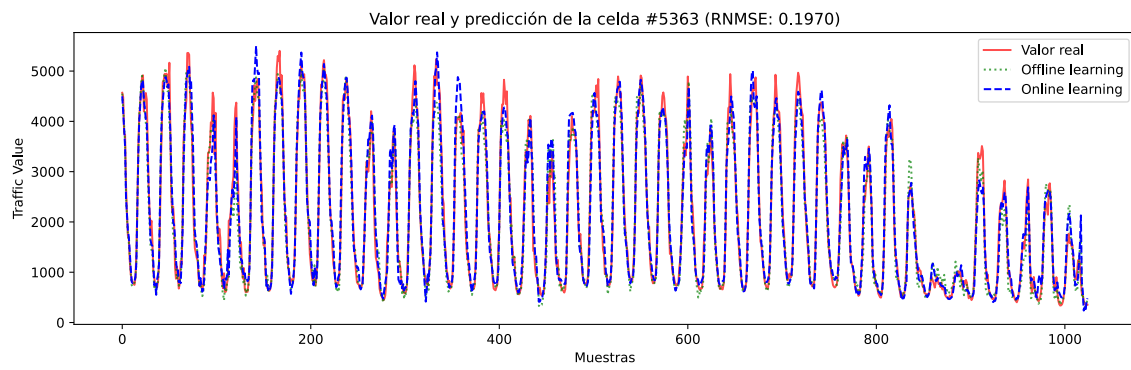


Figura 44: Predicciones del mejor modelo para la celda 5357. Elaboración propia.

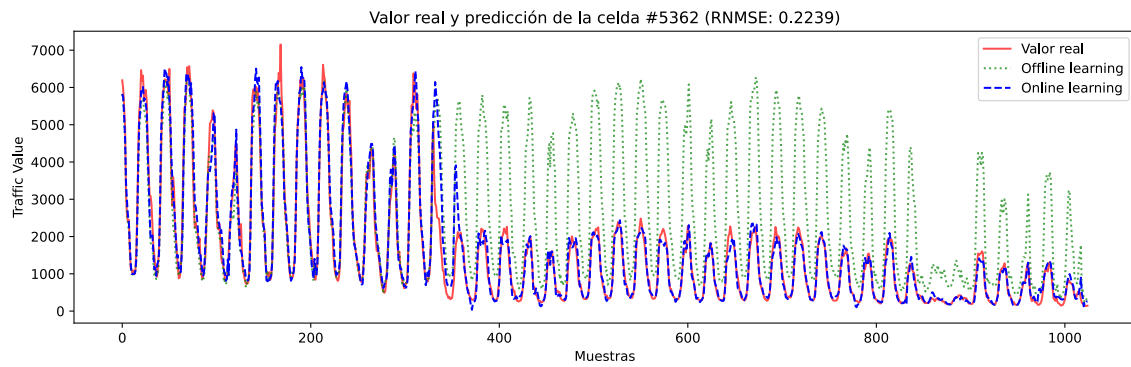


Figura 45: Predicciones del mejor modelo para la celda 5362. Elaboración propia.

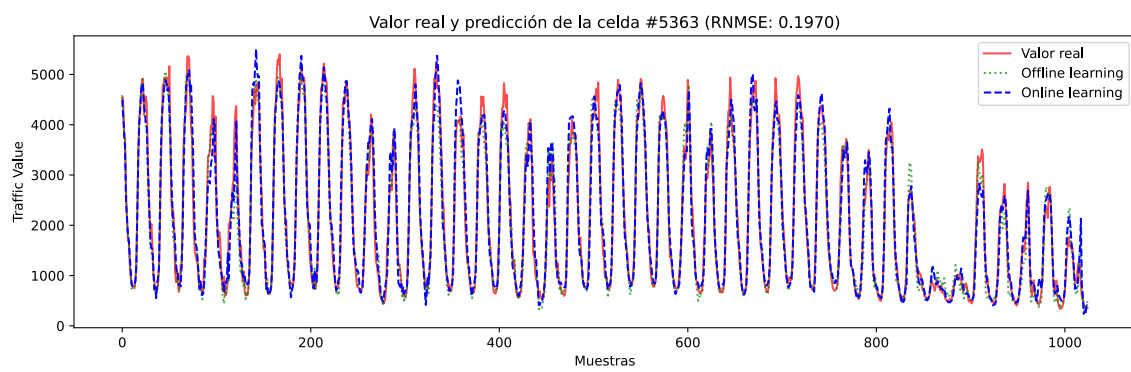


Figura 46: Predicciones del mejor modelo para la celda 5363. Elaboración propia.